

ACM/ICPC 算法模板

The Three Stooges

2026 年 7 月 27 日

目录

1 不要忘记	1	3 图论	11
1.1 土制 cph(仅 linux)	1	3.1 最短路: bitset 优化传递闭包	11
1.2 土制 cph(全平台)	1	3.2 最短路: 有向图上找最小环	11
1.3 随机数	1	3.3 最短路: spfa 判断负环	11
1.4 对拍	2	3.4 最小生成树	12
1.5 解除 python 大数限制	2	3.5 拓扑排序	12
1.6 常用函数	2	3.6 LCA: 倍增 $O(\log N)$ 求 LCA	12
1.7 高精度	2	3.7 LCA: 欧拉序 + ST 表 $O(1)$ 求 LCA	12
1.8 vector 相关	2	3.8 树的直径与中心	12
1.9 整数域三分	2	3.9 点分治与树的重心	12
1.10 运行时间	2	3.10 基环树	13
1.11 运行代码	2	3.11 Hierholzer 求欧拉路	13
1.12 记得快读快输	2	3.12 DSU on tree	13
1.13 !!! 数据范围	2	3.13 tarjan: 强连通分量 (SCC)	14
1.14 __int128 的使用	2	3.14 tarjan: 割边与边双连通分量 (EBCC)	14
1.15 快读模板	3	3.15 tarjan: 割点与点双连通分量 (PBCC)	14
1.16 随机数生成器	3	3.16 树哈希	15
1.17 随机图	3	3.17 最大流	15
1.18 对拍器	3	3.18 最近公共祖先 (LCA)	15
2 数据结构	3	3.19 树链剖分	16
2.1 对顶 multiset	3	3.20 树上倍增	17
2.2 带权并查集	4	3.21 最大路匹配	17
2.3 Trie	4	3.22 Dijkstra 算法	17
2.4 ST 表	4	3.23 分层图最短路	18
2.5 树状数组	4	3.24 双向广搜	18
2.6 树状数组: 二维树状数组	5	3.25 Bellman-Ford	19
2.7 线段树	5	3.26 tarjan(强连通分量)	19
2.8 线段树: 懒标记线段树	5	3.27 边双连通分量	19
2.9 逆序对	6	3.28 割边	20
2.10 笛卡尔树	6	3.29 点双连通分量	20
2.11 可持久化: 可持久化线段树	6	4 数学	20
2.12 可持久化: 可持久化 Trie	6	4.1 线性筛	20
2.13 线段树模板	7	4.2 因数	20
2.14 主席树 (可持久化线段树)	7	4.3 快速幂与乘法逆元	20
2.15 开点线段树	8	4.4 排列组合数	20
2.16 权值线段树	8	4.5 Lucas	20
2.17 线段树上最值操作	9	4.6 卡特兰数	20
2.18 线段树上序列操作	9	4.7 数论分块	20
2.19 非经典线段树	10	4.8 高斯消元	21
2.20 左偏树	11	4.9 矩阵快速幂	21
		4.10 自适应辛普森法	21
		4.11 多项式: FFT	21
		4.12 多项式: NTT	22
		4.13 快速幂	22
		4.14 乘法逆元	22
		4.15 组合数求法	23

4.16 线性基	23
4.17 矩阵快速幂及高斯消元	23
5 字符串	24
5.1 前缀函数与 KMP	24
5.2 Manacher	24
5.3 字符串哈希	24
5.4 AC 自动机	24
5.5 后缀自动机 SAM	25
5.6 KMP 算法	26
5.7 Trie 树	26
5.8 string 常用函数	26
6 动态规划	27
6.1 数位 DP	27
6.2 单调队列优化（滑动窗口）	27
6.3 无向图上回路计数	27
6.4 SOSDP	27
6.5 树形 DP	28
6.6 最长不下降子序列	28
7 计算几何	29
7.1 向量	29
7.2 点线操作	29
7.3 多边形与凸包	29
7.4 计算几何模板	30
8 其它	35
8.1 矩阵加速	35
8.2 并查集	36
8.3 FFT（快速傅里叶变换）	36
8.4 NTT（数论变换）	37
8.5 扫描线算法	37
8.6 最长上升子序列	38
8.7 全排列输出	38
8.8 位运算用法	38
8.9 Map 的使用	39
8.10 优先队列（堆）	39
8.11 扫描线 + 线段树（矩形面积并）	39
8.12 建树参考	39
8.13 Hash 树	40

1 不要忘记

1.1 土制 cph(仅 linux)

```

1 from os import system as e, listdir as l
2 from sys import argv
3 _, q, f = argv # q: 题号, f: 文件名
4
5 # r: 运行命令
6 # 跑 py 需要 `python3 cph.py A A.py`
7 if "py" in f:
8     r = f"python3 {f}"
9 else:
10     e(f"g++ -std=c++23 -o2 -Wall {f}.cpp -o {f}")
11     r = "./" + f
12
13 #
14 d = "samples-" + q.capitalize()
15 for i in l(d):
16     if not "in" in i: continue
17     print(i)
18     p = d + "/" + i[:-2]
19     if e(f"timeout 2 {r}<{p}in>{p}out"): print("TLE or RE")
20     else:
21         with open(f"{p}out") as o, open(f"{p}ans") as a:
22             print("AC" if o.read().split()==a.read().split() else "
                WA")

```

1.2 土制 cph(全平台)

```

1 from os import listdir as l, system as e
2 from os.path import join as jp
3 from sys import argv as a
4 import subprocess as s
5 _,q,f = a
6 if "py" in f:
7     cmd = ["python3", f]
8 else:
9     e(f"g++ -std=gnu++20 -O2 -Wall {f}.cpp -o {f}")
10    cmd = jp(".", f"{f}.exe")
11
12 d='samples-'+q.capitalize()
13 g,r,p,n='\033[32m','\033[31m','\033[35m','\033[0m'
14 for i in l(d):
15     if not 'in' in i: continue
16     t=jp(d,i[:-2])
17     print(i,'测评结果 ',end='',flush=1)
18     try:
19         k = s.run(cmd, timeout=2, stdin=open(f"{t}in"),
20                   stdout=open(f"{t}out", 'w'), stderr=s.PIPE,
21                   text=1)
22         c = lambda o,a : o.read().split() == a.read().split()
23         # c = lambda o,a : all(abs(float(x)-float(y))/max(1,
24                                abs(float(y)))<=1e-4
25                                for x,y in zip(o.read().split(),a.read
26                                                ().split()))
27         print(f'{g}AC{n}' if c(open(f"{t}out"),open(f"{t}ans"))
28               ) else f'{r}WA{n}')
29         if k.stderr:print(f'{r}{k.stderr}{n}')
30     except s.TimeoutExpired as k:
31         print(f'{p}TLE or RE{n}')
32     if k.stderr: print(f'{r}{k.stderr}{n}')

```

1.3 随机数

```

1 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().
    count());
2 int Rand(int x, int y) {
3     return uniform_int_distribution<int>(x, y)(rng);
4 }
5 shuffle(a.begin(), a.end(), rng);

```

1.4 对拍

```

1 // checker.cpp
2 while (1) {
3     system("A_gen > _data.in"); // Linux 下换成 ./xxx
4     system("A_bf < _data.in > A.ans");
5     system("A < _data.in > A.ans");
6     if (system("fc A.out A.ans")) { // Linux 下换成 diff
7         cout << "WA\n"; return 0;

```

```

8 } else {
9     cout << "AC\n";
10 }
11 }

```

1.5 解除 python 大数限制

```

1 import sys
2 sys.set_int_max_str_digits(100005)

```

1.6 常用函数

```

1 // 快读
2 inline int read()
3 {
4     int x=0,f=1;char ch=getchar();
5     while (ch<'0' || ch>'9'){if (ch=='-') f=-1;ch=getchar();}
6     while (ch>='0' && ch<='9'){x=x*10+ch-48;ch=getchar();}
7     return x*f;
8 }
9
10 // 取模
11 const int MOD=998244353;
12 ll add(ll x, ll y) { return (x+y)%MOD; }
13 ll del(ll x, ll y) { return add(x, MOD-y); }
14 ll mul(ll x, ll y) { return (x*y)%MOD; }
15 ll qpow(ll a, ll b = MOD - 2) {
16     ll res = 1;
17     for (; b; b >>= 1) {
18         if (b & 1) { res = mul(res, a); }
19         a = mul(a, a);
20     }
21     return res;
22 }
23
24 // 计算一个整数的二进制表示中有多少个 1
25 __builtin_popcountll(i);
26 std::popcount((unsigned long long)i); // c++20
27
28 // 计算一个整数的二进制表示中最高位的 1
29 __lg(i); // i 最多 32 位

```

1.7 高精度

```

1 istream& operator>> (istream& is, __int128 &x)
2 {
3     string s; is>>s;
4     bool f=false;
5     __int128 ans = 0;
6     for (auto c: s) {
7         if (c == '-') { f = true; }
8         if (isdigit(c)) { ans = ans * 10 + (c-'0'); }
9     }
10    x = f? -ans: ans;
11    return is;
12 }
13 ostream& operator<< (ostream& os, __int128 &x)
14 {
15     string ans;
16     function<void(__int128)> write = [&](__int128 x){
17         if (x < 0) { ans += '-'; x=-x; }
18         if (x > 9) { write(x/10); }
19         ans += '0'+x%10;
20     };
21     write(x); return os << ans;
22 }

```

1.8 vector 相关

```

1 // 多维 vector, 需要 c++17, c++14 只能老老实实写 vector<vector
2 <int>>
3 vector a(n+1, vector (n+1, vector<int>(n+1)));
4
5 // 最值
6 int mx = *min_element(vec.begin(), vec.end(), cmp);
7 int mn = *min_element(vec.begin(), vec.end(), cmp);
8
9 // 去重与离散化
10 for (int i = 1; i <= n; ++i) { vec.push_back(a[i]); }
11 sort(vec.begin(), vec.end());

```

```

11 vec.erase(unique(vec.begin(), vec.end()), vec.end());
12 for (int i = 1; i <= n; ++i) {
13     idx[i]=lower_bound(vec.begin(), vec.end(), a[i])-vec.begin()
14         +1; // 下标从1开始
15 }
16 // 前缀和
17 vector <ll> a(n), s(n);
18 partial_sum(a.begin(), a.end(), s.begin());
19 ll sum = accumulate(a.begin(), a.end(), 0LL);
20
21 // 填充 1~n, 令 a[i]=i
22 iota(a.begin(), a.end(), 0);

```

1.9 整数域三分

```

1 // 整数域三分求单峰函数最大值
2 int l, r;
3 while (l + 2 <= r) {
4     int w = (r - l) / 3;
5     int m1 = l + w; int v1 = f(m1);
6     int m2 = r - w; int v2 = f(m2);
7     if (v1 <= v2) {
8         l = m1 + 1;
9     } else {
10        r = m2 - 1;
11    }
12 }
13 cout << max(f(l), f(r)) << "\n";

```

1.10 运行时间

```

1 auto start = chrono::high_resolution_clock::now();
2 auto stop = chrono::high_resolution_clock::now();
3 auto duration = std::chrono::duration_cast<std::chrono::
4     milliseconds>(stop - start).count();
5 if (duration > 998) { cout << ans << "\n"; exit(0); }

```

1.11 运行代码

```

1 g++ A.cpp -o A
2 A

```

1.12 记得快读快输

```

1 ios::sync_with_stdio(0);
2 cin.tie(0), cout.tie(0);

```

1.13 !!! 数据范围

注意爆int

long long: 8字节, -9,223,372,036,854,775,808到9,223,372,036,854,775,807

unsigned long long: 8字节, 0到18,446,744,073,709,551,615

如果爆**long long** 就用__int128,但是注意输入输出

尽量不要使用**double**输入, 容易超时

1.14 __int128 的使用

```

1 #define int __int128
2 inline void read(int &n){
3     int x=0,f=1;
4     char ch=getchar();
5     while(ch<'0' || ch>'9'){
6         if(ch=='-') f=-1;
7         ch=getchar();
8     }
9     while(ch>='0' && ch<='9'){
10        x=(x<<1)+(x<<3)+(ch^48);
11        ch=getchar();
12    }
13    n=x*f;
14 }
15 inline void print(int n){
16     if(n<0){
17         putchar('-');

```

```

18     n*=-1;
19 }
20 if(n>9) print(n/10);
21 putchar(n % 10 + '0');
22 }
23 #undef int

```

1.15 快速模板

```

1 #define re register
2 #define il inline
3 il int read()
4 {
5     re int x=0,f=1;char c=getchar();
6     while(c<'0' || c>'9'){if(c=='-') f=-1;c=getchar();}
7     while(c>='0' && c<='9') x=(x<<3)+(x<<1)+(c^48),c=getchar();
8     return x*f;
9 }

```

1.16 随机数生成器

```

1 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().
2     count());
3 int Rand(int x, int y) {
4     return uniform_int_distribution<int>(x, y)(rng);
5 }
6 shuffle(a.begin(), a.end(), rng);

```

1.17 随机图

```

1 // n 为节点个数
2 // m 为边的个数，默认为 -1，表示建树
3 // root 为根节点编号，默认为 -1，
4 vector<array<int,2>> Graph(int n, int m=-1, int root=-1)
5 {
6     assert(m== -1 || (n-1<=m && m<=1LL*n*(n-1)/2));
7     assert(root== -1 || (1<=root && root<=n));
8     vector<array<int,2>> E;
9     set<array<int,2>> diff;
10    // 先建一棵以 0 为根的树，然后选根偏移
11    for (int i = 1; i < n; ++i) {
12        E.push_back( { i, Rand(0, i-1) } );
13    }
14    if (root == -1) { root = Rand(0, n-1); }
15    for (auto& [x, y]: E) {
16        x = (x+root) % n + 1;
17        y = (y+root) % n + 1;
18        diff.insert( { x, y } );
19        diff.insert( { y, x } ); // 无向图中需要添加这条反向边
20    }
21    // 从树建图
22    if (m != -1) {
23        for (int i = n-1; i < m; ++i) {
24            while (1) {
25                int x = Rand(1, n);
26                int y = Rand(1, n);
27                if (x == y || diff.count({x,y})) { continue; }
28                E.push_back( { x, y } );
29                diff.insert( { x, y } );
30                diff.insert( { y, x } ); // 无向图中需要添加这条反向边
31                break;
32            }
33        }
34    }
35    shuffle(E.begin(), E.end(), rng);
36    return E;
37 }

```

1.18 对拍器

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 int main(){
4     int t=0;
5     while(true){
6         cout<<"test: "<<t++<<endl;
7         system("data.exe>data.in");//data.exe 随机数生成器
8         system("std.exe<data.in>data.out");//std.exe 标准程序
9         system("solve.exe<data.in>solve.out");//solve.exe 暴力求解

```

算法

```

10 if(system("fc std.out solve.out>diff.log")){
11     cout<<"WA"<<endl;
12     break;
13 }
14 }
15 cout<<"AC\n";
16 }
17 /*
18 // checker.cpp
19 while (1) {
20     system("A_gen > _data.in"); // Linux 下换成 ./xxx
21     system("A_bf < _data.in > A.ans");
22     system("A < _data.in > A.ans");
23     if (system("fc A.out A.ans")) { // Linux 下换成 diff
24         cout << "WA\n"; return 0;
25     } else {
26         cout << "AC\n";
27     }
28 }*/

```

2 数据结构

2.1 对顶 multiset

```

1 struct PairingMultiset {
2     int sz;
3     multiset<int> minH, maxH;
4     ll minS, maxS;
5     void clear()
6     {
7         minH.clear(); maxH.clear();
8         sz = minS = maxS = 0;
9     }
10    void move()
11    {
12        while ((int)minH.size() < (sz+1)/2) {
13            minS += *maxH.rbegin();
14            minH.insert(*maxH.rbegin());
15            maxS -= *maxH.rbegin();
16            maxH.erase(maxH.find(*maxH.rbegin()));
17        }
18        while ((int)minH.size() > (sz+1)/2) {
19            maxS += *minH.begin();
20            maxH.insert(*minH.begin());
21            minS -= *minH.begin();
22            minH.erase(minH.find(*minH.begin()));
23        }
24    }
25    void add(int x)
26    { // 向对顶堆中插入一个元素
27        ++sz;
28        if (minH.empty() || x >= *minH.begin()) {
29            minS += x;
30            minH.insert(x);
31        } else {
32            maxS += x;
33            maxH.insert(x);
34        }
35        move();
36    }
37    void del(int x)
38    { // 从对顶堆中删除一个x
39        --sz;
40        if (x >= *minH.begin()) {
41            minS -= x;
42            minH.erase(minH.find(x));
43        } else {
44            maxS -= x;
45            maxH.erase(maxH.find(x));
46        }
47        move();
48    }
49    ll med()
50    { // 求中位数
51        return *minH.begin();
52    }
53    ll dis_to_med()
54    { // 求对顶堆中的数字到中位数的距离和
55        ll minSZ=(sz+1)/2, maxSZ=sz-minSZ, m=med();
56        return (minS-minSZ*m)+(maxSZ*m-maxS);
57    }
58 } p;

```

2.2 带权并查集

```
1 struct DSU {
2     int n;
3     vector<int> f, siz, pre;
4     DSU(int n): n(n),
5         f(vector<int>(n+1)),
6         siz(vector<int>(n+1,1)),
7         pre(vector<int>(n+1))
8     {
9         iota(f.begin(), f.end(), 0);
10    }
11    int find(int x) {
12        if (f[x] == x) { return x; }
13        int fa = find(f[x]);
14        pre[x] += pre[f[x]];
15        return f[x]=fa;
16    }
17    bool same(int x, int y) {
18        return find(x) == find(y);
19    }
20    void merge(int x, int y) {
21        x = find(x);
22        y = find(y);
23        f[x] = y;
24        pre[x] += siz[y];
25        siz[y] += siz[x];
26        siz[x] = 0;
27    }
28    int query(int x, int y) {
29        return abs(pre[x]-pre[y])-1;
30    }
31};
```

2.3 Trie

```
1 const string VAL="
2     abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
3     ";
4 const int S=3e6+5, V=62;
5
6 struct Trie {
7     unordered_map<char,int> p;
8     int tot, ch[S][V], pass[S];
9     int newNode() {
10         ++tot;
11         memset(ch[tot], 0, sizeof(ch[tot]));
12         pass[tot] = 0;
13         return tot;
14     }
15     void init() {
16         tot = -1;
17         newNode();
18     }
19     Trie() {
20         for (int i = 0; i < V; ++i) { p[VAL[i]] = i; }
21         init();
22     }
23     void insert(string s) {
24         int now = 0;
25         for (auto c: s) {
26             if (!ch[now][p[c]]) { ch[now][p[c]] = newNode(); }
27             now = ch[now][p[c]];
28             ++pass[now];
29         }
30     }
31     int query(string s) {
32         int now = 0;
33         for (auto c: s) {
34             if (!ch[now][p[c]]) { return 0; }
35             now = ch[now][p[c]];
36         }
37         return pass[now];
38     }
39 } trie;
```

2.4 ST表

```
1 template <typename T>
2 struct ST {
3     int n=0, l=0;
4     vector<int> Log;
```

```
5     vector<vector<T>> st;
6     ST() { }
7     ST(const vector<T>& a): n(a.size()), Log(vector<int>(n+1))
8     {
9         for (int i = 2; i <= n; ++i) {
10             l=Log[i]=Log[i/2]+1;
11         }
12         st.assign(l+1, vector<T>(n));
13         copy(a.begin(), a.end(), st[0].begin());
14         for (int i = 1; i <= l; ++i) {
15             for (int j = 0; j+(1<<(i-1)) < n; ++j) {
16                 st[i][j] = max( st[i-1][j], st[i-1][j+(1<<(i-1))] );
17             }
18         }
19     }
20     T query(int l, int r) {
21         assert(l <= r); // l > r 时的返回值需要特殊定义
22         int s = Log[r-l+1];
23         return max( st[s][l], st[s][r-(1<<s)+1] );
24     }
25     int find(int l, T x) { // 第一个区间 [l,r] 最大值大于等于 x
26         // 的 r
27         if (l >= n) { return -1; }
28         int rl=l-1, rr=n;
29         while (rl != rr-1) {
30             int mid = (rl + rr) >> 1;
31             if (query(l, mid) >= x) { rr = mid; }
32             else { rl = mid; }
33         }
34         if (rr < n) { return rr; }
35         return -1;
36     }
37     int find(int l, int x) { // 倍增写法
38         assert(l < n);
39         if (st[0][l] < x) { return l; }
40         if (query(l, n-1) >= x) { return n; }
41         int ans = l, g = st[0][l];
42         for (int i = l; i >= 0; --i) {
43             if (ans+(1<<i) < n && gcd(g, st[i][ans]) >= x) {
44                 g = max(g, st[i][ans]);
45                 ans += 1 << i;
46             }
47         }
48         return ans;
49     }
50};
```

2.5 树状数组

```
1 struct BIT {
2     int n;
3     vector<int> t;
4     BIT(int n): n(n), t(n+1) { } // 注意值域树状数组中 n=tot
5     int lowbit(int x) { return x&-x; }
6     void modify(int x, int d) {
7         for (; x <= n; x += lowbit(x)) { t[x] += d; }
8     }
9     // 前缀和
10    int query(int x) {
11        int res = 0;
12        for (; x; x -= lowbit(x)) { res += t[x]; }
13        return res;
14    }
15    // 求最小的前缀和等于x的位置
16    int select(int x) {
17        int ans = 0, sum = 0;
18        for (int i = __lg(n); i >= 0; --i) {
19            if (ans+(1<<i) <= n && sum+t[ans+(1<<i)] < x) {
20                ans += 1<<i;
21                sum += t[ans];
22            }
23        }
24        return ans + 1;
25    }
26};
27 BIT T(n);
```

2.6 树状数组: 二维树状数组

```
1 struct BIT {
2     int n, m;
```

```

3 vector<vector<int>> t;
4 BIT(int n, int m): n(n), m(m), t(n+1, vector<int>(m+1)) { }
5 int lowbit(int x) { return x & -x; }
6 void modify(int x, int y, int d) {
7     for (int i = x; i <= n; i += lowbit(i)) {
8         for (int j = y; j <= m; j += lowbit(j)) {
9             t[i][j] += d;
10        }
11    }
12 }
13 int query(int x, int y) {
14     int res = 0;
15     for (int i = x; i; i -= lowbit(i)) {
16         for (int j = y; j; j -= lowbit(j)) {
17             res += t[i][j];
18         }
19     }
20     return res;
21 }
22 };

```

2.7 线段树

```

1 #define lson (p << 1)
2 #define rson (p << 1 | 1)
3 #define m ((l+r)>>1)
4 template<typename Info>
5 struct segmentTree {
6     int n;
7     vector<Info> a;
8
9     void push_up(int p) {
10         a[p] = a[lson] + a[rson];
11     }
12
13     template<typename F>
14     Info dfs(int p, int l, int r, int x, int y, F&& op) {
15         if (y <= l || r <= x) { return Info(); }
16         if (x <= l && r <= y) { op(p,l,r); return a[p]; }
17         Info res;
18         res = res + dfs(lson, l, m, x, y, op);
19         res = res + dfs(rson, m, r, x, y, op);
20         push_up(p);
21         return res;
22     }
23
24     template<typename F>
25     pair<int,Info> findFirst(int p, int l, int r, int x, int y,
26                             F&& pred) {
27         if (y <= l || r <= x) { return {-1,Info()}; }
28         if (x <= l && r <= y && !pred(a[p])) { return {-1,Info()}; }
29         if (l == r-1) { return {l,a[p]}; }
30         pair<int,Info> res = findFirst(lson, l, m, x, y, pred);
31         if (res.first == -1) {
32             res = findFirst(rson, m, r, x, y, pred);
33         }
34         return res;
35     }
36
37     template<typename F>
38     pair<int,Info> findLast(int p, int l, int r, int x, int y, F
39                             && pred) {
40         if (y <= l || r <= x) { return {-1,Info()}; }
41         if (x <= l && r <= y && !pred(a[p])) { return {-1,Info()}; }
42         if (l == r-1) { return {l,a[p]}; }
43         pair<int,Info> res = findLast(lson, m, r, x, y, pred);
44         if (res.first == -1) {
45             res = findLast(rson, l, m, x, y, pred);
46         }
47         return res;
48     }
49
50     segmentTree(int n = 0): n(n), a(4 << __lg(n)) { }
51     segmentTree(vector<Info>&& b): segmentTree(b.size()) {
52         function<void(int,int,int)> build = [&](int p, int l,
53             int r) -> void {
54             if (r - l == 1) { a[p] = Info(b[l]); return; }
55             build(lson, l, m);
56             build(rson, m, r);
57             push_up(p);
58         };
59     };

```

```

56     build(1, 0, n);
57 }
58
59 void modify(int pos, int x) {
60     dfs(1, 0, n, pos, pos+1, [&](int p, int l, int r){ a[p]
61         .mx = x; });
62 }
63
64 int query(int l, int r) {
65     return dfs(1, 0, n, l, r+1, [&](int,int,int){}).mx;
66 }
67
68 void Debug(int p, int l, int r) {
69     cerr << format("a[{}] = [ {}, {}, {} ]\n", p, l, r, a[
70         p].mx);
71     if (l == r - 1) { return; }
72     Debug(lson, l, m);
73     Debug(rson, m, r);
74 }
75
76 void Debug() { Debug(1, 0, n); }
77
78 #undef lson
79 #undef rson
80 #undef m
81 struct Info {
82     int mx;
83     Info(int mx = 0): mx(mx) { }
84     friend Info operator+ (Info u, Info v) {
85         return Info( max(u.mx, v.mx) );
86     }
87 };

```

2.8 线段树: 懒标记线段树

```

1 #define lson (p << 1)
2 #define rson (p << 1 | 1)
3 #define m ((l+r)>>1)
4 template<typename Info, typename Lazy>
5 struct segmentTree {
6     int n;
7     vector<Info> a;
8     vector<Lazy> t;
9
10     segmentTree(int n): n(n), a(4 << __lg(n)), t(4 << __lg(n))
11         { }
12     segmentTree(vector<Info>&& vec): segmentTree(vec.size()) {
13         function<void(int,int,int)> build = [&](int p, int l,
14             int r) -> void {
15             if (r - l == 1) { a[p] = vec[l]; return; }
16             build(lson, l, m);
17             build(rson, m, r);
18             push_up(p);
19         };
20         build(1, 0, n);
21     }
22
23     void push_up(int p) {
24         a[p] = a[lson] + a[rson];
25     }
26
27     void apply(int p, Lazy v) {
28         a[p] = a[p] + v;
29         t[p] = t[p] + v;
30     }
31
32     void spread_down(int p) {
33         apply(lson, t[p]);
34         apply(rson, t[p]);
35         t[p] = Lazy();
36     }
37
38     template<typename F>
39     Info dfs(int p, int l, int r, int x, int y, F&& op) {
40         if (y <= l || r <= x) { return Info(); }
41         if (x <= l && r <= y) { op(p,l,r); return a[p]; }
42         spread_down(p);
43         Info res;
44         res = res + dfs(lson, l, m, x, y, op);
45         res = res + dfs(rson, m, r, x, y, op);
46         push_up(p);
47         return res;
48     }

```

```

48 template<typename F>
49 pair<int,Info> findFirst(int p, int l, int r, int x, int y
    , F&& pred) {
50     if (y <= l || r <= x) { return {-1,Info()}; }
51     if (x <= l && r <= y && !pred(a[p])) { return {-1,Info
        ()}; }
52     if (l == r-1) { return {l,a[p]}; }
53     spread_down(p);
54     pair<int,Info> res = findFirst(lson, l, m, x, y, pred)
        ;
55     if (res.first == -1) {
56         res = findFirst(rson, m, r, x, y, pred);
57     }
58     return res;
59 }
60
61 void modify(int x, int y, Lazy v) {
62     dfs(1, 0, n, x, y+1, [&](int p, int l, int r){ apply(p
        , v); });
63 }
64
65 Info query(int x, int y) {
66     return dfs(1, 0, n, x, y+1, [](int,int,int){});
67 }
68 };
69 #undef lson
70 #undef rson
71 #undef m
72 struct Info {
73     int len;
74     ll s;
75     Info(ll s=0, int len=1): s(s), len(len) { }
76 };
77 struct Lazy {
78     ll a, m;
79     Lazy(ll a=0, ll m=1): a(a), m(m) { }
80 };
81 Info operator+ (Info u, Info v) {
82     return Info(add(u.s, v.s), add(u.len,v.len));
83 }
84 Info operator+ (Info u, Lazy v) {
85     return Info(add(mul(u.s, v.m), mul(v.a, u.len)), u.len);
86 }
87 Lazy operator+ (Lazy u, Lazy v) {
88     return Lazy(add(mul(u.a, v.m), v.a), mul(u.m, v.m));
89 }

```

2.9 逆序对

```

1 ll msort(vector<int> &a, int l, int r)
2 {
3     if (l == r) { return 0; }
4     int mid = (l + r) >> 1;
5     ll ans=0;
6     ans += msort(a, l, mid);
7     ans += msort(a, mid+1, r);
8     int i=l, j=mid+1;
9     vector<int> b;
10    while (i <= mid && j <= r) {
11        if (a[i] <= a[j]) { b.push_back(a[i++]); }
12        else { b.push_back(a[j++]); ans+=mid-i+1;
            }
13    }
14    while (i <= mid) { b.push_back(a[i++]); }
15    while (j <= r) { b.push_back(a[j++]); }
16    copy(b.begin(), b.end(), a.begin()+l);
17    return ans;
18 }

```

2.10 笛卡尔树

```

1 vector<int> stk;
2 for (int i = 1; i <= n; ++i) {
3     int flag = 0;
4     while (!stk.empty()) {
5         if (a[stk.back()] > a[i]) { // 小根堆
6             flag = stk.back(); stk.pop_back();
7         } else {
8             break;
9         }
10    }
11    if (flag) { L[i] = flag; }

```

```

12     if (!stk.empty()) { R[stk.back()] = i; }
13     stk.push_back(i);
14 }
15 function<void(int)> dfs = [&](int x){
16     cerr << "now node " << x << "\n";
17     cerr << ":: L = " << L[x] << "\n";
18     cerr << ":: R = " << R[x] << "\n";
19     if (L[x]) dfs(L[x]);
20     if (R[x]) dfs(R[x]);
21 };
22 dfs(stk[0]);

```

2.11 可持久化: 可持久化线段树

```

1 5 10
2 59 46 14 87 41
3 0 2 1
4 0 1 1 14
5 0 1 1 57
6 0 1 1 88
7 4 2 4
8 0 2 5
9 0 2 4
10 4 2 1
11 2 2 2
12 1 1 5 91
13 ---
14 59
15 87
16 41
17 87
18 88
19 46

```

2.12 可持久化: 可持久化 Trie

```

1 const int N=2e7+9;
2 struct Trie {
3     int ch[N][2];
4     int root[N], cnt[N], top=0, siz=0;
5     void insert(int x) {
6         root[++top] = ++siz;
7         int rt1 = root[top-1], rt2 = root[top];
8         for (int i = 24; i >= 0; --i) {
9             int b = x >> i & 1;
10            ch[rt2][!b] = ch[rt1][!b];
11            ch[rt2][b] = ++siz;
12            rt1 = ch[rt1][b];
13            rt2 = ch[rt2][b];
14            cnt[rt2] = cnt[rt1] + 1;
15        }
16    }
17    int query(int l, int r, int x) {
18        int ans = 0;
19        int rt1 = root[l], rt2 = root[r];
20        for (int i = 24; i >= 0; --i) {
21            int b = x >> i & 1;
22            if (cnt[ch[rt2][!b]] > cnt[ch[rt1][!b]]) {
23                ans += (1 << i);
24                b = !b;
25            }
26            rt1 = ch[rt1][b];
27            rt2 = ch[rt2][b];
28        }
29        return ans;
30    }
31 } tr;
32
33 void solve()
34 {
35     int n, m;
36     cin >> n >> m;
37     tr.insert(0);
38     int sum = 0;
39     for (int i = 1; i <= n; ++i) {
40         int x; cin >> x;
41         tr.insert(sum ^ x);
42     }
43     for (int i = 1; i <= m; ++i) {
44         char op; cin >> op;
45         if (op == 'A') {
46             int x; cin >> x;

```

```

47         tr.insert(sum ^= x);
48     } else {
49         int l, r, x;
50         cin >> l >> r >> x;
51         cout << tr.query(l-1, r, x^sum) << "\n";
52     }
53 }
54 }

```

2.13 线段树模板

```

1 //区间乘法
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define maxn 1000010
5 #define ll long long
6 int p;
7 ll a[maxn], w[maxn*4];
8 ll lzy_add[maxn*4], lzy_mul[maxn*4];
9
10 void pushup(const int u) {
11     w[u] = (w[u<<1] + w[u<<1|1]) % p;
12 }
13
14 void build(const int u, int L, int R) {
15     lzy_mul[u] = 1;
16     lzy_add[u] = 0;
17     if (L == R) {
18         w[u] = a[L] % p;
19         return;
20     }
21     int M = (L+R)>>1;
22     build(u<<1, L, M);
23     build(u<<1|1, M+1, R);
24     pushup(u);
25 }
26
27 bool InRange(int L, int R, int l, int r) {
28     return (l <= L) && (R <= r);
29 }
30
31 bool OutofRange(int L, int R, int l, int r) {
32     return (R < l) || (L > r);
33 }
34
35 void maketag(int u, int L, int R, ll x, int type) {
36     if (type == 1) {
37         (lzy_mul[u] *= x) %= p;
38         (lzy_add[u] *= x) %= p;
39         (w[u] *= x) %= p;
40     } else {
41         (lzy_add[u] += x) %= p;
42         (w[u] += (R-L+1) * x) %= p;
43     }
44 }
45
46 void pushdown(int u, int L, int R) {
47     int M = (L+R)>>1;
48     // 左子节点
49     maketag(u<<1, L, M, lzy_mul[u], 1);
50     maketag(u<<1, L, M, lzy_add[u], 2);
51     // 右子节点
52     maketag(u<<1|1, M+1, R, lzy_mul[u], 1);
53     maketag(u<<1|1, M+1, R, lzy_add[u], 2);
54     lzy_mul[u] = 1;
55     lzy_add[u] = 0;
56 }
57
58 ll query(int u, int L, int R, int l, int r) {
59     if (InRange(L, R, l, r)) {
60         return w[u];
61     } else if (!OutofRange(L, R, l, r)) {
62         int M = (L+R)>>1;
63         pushdown(u, L, R);
64         return (query(u<<1, L, M, l, r) + query(u<<1|1, M+1, R, l, r)) % p;
65     } else {
66         return 0;
67     }
68 }
69
70 void update(int u, int L, int R, int l, int r, ll x, int type)
71 {

```

```

71     if (InRange(L, R, l, r)) {
72         maketag(u, L, R, x, type);
73     } else if (!OutofRange(L, R, l, r)) {
74         int M = (L+R)>>1;
75         pushdown(u, L, R);
76         update(u<<1, L, M, l, r, x, type);
77         update(u<<1|1, M+1, R, l, r, x, type);
78         pushup(u);
79     }
80 }
81
82 int main() {
83     ios::sync_with_stdio(false);
84     cin.tie(nullptr);
85     ll n, m;
86     cin >> n >> m >> p;
87     for (int i = 1; i <= n; i++)
88         cin >> a[i];
89     build(1, 1, n);
90     while (m--) {
91         int op, x, y;
92         cin >> op >> x >> y;
93         if (op == 1 || op == 2) {
94             ll k;
95             cin >> k;
96             update(1, 1, n, x, y, k, op);
97         } else {
98             cout << query(1, 1, n, x, y) << endl;
99         }
100     }
101     return 0;
102 }

```

2.14 主席树 (可持久化线段树)

```

1 //对于单点历史修改与查询
2 #include<bits/stdc++.h>
3 using namespace std;
4 #define int long long
5 const int maxn=(1e6+1);
6 int top,root[maxn],arr[maxn],n,m,cnt=0;
7 struct kkk{
8     int l,r,val;
9 } a[maxn*30];
10 int build(int l,int r){
11     int rt=++cnt;
12     if(l==r)a[rt].val=arr[l];
13     else {
14         int mid=(l+r)>>1;
15         a[rt].l=build(l,mid);
16         a[rt].r=build(mid+1,r);
17     }
18     return rt;
19 }
20 int update(int ji,int jv,int l,int r,int i)//单点修改
21 {
22     int rt=++cnt;
23     a[rt].l=a[i].l;
24     a[rt].r=a[i].r;
25     if(l==r) a[rt].val=jv;
26     else {
27         int mid=(l+r)>>1;
28         if(ji<=mid)a[rt].l=update(ji,jv,l,mid,a[rt].l);
29         else a[rt].r=update(ji,jv,mid+1,r,a[rt].r);
30     }
31     return rt;
32 }
33 int query(int ji,int l,int r,int i){
34     if(l==r) return a[i].val;
35     int mid=(l+r)>>1;
36     if(ji<=mid) return query(ji,l,mid,a[i].l);
37     else return query(ji,mid+1,r,a[i].r);
38 }
39 signed main(){
40     ios_base::sync_with_stdio(false);
41     cin.tie(nullptr);
42     cin>>n>>m;
43     for(int i=1;i<=n;i++){
44         cin>>arr[i];
45     }
46     root[0]=build(1,n);
47     for(int i=1,version,op,x,v;i<=m;i++){
48         cin>>version>>op>>x;

```



```

49     if(op==1){
50         cin>>v;
51         root[i]=update(x,v,1,n,root[version]);
52     }else {
53         root[i]=root[version];
54         cout<<query(x,1,n,root[i])<<endl;
55     }
56 }
57 }

```

2.15 开点线段树

```

1 //适用于范围很大，但是操作较少
2 //用于减小空间复杂度
3 //如果操作过多，则与没有优化相同，因此可能被卡
4 //在线段树的合并与分裂中不可替代，树链剖分
5 #include<bits/stdc++.h>
6 using namespace std;
7 typedef long long ll;
8 const int maxn=100001;
9 ll le[maxn],ri[maxn],sum[maxn],add[maxn],cnt;
10 void up(ll i, ll l, ll r){
11     sum[i]=sum[l]+sum[r];
12 }
13 void lazy(ll i,ll v, ll n){
14     sum[i]+=v*n;
15     add[i]+=v;
16 }
17 void down(ll i, ll ln, ll rn){
18     if(add[i]!=0){
19         if(le[i]==0){
20             le[i]++;cnt;
21         }
22         if(ri[i]==0){
23             ri[i]++;cnt;
24         }
25         lazy(le[i],add[i],ln);
26         lazy(ri[i],add[i],rn);
27         add[i]=0;
28     }
29 }
30 void Add(ll jl,ll jr,ll v,ll l,ll r,ll i){
31     if(jl<=l&&r<=jr){
32         lazy(i,v,r-l+1);
33         return;
34     }
35     ll mid=(l+r)>>1;
36     down(i,mid-l+1,r-mid);
37     if(jl<=mid){
38         if(le[i]==0){
39             le[i]++;cnt;
40         }
41         Add(jl,jr,v,l,mid,le[i]);
42     }
43     if(jr>mid){
44         if(ri[i]==0){
45             ri[i]++;cnt;
46         }
47         Add(jl,jr,v,mid+1,r,ri[i]);
48     }
49     up(i,le[i],ri[i]);
50 }
51 }
52 ll query(ll jl,ll jr,ll l, ll r,ll i){
53     if(jl<=l&&r<=jr){
54         return sum[i];
55     }
56     ll mid=(l+r)>>1;
57     down(i,mid-l+1,r-mid);
58     ll ans=0;
59     if(jl<=mid){
60         if(le[i]!=0){
61             ans+=query(jl,jr,l,mid,le[i]);
62         }
63     }
64     if(jr>mid){
65         if(ri[i]!=0){
66             ans+=query(jl,jr,mid+1,r,ri[i]);
67         }
68     }
69     return ans;
70 }
71 }

```

```

72 int main(){
73     int n,m;
74     cin>>n>>m;
75     cnt=1;
76     while(m--){
77         int opt;
78         cin>>opt;
79         if(opt==1){
80             int l,r,v;
81             cin>>l>>r>>v;
82             Add(l,r,v,1,n,1);
83         }
84         else {
85             int l,r;
86             cin>>l>>r;
87             cout<<query(l,r,1,n,1)<<endl;
88         }
89     }
90 }

```

2.16 权值线段树

```

1 权值线段树模板
2 #define lson pos<<1
3 #define rson pos<<1|1
4 void build(int pos,int l,int r)
5 {
6     int mid=(l+r)>>1;
7     if(l==r)
8     {
9         tree[pos]=a[l];//a[l]表示数l有多少个
10        return;
11    }
12    build(lson,l,mid);
13    build(rson,mid+1,r);
14    tree[pos]=tree[lson]+tree[rson];
15 }
16 void update(int pos,int l,int r,int k,int cnt)//表示数k的个数
17    多cnt个
18 {
19     int mid=(l+r)>>1;
20     if(l==r)
21     {
22         tree[pos]+=cnt;
23         return;
24     }
25     if(k<=mid)
26         update(lson,l,mid,k,cnt);
27     else
28         update(rson,mid+1,r,k,cnt);
29     tree[pos]=tree[lson]+tree[rson];
30 }
31 int query(int pos,int l,int r,int k)//查询数k有多少个
32 {
33     int mid=(l+r)>>1;
34     if(l==r)
35         return tree[pos];
36     if(k<=mid)
37         return query(lson,l,mid,k);
38     else
39         return query(rson,mid+1,r,k);
40 }
41 int kth(int pos,int l,int r,int k)//查询第k大值是多少
42 {
43     int mid=(l+r)>>1;
44     if(l==r)
45         return l;
46     if(k<=mid)
47         return kth(lson,l,mid,k);
48     else
49         return kth(rson,mid+1,r,k-mid);
50 }

```

2.17 线段树上最值操作

```

1 //基本的实现
2 #include<bits/stdc++.h>
3 using namespace std;
4 const int maxn=1000001;
5 const int minest=-1000001;
6 long long arr[maxn<<2],sum[maxn<<2],mx[maxn<<2],sem[maxn<<2],
7     cnt[maxn<<2];

```

```

7 void up(int i){
8     int l=i<<1;
9     int r=i<<1|1;
10    sum[i]=sum[l]+sum[r];
11    mx[i]=max(mx[l],mx[r]);
12    if(mx[l]>mx[r]){
13        cnt[i]=cnt[l];
14        sem[i]=max(sem[l],mx[r]);
15    }else if(mx[r]>mx[l]){
16        cnt[i]=cnt[r];
17        sem[i]=max(sem[r],mx[l]);
18    }else {
19        sem[i]=max(sem[l],sem[r]);
20        cnt[i]=cnt[l]+cnt[r];
21    }
22 }
23 void lazy(int i,int v){
24     if(v<mx[i]){
25         sum[i]-=(mx[i]-v)*cnt[i];
26         mx[i]=v;
27     }
28 }
29 void down(int i){
30     lazy(i<<1,mx[i]);
31     lazy(i<<1|1,mx[i]);
32 }
33 void setmin(int jl,int jr,int v,int l,int r,int i){
34     if(v>mx[i]) return;
35     if(jl<=l&&r<=jr&&v<=sem[i]){
36         lazy(i,v);
37         return;
38     }else {
39         down(i);
40         int mid=(l+r)>>1;
41         if(jl<=mid) setmin(jl,jr,v,l,mid,i<<1);
42         if(jr>mid) setmin(jl,jr,v,mid+1,r,i<<1|1);
43     }
44     up(i);
45 }
46 long long querymax(int jl,int jr,int l,int r,int i){
47     if(jl<=l&&r<=jr){
48         return mx[i];
49     }
50     down(i);
51     int mid=(l+r)>>1;
52     long long lm=minest,rm=minest;
53     if(jl<=mid) lm=querymax(jl,jr,l,mid,i<<1);
54     if(jr>mid) rm=querymax(jl,jr,mid+1,r,i<<1|1);
55     return max(lm,rm);
56 }
57 long long querysum(int jl,int jr,int l,int r,int i){
58     if(jl<=l&&r<=jr){
59         return sum[i];
60     }
61     down(i);
62     int mid=(l+r)>>1;
63     long long ans=0;
64     if(jl<=mid) ans+=querysum(jl,jr,l,mid,i<<1);
65     if(jr>mid) ans+=querysum(jl,jr,mid+1,r,i<<1|1);
66     return ans;
67 }
68 void build(int l,int r,int i){
69     if(l==r){
70         sum[i]=arr[l];
71         mx[i]=arr[l];
72         sem[i]=minest;
73         cnt[i]=1;
74         return;
75     }
76     int mid=(l+r)>>1;
77     build(l,mid,i<<1),build(mid+1,r,i<<1|1);
78     up(i);
79 }
80 int main(){
81     ios_base::sync_with_stdio(false);
82     cin.tie(nullptr);
83     int t;
84     cin>>t;
85     while(t--){
86         int n,m;
87         cin>>n>>m;
88         for(int i=1;i<=n;i++){
89             cin>>arr[i];
90         }

```

```

91         build(1,n,1);
92         while(m--){
93             int opt;
94             cin>>opt;
95             if(opt==0){
96                 int l,r;
97                 long long mi;
98                 cin>>l>>r>>mi;
99                 setmin(1,r,mi,1,n,1);
100             }else if(opt==1){
101                 int l,r;
102                 cin>>l>>r;
103                 cout<<querymax(1,r,1,n,1)<<endl;
104             }else {
105                 int l,r;
106                 cin>>l>>r;
107                 cout<<querysum(1,r,1,n,1)<<endl;
108             }
109         }
110     }
111 }

```

2.18 线段树上序列操作

```

1 //线段树上的序列操作，稍难一点
2 //求连续1的最长子列，有点难度，容易打错
3
4 #include<bits/stdc++.h>
5 using namespace std;
6 const int maxn=100001;
7 int arr[maxn],sum[maxn<<2],pre0[maxn<<2],len0[maxn<<2],suf0[
8     maxn<<2],pre1[maxn<<2],len1[maxn<<2],suf1[maxn<<2];
9 int lazy[maxn<<2];
10 bool update[maxn<<2];
11 bool rever[maxn<<2];
12 void up(int i,int ln,int rn){
13     int l=i<<1;
14     int r=i<<1|1;
15     sum[i]=sum[l]+sum[r];
16     len0[i]=max(max(len0[l],len0[r]),suf0[l]+pre0[r]);
17     pre0[i]=len0[l]<ln?suf0[l]:pre0[l]+pre0[r];
18     suf0[i]=len0[r]<rn?suf0[r]:suf0[l]+suf0[r];
19     len1[i]=max(max(len1[l],len1[r]),suf1[l]+pre1[r]);
20     pre1[i]=len1[l]<ln?pre1[l]:pre1[l]+pre1[r];
21     suf1[i]=len1[r]<rn?suf1[r]:suf1[l]+suf1[r];
22 }
23 void updateLazy(int i,int v,int n){
24     sum[i]=v*n;
25     pre0[i]=suf0[i]=len0[i]=v==0?n:0;
26     pre1[i]=suf1[i]=len1[i]=v==1?n:0;
27     lazy[i]=v;
28     update[i]=true;
29     rever[i]=false;
30 }
31 void reverseLazy(int i,int n){
32     sum[i]=n-sum[i];
33     swap(pre0[i],pre1[i]);
34     swap(len0[i],len1[i]);
35     swap(suf0[i],suf1[i]);
36     rever[i]=!rever[i];
37 }
38 void down(int i,int ln,int rn){
39     if(update[i]){
40         updateLazy(i<<1,lazy[i],ln);
41         updateLazy(i<<1|1,lazy[i],rn);
42         update[i]=false;
43     }
44     if(rever[i]){
45         reverseLazy(i<<1,ln);
46         reverseLazy(i<<1|1,rn);
47         rever[i]=false;
48     }
49 }
50 void build(int l,int r,int i){
51     if(l==r){
52         sum[i]=arr[l];
53         pre0[i]=arr[l]?0:1;
54         suf0[i]=arr[l]?0:1;
55         len0[i]=arr[l]?0:1;
56         pre1[i]=arr[l]?1:0;
57         suf1[i]=arr[l]?1:0;
58         len1[i]=arr[l]?1:0;
59         return;
60     }

```

```

59     }
60     int mid=(l+r)>>1;
61     build(l,mid,i<<1),build(mid+1,r,i<<1|1);
62     up(i,mid-1+1,r-mid);
63     rever[i]=0;
64     update[i]=0;
65 }
66 void Update(int jl,int jr,int v,int l,int r,int i){
67     if(jl<=l&&r<=jr){
68         updateLazy(i,v,r-l+1);
69     }else {
70         int mid=(l+r)>>1;
71         down(i,mid-1+1,r-mid);
72         if(jl<=mid) Update(jl,jr,v,l,mid,i<<1);
73         if(jr>mid) Update(jl,jr,v,mid+1,r,i<<1|1);
74         up(i,mid-1+1,r-mid);
75     }
76 }
77 void Reverse(int jl,int jr,int l,int r,int i){
78     if(jl<=l&&r<=jr){
79         reverseLazy(i,r-l+1);
80     }else {
81         int mid=(l+r)>>1;
82         down(i,mid-1+1,r-mid);
83         if(jl<=mid) Reverse(jl,jr,l,mid,i<<1);
84         if(jr>mid) Reverse(jl,jr,mid+1,r,i<<1|1);
85         up(i,mid-1+1,r-mid);
86     }
87 }
88 tuple<int,int,int> querylength(int jl,int jr,int l,int r,int i
89 ){
90     if(jl<=l&&r<=jr){
91         return make_tuple(len1[i],pre1[i],suf1[i]);
92     }
93     int mid=(l+r)>>1;
94     down(i,mid-1+1,r-mid);
95     if(jr<=mid) return querylength(jl,jr,l,mid,i<<1);
96     if(jl>mid) return querylength(jl,jr,mid+1,r,i<<1|1);
97     tuple<int,int,int> l1=querylength(jl,jr,l,mid,i<<1);
98     tuple<int,int,int> r1=querylength(jl,jr,mid+1,r,i<<1|1);
99     int llen=get<0>(l1),lpre=get<1>(l1),lsuf=get<2>(l1);
100    int rlen=get<0>(r1),rpre=get<1>(r1),rsuf=get<2>(r1);
101    int len=max(max(llen,rlen),lsuf+rpre);
102    int pre=lpre<mid-max(1,jl)+1?lpre:lpre+rpre;
103    int suf=rsuf<min(r,jr)-mid?rsuf:rsuf+lsuf;
104    return make_tuple(len,pre,suf);
105 }
106 int query(int jl,int jr,int l,int r,int i){
107     if(jl<=l&&r<=jr){
108         return sum[i];
109     }
110     int mid=(l+r)>>1;
111     down(i,mid-1+1,r-mid);
112     int ans=0;
113     if(jl<=mid) ans+=query(jl,jr,l,mid,i<<1);
114     if(jr>mid) ans+=query(jl,jr,mid+1,r,i<<1|1);
115     return ans;
116 }
117 int main(){
118     int n,m;
119     cin>>n>>m;
120     for(int i=1;i<=n;i++) cin>>arr[i];
121     build(1,n,1);
122     while(m--){
123         int opt,l,r;
124         cin>>opt>>l>>r;
125         l+=1;
126         r+=1;
127         if(opt==0){
128             Update(l,r,0,1,n,1);
129         }else if(opt==1){
130             Update(l,r,1,1,n,1);
131         }else if(opt==2){
132             Reverse(l,r,1,n,1);
133         }else if(opt==3){
134             cout<<query(l,r,1,n,1)<<endl;
135         }else {
136             tuple<int,int,int>a=querylength(l,r,1,n,1);
137             cout<<get<0>(a)<<endl;
138         }
139     }
140 }

```

2.19 非经典线段树

```

1 // 这类题目一般是可以看出使用线段树，但是不能满足一般的懒更
2 // 新的条件
3 // 简单的一般需要进行势能分析，再减枝就可以解决
4 // 例如 洛谷 P4145，此题是开根号，换为取模同理
5 // P1471 求方差
6 #include<bits/stdc++.h>
7 using namespace std;
8
9 const int maxn=100001;
10 double a[maxn],sum[maxn<<2],pf[maxn<<2],lazy[maxn<<2];
11 void up(int i){
12     sum[i]=sum[i<<1]+sum[i<<1|1];
13     pf[i]=pf[i<<1]+pf[i<<1|1];
14 }
15 void Lazy(int i,double v,int n){
16     lazy[i]+=v;
17     pf[i]+=2*v*sum[i]+v*v*n;
18     sum[i]+=v*n;
19 }
20 void down(int i,int ln,int rn){
21     if(lazy[i]!=0){
22         Lazy(i<<1,lazy[i],ln);
23         Lazy(i<<1|1,lazy[i],rn);
24         lazy[i]=0;
25     }
26 }
27 void add(int jl,int jr,double v,int l,int r,int i){
28     if(jl<=l&&r<=jr){
29         Lazy(i,v,r-l+1);
30         return;
31     }
32     int mid=(l+r)>>1;
33     down(i,mid-1+1,r-mid);
34     if(jl<=mid){
35         add(jl,jr,v,l,mid,i<<1);
36     }
37     if(jr>mid) add(jl,jr,v,mid+1,r,i<<1|1);
38     up(i);
39 }
40 double query(int jl,int jr,int jt,int l,int r,int i){
41     if(jl<=l&&r<=jr){
42         if(jt){
43             return pf[i];
44         }else return sum[i];
45     }
46     int mid=(l+r)>>1;
47     down(i,mid-1+1,r-mid);
48     double ans=0;
49     if(jl<=mid) ans+=query(jl,jr,jt,l,mid,i<<1);
50     if(jr>mid) ans+=query(jl,jr,jt,mid+1,r,i<<1|1);
51     return ans;
52 }
53 void build(int l,int r,int i){
54     if(l==r){
55         sum[i]=a[l];
56         pf[i]=a[l]*a[l];
57     }
58     else {
59         int mid=(l+r)>>1;
60         build(l,mid,i<<1);
61         build(mid+1,r,i<<1|1);
62         up(i);
63     }
64     lazy[i]=0;
65 }
66
67 int main(){
68     ios_base::sync_with_stdio(false);
69     cin.tie(nullptr);
70     int n,m;
71     cin>>n>>m;
72     for(int i=1;i<=n;i++){
73         cin>>a[i];
74     }
75     build(1,n,1);
76     while(m--){
77         int opt;
78         cin>>opt;
79         if(opt==1){
80             int l,r;
81             double v;

```

```

82         cin>>l>>r>>v;
83         add(1,r,v,1,n,1);
84     }else if(opt==2){
85         int l,r;
86         cin>>l>>r;
87         printf("%.4lf\n", query(1,r,0,1,n,1)/(r-l+1));
88     }else {
89         int l,r;
90         cin>>l>>r;
91         printf("%.4lf\n", query(1,r,1,1,n,1)/(r-l+1)-query(
92             1,r,0,1,n,1)/(r-l+1)*query(1,r,0,1,n,1)/(r-l
93             +1));
94     }
95 }
96 }
97 }
98 }
99 }

```

2.20 左偏树

```

1 //可并堆结构，两个堆的合并时间复杂度为Log (n)
2 //查询操作需要并查集的优化
3 //P3377, 模板, 删除一个堆中最小值
4 #include<bits/stdc++.h>
5 using namespace std;
6 const int maxn=100001;
7 int n,m;
8 int num[maxn],lef[maxn],righ[maxn],dis[maxn];
9 int father[maxn];
10 //father并不是代表该节点的父亲节点，而是代表并查集所对应的路径
   信息
11 //需要保证，并查集最上方代表节点=左偏树的头，也就是堆顶
12 void prepare(){
13     dis[0]=-1;
14     for(int i=1;i<=n;i++){
15         lef[i]=0;
16         righ[i]=0;
17         dis[i]=0;
18         father[i]=i;
19     }
20 }
21 int find(int i){
22     father[i]=father[i]==i?i:find(father[i]);
23     return father[i];
24 }
25 int merge(int i,int j){
26     if(i==0||j==0) return i+j;
27     //维护小根堆，如果值一样，编号小的节点做头，大根堆同理
28     if(num[i]>num[j]||(num[i]==num[j]&&i>j)){
29         swap(i,j);
30     }
31     righ[i]=merge(righ[i],j);
32     if(dis[lef[i]]<dis[righ[i]]){
33         swap(lef[i],righ[i]);
34     }
35     dis[i]=dis[righ[i]]+1;
36     father[lef[i]]=father[righ[i]]=i;
37     return i;
38 }
39 //节点i一定是左偏树的头，在左偏树上删除节点i，返回新树的头结点
   编号
40 int pop(int i){
41     father[lef[i]]=lef[i];
42     father[righ[i]]=righ[i];
43     father[i]=merge(lef[i],righ[i]); //保证删除节点后，能够正确
   再次查询
44     lef[i]=righ[i]=dis[i]=0;
45     return father[i];
46 }
47 int main(){
48     cin>>n>>m;
49     prepare();
50     for(int i=1;i<=n;i++) cin>>num[i];
51     for(int i=1;i<=m;i++){
52         int op;
53         cin>>op;
54         if(op==1){
55             int l,r;
56             cin>>l>>r;
57             if(num[l]!=-1&&num[r]!=-1){
58                 l=find(l),r=find(r);
59                 if(l!=r) merge(l,r);
60             }
61         }else {
62             int x;

```

```

63         cin>>x;
64         if(num[x]==-1) cout<<-1<<endl;
65         else {
66             int ans=find(x);
67             cout<<num[ans]<<endl;
68             pop(ans);
69             num[ans]=-1;
70         }
71     }
72 }
73 }

```

3 图论

3.1 最短路: bitset 优化传递闭包

```

1 bitset<N> b[N];
2 for (int j = 1; j <= n; ++j) { // 注意中转点在最外层
3     for (int i = 1; i <= n; ++i) {
4         if (b[i][j]) { b[i] |= b[j]; }
5     }
6 }

```

3.2 最短路: 有向图上找最小环

```

1 void Dijkstra(int s)
2 {
3     priority_queue <pii, vector<pii>, greater<pii>> q;
4     for (int i = 1; i <= n; ++i) { vis[i]=0; dis[i]=inf; }
5     dis[s] = 0;
6     q.push( {0, s} );
7     while (!q.empty()) {
8         int u = q.top().second; q.pop();
9         if (vis[u]) { continue; }
10        vis[u] = 1;
11        for (auto [v,p]: G[u]) {
12            if (dis[v] > dis[u] + p) {
13                dis[v] = dis[u] + p;
14                if (!vis[v]) { q.push( {dis[v], v}); }
15            }
16            // 与 Dijkstra 唯一的区别
17            if (v == s) { mnc = min(mnc, dis[u]+p); }
18        }
19    }
20 }

```

3.3 最短路: spfa 判断负环

```

1 bool spfa(int s=1)
2 {
3     queue <int> q;
4     vector <int> dis(n+1, inf), cnt(n+1, 0);
5     vector <bool> inq(n+1, false);
6     dis[s] = 0;
7     inq[s] = true;
8     q.push(s);
9     while (!q.empty()) {
10        int u = q.front(); q.pop();
11        inq[u] = false;
12        for (auto& [v, w]: G[u]) {
13            if (dis[v] > dis[u] + w) {
14                dis[v] = dis[u] + w;
15                cnt[v] = cnt[u] + 1; // 最短路径上的节点数量，无负环
   则至多为 n-1
16                if (cnt[v] >= n) { return true; }
17                if (!inq[v]) { q.push(v); inq[v] = true; }
18            }
19        }
20    }
21    return false;
22 }

```

3.4 最小生成树

```

1 int n, m;
2 vector<pii> G[N];
3 int kruskal()
4 { // 默认是一张连通图
5     vector <array<int,3>> e(m);
6     for (auto& [w, u, v]: e) { cin >> u >> v >> w; }

```

```

7 sort(e.begin(), e.end());
8 int ans = 0, cnt = 0;
9 d = dsu(n); // 见并查集模板
10 for (auto& [w, u, v]: e) {
11     if (d.find(u) != d.find(v)) {
12         d.merge(u, v);
13         ans += w;
14         ++cnt;
15         // 以下两行为建图，完成后 G 中存放了最小生成树
16         G[u].push_back( { v, w } );
17         G[v].push_back( { u, w } );
18     }
19     if (cnt == n-1) { break; }
20 }
21 return ans;
22 }

```

3.5 拓扑排序

```

1 queue<int> q;
2 for (int i = 1; i <= n; ++i) {
3     if (deg[i] == 0) { q.push(i); }
4 }
5 vector<int> topo;
6 while (!q.empty()) {
7     int u = q.front(); q.pop();
8     topo.push_back(u);
9     for (int v: G[u]) {
10         if (--deg[v] == 0) { q.push(v); }
11     }
12 }

```

3.6 LCA: 倍增 $O(\log N)$ 求 LCA

```

1 const int N=5e5+5, I=20;
2 int n, m, s, dep[N], f[I+5][N], mn[I+5][N];
3 vector<pii> G[N];
4
5 void dfs(int u, int fa)
6 { // 预处理父节点、深度、路径上的最小值
7     f[0][u] = fa;
8     dep[u] = dep[fa] + 1;
9     for (int i = 1; i <= I; ++i) {
10         f[i][u] = f[i-1][ f[i-1][u] ];
11         mn[i][u] = min( mn[i-1][u], mn[i-1][ f[i-1][u] ] );
12     }
13     for (auto [v, w]: G[u]) {
14         if (v != fa) {
15             mn[0][v] = w;
16             dfs(v, u);
17         }
18     }
19 }
20
21 int LCA(int u, int v)
22 { // 求 u, v 到其 LCA 路径上的最小值
23     int ans = numeric_limits<int>::max();
24     function<void(int&,int)> jump = [&](int &u, int i) {
25         ans = min(ans, mn[i][u]);
26         u = f[i][u];
27     };
28     if (dep[u] < dep[v]) { swap(u, v); }
29     for (int i = I; i >= 0; --i) {
30         if (dep[f[i][u]] >= dep[v]) { jump(u, i); }
31     }
32     if (u == v) { return ans; }
33     for (int i = I; i >= 0; --i) {
34         if (f[i][u] != f[i][v]) {
35             jump(u, i); jump(v, i);
36         }
37     }
38     jump(u, 0); jump(v, 0);
39     return ans;
40 }

```

3.7 LCA: 欧拉序 +ST 表 $O(1)$ 求 LCA

```

1 // 见 ST 表板子
2 // 用 pair 实现深度比较，比较函数为 min
3 template <typename T>
4 struct ST {};

```

```

5
6 int first[N], dep[N];
7 vector<int> G[N];
8 vector<pii> euler;
9 void dfs(int u, int fa)
10 { // 预处理深度和欧拉序
11     first[u] = euler.size();
12     dep[u] = dep[fa] + 1;
13     euler.push_back( {dep[u], u} );
14     for (int v: G[u]) {
15         if (v != fa) {
16             dfs(v, u);
17             euler.push_back( {dep[u], u} );
18         }
19     }
20 }
21
22 ST<pii> st;
23 int lca(int u, int v)
24 {
25     u = first[u];
26     v = first[v];
27     if (u > v) { swap(u, v); }
28     return st.query(u, v).second;
29 }
30
31 int main()
32 {
33     // 预处理欧拉序，构建 ST 表，之后就可以直接调用 lca 进行求解
34     dfs(s, 0);
35     st = ST<pii>(euler);
36 }

```

3.8 树的直径与中心

```

1 int d[N], mxd;
2 void dfs(int u, int fa)
3 {
4     for (auto v: G[u]) {
5         if (v != fa) {
6             dfs(v, u);
7             mxd = max(mxd, d[u]+d[v]+1);
8             d[u] = max(d[u], d[v]+1);
9         }
10     }
11 }

```

3.9 点分治与树的重心

```

1 int n, siz[N], dis[N];
2 bool vis[N];
3 vector<pii> G[N];
4
5 void calcsiz(int u, int fa, int sum, int &core)
6 { // 求树的重心，sum 表述当前树的大小
7     siz[u] = 1;
8     int mxz = 0;
9     for (auto [v, w]: G[u]) {
10         if (v != fa && !vis[v]) {
11             calcsiz(v, u, sum, core);
12             siz[u] += siz[v];
13             mxz = max(mxz, siz[v]);
14         }
15     }
16     mxz = max(mxz, sum-siz[u]);
17     if (mxz * 2 <= sum) { core = u; }
18 }
19
20 void calcdis(int u, int fa)
21 {
22     for (auto [v, w]: G[u]) {
23         if (v != fa && !vis[v]) {
24             dis[v] = dis[u] + w;
25             calcdis(v, u);
26         }
27     }
28 }
29
30 void work(int u, int fa)
31 {
32     vis[u] = true;
33     for (auto [v, w]: G[u]) {

```

```

34     if (v != fa && !vis[v]) {
35         dis[v] = w;
36         calcds(v, u);
37     }
38 }
39 for (auto [v, w]: G[u]) {
40     if (v != fa && !vis[v]) {
41         int sum = siz[v];
42         int core = 0;
43         calcsiz(v, u, sum, core);
44         calcsiz(core, u, sum, core); // 第二次调用更新
45         siz[]
46         work(core, u);
47     }
48 }
49 }
50 int main()
51 {
52     int core = 0;
53     calcsiz(1, 0, n, core);
54     calcsiz(core, 0, n, core);
55     work(core, 0);
56 }

```

3.10 基环树

```

1 vector<int> G[N], cycle;
2 void dfs(int u, int fa)
3 {
4     cycle.push_back(u);
5     if (u == t) {
6         sort(cycle.begin(), cycle.end());
7         for (int x: cycle) { cout << x << " "; }
8         exit(0);
9     }
10    for (int v: G[u]) {
11        if (v != fa) { dfs(v, u); }
12    }
13    cycle.pop_back();
14 }
15 int main()
16 {
17     dsu d(n); // 见并查集板子
18     for (int i = 1; i <= n; ++i) {
19         int u, v;
20         cin >> u >> v;
21         G[u].push_back(v);
22         G[v].push_back(u);
23         if (d.same(u, v)) { s=u; t=v; }
24         d.merge(u, v);
25     }
26     dfs(s, 0);
27 }

```

3.11 Hierholzer 求欧拉路

```

1 int n, deg[N<<1], cur[N<<1];
2 bool vis[N];
3 vector<int> ans;
4 vector<pair<int,int>> G[N<<1];
5
6 void dfs(int u)
7 {
8     for (int &j = cur[u]; j < G[u].size(); ++j) {
9         auto [v, i] = G[u][j];
10        if (!vis[i]) {
11            vis[i] = true;
12            dfs(v);
13            ans.push_back(i);
14        }
15    }
16 }
17
18 int main()
19 {
20     int s = 1;
21     while (s <= 2*n && deg[s] % 2 == 0) { ++s; }
22     if (s > 2 * n) {
23         s = 1;
24         while (!deg[s]) { ++s; }
25     }

```

```

26     int cnt = 0;
27     for (int i = 1; i <= 2*n; ++i) {
28         cnt += deg[i] % 2;
29     }
30     if (cnt > 2) {
31         cout << "NO\n"; return;
32     }
33     dfs(s);
34
35     if (ans.size() != n) {
36         cout << "NO\n";
37     } else {
38         cout << "YES\n";
39         for (auto x: ans) {
40             cout << x << " \n"[x == ans.back()];
41         }
42     }
43 }
44 }
45 }

```

3.12 DSU on tree

```

1 const int N=2e5+5;
2 int n, c[N], siz[N], hvs[N];
3 int L[N], R[N], dfn[N], totdfn;
4 vector<int> G[N];
5
6 void dfs_init(int u, int fa)
7 { // 预处理 dfn 和重儿子
8     siz[u] = 1;
9     L[u] = ++totdfn;
10    dfn[totdfn] = u;
11    for (int v: G[u]) {
12        if (v != fa) {
13            dfs_init(v, u);
14            siz[u] += siz[v];
15            if (hvs[u] == 0 || siz[v] > siz[hvs[u]]) {
16                hvs[u] = v;
17            }
18        }
19    }
20    R[u] = totdfn;
21 }
22
23 void add(int u, int dt)
24 {
25     --ccnt[cnt[c[u]]];
26     cnt[c[u]] += dt;
27     ++ccnt[cnt[c[u]]];
28 }
29
30 void dfs_solve(int u, int fa, bool keep)
31 {
32     for (auto v: G[u]) { // 先做轻儿子, 不保留
33         if (v != fa && v != hvs[u]) {
34             dfs_solve(v, u, false);
35         }
36     }
37     if (hvs[u]) { // 再做重儿子, 保留
38         dfs_solve(hvs[u], u, true);
39     }
40     for (auto v: G[u]) { // 再把轻儿子的信息加上
41         if (v != fa && v != hvs[u]) {
42             for (int i = L[v]; i <= R[v]; ++i) {
43                 add(dfn[i], 1);
44             }
45         }
46     }
47     add(u, 1);
48     ans += (cnt[c[u]] * ccnt[cnt[c[u]]] == siz[u]);
49     if (keep) { return; }
50     for (int i = L[u]; i <= R[u]; ++i) {
51         add(dfn[i], -1);
52     }
53 }

```

3.13 tarjan: 强连通分量 (SCC)

```

1 struct SCC {
2     int n, totdfn=0, colcnt=0;
3     stack<int> stk;

```

```

4 vector<int> dfn, low, col;
5 vector<vector<int>> G;
6 SCC(int n): n(n) {
7     G.assign(n+1, {});
8     dfn.assign(n+1, 0);
9     low.assign(n+1, 0);
10    col.assign(n+1, 0);
11 }
12 void add_edge(int u, int v) {
13     G[u].push_back(v);
14 }
15 void dfs(int u) {
16     stk.push(u);
17     dfn[u] = low[u] = ++totdfn;
18     for (auto v: G[u]) {
19         if (!dfn[v]) {
20             dfs(v);
21             low[u] = min(low[u], low[v]);
22         } else if (!col[v]) {
23             low[u] = min(low[u], dfn[v]);
24         }
25     }
26     if (low[u] == dfn[u]) {
27         col[u] = ++colcnt;
28         int v;
29         do {
30             v = stk.top(); stk.pop();
31             col[v] = colcnt;
32         } while (v != u);
33     }
34 }
35 vector<int> work() {
36     for (int i = 1; i <= n; ++i) {
37         if (!dfn[i]) { dfs(i); }
38     }
39     return col;
40 }
41 };

```

3.14 tarjan: 割边与边双连通分量 (EBCC)

```

1 struct EBCC {
2     int n, totdfn=0, colcnt=0;
3     stack<int> stk;
4     vector<pii> B;
5     vector<int> dfn, low, col;
6     vector<vector<pii>> G;
7     EBCC(int n): n(n) {
8         G.assign(n+1, {});
9         dfn.assign(n+1, 0);
10        low.assign(n+1, 0);
11        col.assign(n+1, 0);
12    }
13    void add_edge(int u, int v, int id) {
14        G[u].emplace_back(v, id);
15        G[v].emplace_back(u, id);
16    }
17    void dfs(int u, int uid) {
18        stk.push(u);
19        dfn[u] = low[u] = ++totdfn;
20        for (auto [v, vid]: G[u]) {
21            if (uid != vid) {
22                if (!dfn[v]) {
23                    dfs(v, vid);
24                    low[u] = min(low[u], low[v]);
25                    if (low[v] > dfn[u]) { // 求桥
26                        B.emplace_back(min(u,v), max(u,v));
27                    }
28                } else if (!col[v] && dfn[v] < dfn[u]) {
29                    low[u] = min(low[u], dfn[v]);
30                }
31            }
32        }
33        if (low[u] == dfn[u]) {
34            col[u] = ++colcnt;
35            int v;
36            do {
37                v = stk.top(); stk.pop();
38                col[v] = colcnt;
39            } while (v != u);
40        }
41    }
42    vector<pii> work() { // 求桥

```

```

43        for (int i = 1; i <= n; ++i) {
44            if (!dfn[i]) { dfs(i, 0); }
45        }
46        return B;
47    }
48    struct Graph {
49        int n;
50        vector<array<int,2>> E;
51        vector<int> siz, cnt;
52        Graph(int n): n(n) {
53            siz.assign(n+1, 0); // EBCC 内部点的数量
54            cnt.assign(n+1, 0); // EBCC 内部边的数量
55        }
56    };
57    Graph compress() {
58        Graph g(colcnt);
59        for (int i = 1; i <= n; ++i) {
60            ++g.siz[col[i]];
61            for (auto [v, vid]: G[i]) {
62                if (col[v] < col[i]) {
63                    g.E.push_back( { col[i], col[v] } );
64                } else if (i < v) {
65                    ++g.cnt[col[i]];
66                }
67            }
68        }
69        return g;
70    }
71 };

```

3.15 tarjan: 割点与点双连通分量 (PBCC)

```

1 struct PBCC {
2     int n, root, totdfn=0, colcnt=0;
3     stack<int> stk;
4     vector<pii> B;
5     vector<int> dfn, low;
6     vector<bool> ver;
7     vector<vector<int>> pbcc;
8     vector<vector<pii>> G;
9     PBCC(int n): n(n) {
10        G.assign(n+1, {});
11        dfn.assign(n+1, 0);
12        low.assign(n+1, 0);
13        ver.assign(n+1, 0);
14    }
15    void add_edge(int u, int v, int id) {
16        G[u].emplace_back(v, id);
17        G[v].emplace_back(u, id);
18    }
19    void dfs(int u, int uid) {
20        stk.push(u);
21        int son = 0;
22        dfn[u] = low[u] = ++totdfn;
23        for (auto [v, vid]: G[u]) {
24            if (!dfn[v]) {
25                ++son;
26                dfs(v, vid);
27                low[u] = min(low[u], low[v]);
28                if (low[v] >= dfn[u]) {
29                    if (u != root) { ver[u] = true; }
30                    pbcc.push_back({});
31                    while (stk.top() != v) {
32                        pbcc.back().push_back(stk.top()); stk.
33                            pop();
34                    }
35                    pbcc.back().push_back(v); stk.pop();
36                    pbcc.back().push_back(u);
37                }
38            } else if (uid != vid) {
39                low[u] = min(low[u], dfn[v]);
40            }
41        }
42        if (u == root) {
43            if (son >= 2) { ver[u] = true; }
44            if (!son) { pbcc.push_back( { u } ); }
45        }
46    }
47    vector<int> work() { // 求割点
48        vector<int> V;
49        for (int i = 1; i <= n; ++i) {
50            if (!dfn[i]) { root=i; dfs(i, 0); }
51            if (ver[i]) { V.push_back(i); }

```

```

51     }
52     return v;
53 }
54 };

```

3.16 树哈希

```

1 using ull = unsigned long long;
2 const ull msk = chrono::steady_clock::now().time_since_epoch()
3     .count();
4 // ull h(ull x) { return x * x * x * 1237123 + 19260817; }
5 // ull f(ull x) { return h(x & ((1 << 31) - 1)) + h(x >> 31); }
6 ull shift(ull x)
7 {
8     x ^= msk;
9     x ^= x << 13;
10    x ^= x >> 7;
11    x ^= x << 17;
12    x ^= msk;
13    return x;
14 }
15
16 ull has[N];
17 void getHash(int u, int fa)
18 {
19     has[u] = 1;
20     for (auto v: G[u]) {
21         if (v == fa) { continue; }
22         getHash(v, u);
23         has[u] += shift(has[v]);
24     }
25 }

```

3.17 最大流

```

1 const ll INF = 9e18;
2 struct Flow {
3     vector<tuple<int,ll,int>> G[N]; // 终点, 流量, 反边
4     int dis[N], now[N]; // now[u] 为当前弧
5     void add_edge(int u, int v, ll w) {
6         G[u].push_back( { v, w, G[v].size() } );
7         G[v].push_back( { u, 0, G[u].size()-1 } );
8     }
9     void bfs(int st) { // 求层次图
10        queue<int> q;
11        memset(dis, 0, sizeof(dis));
12        q.push(st); dis[st] = 1;
13        while (!q.empty()) {
14            int u = q.front(); q.pop();
15            for (auto& [v, w, inv]: G[u]) {
16                if (w != 0 && dis[v] == 0) {
17                    dis[v] = dis[u] + 1;
18                    q.push(v);
19                }
20            }
21        }
22    }
23    ll dfs(int u, int t, ll flow) { // 求阻塞流
24        if (u == t) { return flow; }
25        for (int& i=now[u]; i < (int)G[u].size(); ++i) {
26            auto& [v, w, inv] = G[u][i];
27            if (w != 0 && dis[v] > dis[u]) {
28                ll d = dfs(v, t, min(flow, w));
29                if (d > 0) {
30                    w -= d;
31                    get<1>(G[v][inv]) += d;
32                    return d;
33                }
34            }
35        }
36        return 0;
37    }
38    ll dicnic(int st, int ed) {
39        for (ll flow=0, res;;) {
40            bfs(st);
41            if (dis[ed] == 0) { return flow; }
42            memset(now, 0, sizeof(now));
43            while ((res=dfs(st,ed,INF)) > 0) {
44                flow += res;
45            }
46        }
47    }
48 }

```

```

46     }
47 }
48 };

```

3.18 最近公共祖先 (LCA)

```

1 //利用树链剖分的方法
2 #include<bits/stdc++.h>
3 using namespace std;
4 const int N=5e5+5;
5 struct node{
6     int next,to;
7 }e[N<<1];
8 int tot,head[N];
9 int dep[N],siz[N],hson[N],fa[N];
10 int top[N];
11 int n,m,s;
12 void add(int x,int y){
13     e[++tot].to=y;
14     e[tot].next=head[x];
15     head[x]=tot;
16 }
17 void dfs1(int x,int f){
18     siz[x]=1;
19     fa[x]=f;
20     dep[x]=dep[f]+1;
21     for(int i=head[x];i;i=e[i].next){
22         int y=e[i].to;
23         if(y!=f){
24             dfs1(y,x);
25             siz[x]+=siz[y];
26             if(siz[hson[x]]<siz[y] || !hson[x]){
27                 hson[x]=y;
28             }
29         }
30     }
31 }
32 void dfs2(int x,int t){
33     top[x]=t;
34     if(!hson[x])return ;
35     dfs2(hson[x],t);
36     for(int i=head[x];i;i=e[i].next){
37         int y=e[i].to;
38         if(y!=fa[x] && y!=hson[x]){
39             dfs2(y,y);
40         }
41     }
42 }
43 int LCA(int x,int y){
44     while(top[x]!=top[y]){
45         if(dep[top[x]]<dep[top[y]]){
46             swap(x,y);
47         }
48         x=fa[top[x]];
49     }
50     return dep[x]<dep[y]?x:y;
51 }
52 int main(){
53     ios::sync_with_stdio(false);
54     cin.tie(NULL);
55     cout.tie(NULL);
56     cin>>n>>m>>s;
57     for(int i=1;i<n;i++){
58         int x,y;
59         cin>>x>>y;
60         add(x,y);
61         add(y,x);
62     }
63     dfs1(s,0);
64     dfs2(s,s);
65     for(int i=1;i<=m;i++){
66         int a,b;
67         cin>>a>>b;
68         cout<<LCA(a,b)<<'\\n';
69     }
70     return 0;
71 }

```

3.19 树链剖分

```

1 #include<algorithm>
2 #include<iostream>

```



```

3 #include<cstdlib>
4 #include<cstring>
5 #include<cstdio>
6 #define Rint register int
7 #define mem(a,b) memset(a,(b),sizeof(a))
8 #define Temp template<typename T>
9 using namespace std;
10 typedef long long LL;
11 Temp inline void read(T &x){
12     x=0;T w=1,ch=getchar();
13     while(!isdigit(ch)&&ch!='-')ch=getchar();
14     if(ch=='-')w=-1,ch=getchar();
15     while(isdigit(ch))x=(x<<3)+(x<<1)+(ch^'0'),ch=getchar();
16     x=x*w;
17 }
18
19 #define mid ((l+r)>>1)
20 #define lson rt<<1,l,mid
21 #define rson rt<<1|1,mid+1,r
22 #define len (r-l+1)
23
24 const int maxn=200000+10;
25 int n,m,r,mod;
26 //见题意
27 int e,beg[maxn],nex[maxn],to[maxn],w[maxn],wt[maxn];
28 //链式前向星数组, w[], wt[]初始点权数组
29 int a[maxn<<2],laz[maxn<<2];
30 //线段树数组、Lazy操作
31 int son[maxn],id[maxn],fa[maxn],cnt,dep[maxn],siz[maxn],top[
    maxn];
32 //son[]重儿子编号,id[]新编号,fa[]父亲节点,cnt dfs_clock/dfs序,
    dep[]深度,siz[]子树大小,top[]当前链顶端节点
33 int res=0;
34 //查询答案
35
36 inline void add(int x,int y){//链式前向星加边
37     to[++e]=y;
38     nex[e]=beg[x];
39     beg[x]=e;
40 }
41 //----- 以下为线段树
42 inline void pushdown(int rt,int len){
43     laz[rt<<1]+=laz[rt];
44     laz[rt<<1|1]+=laz[rt];
45     a[rt<<1]+=laz[rt]*(len-(len>>1));
46     a[rt<<1|1]+=laz[rt]*(len>>1);
47     a[rt<<1]%=mod;
48     a[rt<<1|1]%=mod;
49     laz[rt]=0;
50 }
51
52 inline void build(int rt,int l,int r){
53     if(l==r){
54         a[rt]=wt[l];
55         if(a[rt]>mod)a[rt]%=mod;
56         return;
57     }
58     build(lson);
59     build(rson);
60     a[rt]=(a[rt<<1]+a[rt<<1|1])%mod;
61 }
62
63 inline void query(int rt,int l,int r,int L,int R){
64     if(L<=l&&r<=R){res+=a[rt];res%=mod;return;}
65     else{
66         if(laz[rt])pushdown(rt,len);
67         if(L<=mid)query(lson,L,R);
68         if(R>mid)query(rson,L,R);
69     }
70 }
71
72 inline void update(int rt,int l,int r,int L,int R,int k){
73     if(L<=l&&r<=R){
74         laz[rt]+=k;
75         a[rt]+=k*len;
76     }
77     else{
78         if(laz[rt])pushdown(rt,len);
79         if(L<=mid)update(lson,L,R,k);
80         if(R>mid)update(rson,L,R,k);
81         a[rt]=(a[rt<<1]+a[rt<<1|1])%mod;
82     }
83 }
84 //----- 以上为线段树

```

```

85 inline int qRange(int x,int y){
86     int ans=0;
87     while(top[x]!=top[y]){//当两个点不在同一条链上
88         if(dep[top[x]]<dep[top[y]])swap(x,y);//把x点改为所在链顶端
            的深度更深的那个点
89         res=0;
90         query(1,1,n,id[top[x]],id[x]);//ans加上x点到x所在链顶端 这
            一段区间的点权和
91         ans+=res;
92         ans%=mod;//按题意取模
93         x=fa[top[x]);//把x跳到x所在链顶端的那个点的上面一个点
94     }
95     //直到两个点处于一条链上
96     if(dep[x]>dep[y])swap(x,y);//把x点深度更深的那个点
97     res=0;
98     query(1,1,n,id[x],id[y]);//这时再加上此时两个点的区间和即可
99     ans+=res;
100     return ans%mod;
101 }
102
103 inline void updRange(int x,int y,int k){//同上
104     k%=mod;
105     while(top[x]!=top[y]){
106         if(dep[top[x]]<dep[top[y]])swap(x,y);
107         update(1,1,n,id[top[x]],id[x],k);
108         x=fa[top[x]];
109     }
110     if(dep[x]>dep[y])swap(x,y);
111     update(1,1,n,id[x],id[y],k);
112 }
113
114 inline int qSon(int x){
115     res=0;
116     query(1,1,n,id[x],id[x]+siz[x]-1);//子树区间右端点为id[x]+
        siz[x]-1
117     return res;
118 }
119
120 inline void updSon(int x,int k){//同上
121     update(1,1,n,id[x],id[x]+siz[x]-1,k);
122 }
123
124 inline void dfs1(int x,int f,int deep){//x当前节点, f父亲,
        deep深度
125     dep[x]=deep;//标记每个点的深度
126     fa[x]=f;//标记每个点的父亲
127     siz[x]=1;//标记每个非叶子节点的子树大小
128     int maxson=-1;//记录重儿子的儿子数
129     for(Rint i=beg[x];i;i=nex[i]){
130         int y=to[i];
131         if(y==f)continue;//若为父亲则continue
132         dfs1(y,x,deep+1);//dfs其儿子
133         siz[x]+=siz[y];//把它的儿子数加到它身上
134         if(siz[y]>maxson)son[x]=y,maxson=siz[y];//标记每个非叶子节
            点的重儿子编号
135     }
136 }
137
138 inline void dfs2(int x,int topf){//x当前节点, topf当前链的最顶
        端的节点
139     id[x]=++cnt;//标记每个点的新编号
140     wt[cnt]=w[x]);//把每个点的初始值赋到新编号上来
141     top[x]=topf;//这个点所在链的顶端
142     if(!son[x])return;//如果没有儿子则返回
143     dfs2(son[x],topf);//按先处理重儿子, 再处理轻儿子的顺序递归处
        理
144     for(Rint i=beg[x];i;i=nex[i]){
145         int y=to[i];
146         if(y==fa[x]||y==son[x])continue;
147         dfs2(y,y);//对于每一个轻儿子都有一条从它自己开始的链
148     }
149 }
150
151 int main(){
152     read(n);read(m);read(r);read(mod);
153     for(Rint i=1;i<=n;i++)read(w[i]);
154     for(Rint i=1;i<=n;i++){
155         int a,b;
156         read(a);read(b);
157         add(a,b);add(b,a);
158     }
159     dfs1(r,0,1);
160     dfs2(r,r);
161     build(1,1,n);

```

```

162 while(m--){
163     int k,x,y,z;
164     read(k);
165     if(k==1){
166         read(x);read(y);read(z);
167         updRange(x,y,z);
168     }
169     else if(k==2){
170         read(x);read(y);
171         printf("%d\n",qRange(x,y));
172     }
173     else if(k==3){
174         read(x);read(y);
175         updSon(x,y);
176     }
177     else{
178         read(x);
179         printf("%d\n",qSon(x));
180     }
181 }
182 }

```

3.20 树上倍增

```

1 //给定任意节点i，可以快速查询，从i节点往上走的路径中位于第s层
  的编号
2 //生成复杂度 nLog(n)，查询复杂度 Log(n)
3 #include<bits/stdc++.h>
4 using namespace std;
5 const int maxn=5e5+10;
6 int limit=32;
7 int power;
8 int log2(int x){
9     int ans=0;
10    while((1<<ans)<=(n>>1)){
11        ans++;
12    }
13    return ans;;
14 }
15 struct tree{
16     int n,t;
17 }tr[maxn];
18 int head[maxn],deep[maxn],st[maxn][limit],tot=1;
19 void add(int u,int v){
20     tr[++tot].t=v;
21     tr[tot].n=head[u];
22     head[u]=tot;
23 }
24 void dfs(int i,int f){
25     if(i==0){//0节点为头节点
26         deep[i]=1;
27     }else deep[i]=deep[fa]+1;
28     st[i][0]=f;
29     for(int p=1;(1<<p)<=deep[i];p++){
30         st[i][p]=st[st[i][p-1]][p-1];
31     }
32     for(int ei=head[i];ei;ei=tr[ei].n){
33         dfs(tr[ei].t,i);
34     }
35 }
36 int getkthAncestor(int i,int k){
37     if(deep[i]<=k) return -1;
38     int s=deep[i]-k;
39     for(int p=power;p>=0;p--){
40         if(deep[st[i][p]]>=s){
41             i=st[i][p];
42         }
43     }
44     return i;
45 }

```

3.21 最大路匹配

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 const int N=510;
4 vector<int>g[N];
5 bool vis[N];
6 int match[N];
7 bool dfs(int u){
8     for(int v:g[u]){
9         if(!vis[v]){

```

```

10         vis[v]=1;
11         if(match[v]==0||dfs(match[v])){
12             match[v]=u;
13             return true;
14         }
15     }
16     return false;
17 }
18 int main(){
19     int n,m,ans=0;
20     cin>>n>>m;
21     for(int i=1;i<=n;i++){
22         int x;
23         cin>>x;
24         for(int j=1;j<=x;j++){
25             int y;
26             cin>>y;
27             g[i].push_back(y);
28         }
29     }
30     for(int i=1;i<=n;i++){
31         memset(vis,0,sizeof(vis));
32         if(dfs(i)) ans++;
33     }
34     cout<<ans;
35 }

```

3.22 Dijkstra 算法

```

1 //小根堆实现较为简单，这里采用以点为复杂度的反向索引堆的优化，
  一般不用
2 #include<bits/stdc++.h>
3 using namespace std;
4 const int maxn=100010;
5 const int inf=1e8;
6 struct graph{
7     int next,to,w;
8 }g[maxn*2];
9 int head[maxn],cnt,n,m;
10 int heap[maxn];//反向索引堆
11 int where[maxn];//表示点在堆上位置，初始化为-1
12 int heapSize;
13 int dis[maxn];
14 void build(int n){
15     cnt=1;
16     heapSize=0;
17     for(int i=1;i<=n;i++){
18         heap[i]=0;
19         where[i]=-1;
20         head[i]=0;
21         dis[i]=inf;
22     }
23 }
24 void add(int u,int v,int w){
25     g[cnt].next=head[u];
26     g[cnt].to=v;
27     g[cnt].w=w;
28     head[u]=cnt++;
29 }
30 void Swap(int i,int j){
31     swap(heap[i],heap[j]);
32     where[heap[i]]=i;
33     where[heap[j]]=j;
34 }
35 void heapInsert(int i){
36     while(dis[heap[i]]<dis[heap[(i-1)/2]]){
37         Swap(i,(i-1)/2);
38         i=(i-1)/2;
39     }
40 }
41 void addOrUpdateorignore(int v,int c){
42     if(where[v]==-1){
43         heap[heapSize]=v;
44         where[v]=heapSize++;
45         dis[v]=c;
46         heapInsert(where[v]);
47     }else if(where[v]>=0){
48         dis[v]=min(dis[v],c);
49         heapInsert(where[v]);
50     }
51 }
52 void heapify(int i){

```

```

53     int l=1;
54     while(l<heapSize){
55         int best=l+1<heapSize&&dis[heap[l+1]]<dis[heap[l]]?l
56             +1:l;
57         best=dis[heap[best]]<dis[heap[i]]?best:i;
58         if(best==i) break;
59         Swap(best,i);
60         i=best;
61         l=i*2+1;
62     }
63     bool isEmpty(){
64         return heapSize==0;
65     }
66     int pop(){
67         int ans=heap[0];
68         swap(heap[0],heap[--heapSize]);
69         heapify(0);
70         where[ans]=-2;
71         return ans;
72     }
73     int main(){
74         cin>>n>>m;
75         int s;
76         cin>>s;
77         build(n);
78         for(int i=1;i<=m;i++){
79             int u,v,w;
80             cin>>u>>v>>w;
81             add(u,v,w);
82         }
83         addorupdateorignore(s,0);
84         while(!isEmpty()){
85             int u=pop();
86             for(int ei=head[u];ei>0;ei=g[ei].next){
87                 addorupdateorignore(g[ei].to,dis[u]+g[ei].w);
88             }
89         }
90         int ans=-inf;
91         for(int i=1;i<=n;i++){
92             /*if(dis[i]==-inf){
93                 cout<<-1<<endl;
94             }
95             ans=max(ans,dis[i]);*/
96             cout<<dis[i]<<' ';
97         }
98         //cout<<ans<<endl;
99     }

```

```

32     for(int i=1;i<=m;i++){
33         int u,v,w;
34         cin>>u>>v>>w;
35         add(u,v,w);
36         add(v,u,w);
37     }
38     for(int i=0;i<=n;i++){
39         for(int j=0;j<=k;j++){
40             dis[i][j]=inf;
41         }
42     }
43     dis[s][0]=0;
44     q.push({s,0,0});
45     while(!q.empty()){
46         int xn=q.top().x;
47         int cost=q.top().cost;
48         int kn=q.top().k;
49         q.pop();
50         if(vis[xn][kn]) continue;
51         vis[xn][kn]=1;
52         if(xn==t){
53             cout<<cost<<endl;
54             return 0;
55         }
56         for(int i=head[xn];i>0;i=g[i].next){
57             int v=g[i].to;
58             if(kn<k&&cost<dis[v][kn+1]){
59                 dis[v][kn+1]=cost;
60                 q.push({v,cost,kn+1});
61             }
62             if(cost+g[i].w<dis[v][kn]){
63                 dis[v][kn]=cost+g[i].w;
64                 q.push({v,cost+g[i].w,kn});
65             }
66         }
67     }
68 }
69 /*
70 struct Compare {
71     bool operator()(const State& a, const State& b) {
72         return a.cost > b.cost; // 小根堆
73     }
74 };
75 priority_queue<State, vector<State>, Compare> q;
76 这样子定义也可以
77 */

```

3.23 分层图最短路

```

1 //无论是在bfs上分层还是dijkstra上，本质其实就是把原图上的点与
2   其的不同状态整合出更多的点
3 //难点在于如何将点上的状态给储存下来，以及如何转移
4 #include<bits/stdc++.h>
5 using namespace std;
6 const int maxn=50010;
7 const int inf=1e8;
8 struct dot{
9     int x,cost,k;
10 };
11 struct graph{
12     int next,to,w;
13 }g[maxn<<1];
14 struct compare{
15     bool operator()(dot x,dot y){
16         return x.cost>y.cost;
17         // 小根堆
18     }
19 };
20 int head[maxn<<1],cnt=1,n,m,k,dis[maxn][20];
21 bool vis[maxn][20];
22 priority_queue<dot,vector<dot>,compare>q;
23 void add(int u,int v,int w){
24     g[cnt].next=head[u];
25     g[cnt].to=v;
26     g[cnt].w=w;
27     head[u]=cnt++;
28 }
29 int main(){
30     cin>>n>>m>>k;
31     int s,t;
32     cin>>s>>t;

```

3.24 双向广搜

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 ll n,m;
5 ll an[45];
6 ll l[1<<20+1],r[1<<20+1];
7 ll f(int i,int e,ll s,ll w,ll a[],ll j){
8     if(s>w) return j;
9     if(i==e) a[j++]=s;
10    else {
11        j=f(i+1,e,s,w,a,j);
12        j=f(i+1,e,s+an[i],w,a,j);
13    }
14    return j;
15 }
16 ll compute(){
17     ll lsize=f(0,n>>1,0,m,l,0);
18     ll rsize=f(n>>1,n,0,m,r,0);
19     sort(l,l+lsize);
20     sort(r,r+rsize);
21     ll ans=0;
22     for(int i=lsize-1,j=0;i>=0;i--){
23         while(j<rsize&&l[i]+r[j]<=m){
24             j++;
25         }
26         ans+=j;
27     }
28     return ans;
29 }
30 int main(){
31     cin>>n>>m;
32     for(int i=0;i<n;i++) cin>>an[i];
33     sort(an,an+n);
34     cout<<compute();

```

35

}

3.25 Bellman-Ford

```

1 //没有优化的Bellman-Ford
2 //每一轮操作都把所有边带进去，看距离是否变化，若不变化，就停止
3 //最多n-1轮，因此时间复杂度为O(n*m)
4 //适用于单源点，找最短路问题
5 //可以判断从某个点出发会不会遇到负环，若轮数超过n-1，就经过负环
6 //单独的Bellman-Ford不会单独使用
7 //一般要和SPFA优化
8 //SPFA只做到了常数级优化，仍然为O(n*m)
9 //因此没有负边时，仍然使用dijkstra
10
11 //判断是否存在负环，P3385
12 #include<bits/stdc++.h>
13 using namespace std;
14 const int maxn=2010,maxm=3010,inf=1e8;
15 int head[maxn<<1],n,m,cnt,upfill[maxn],dis[maxn],t;
16 queue<int>q;
17 bool enter[maxn];
18 struct graph{
19     int next,to,w;
20 }g[maxn<<1];
21 void add(int u,int v,int w){
22     g[cnt].next=head[u];
23     g[cnt].to=v;
24     g[cnt].w=w;
25     head[u]=cnt++;
26 }
27 void build(int n){
28     cnt=1;
29     while(!q.empty()) q.pop();
30     for(int i=1;i<=n;i++){
31         head[i]=0;
32         upfill[i]=0;
33         dis[i]=inf;
34         enter[i]=0;
35     }
36 }
37 bool spfa(){
38     dis[1]=0;
39     upfill[1]++;
40     q.push(1);
41     enter[1]=true;
42     while(!q.empty()){
43         int x=q.front();
44         q.pop();
45         enter[x]=false;
46         for(int ei=head[x];ei;ei=g[ei].next){
47             int v=g[ei].to;
48             int w=g[ei].w;
49             if(dis[x]+w<dis[v]){
50                 dis[v]=dis[x]+w;
51                 if(!enter[v]){
52                     if(upfill[v]++==n){
53                         return true;
54                     }
55                 }
56                 enter[v]=true;
57                 q.push(v);
58             }
59         }
60     }
61     return false;
62 }
63 void solve(){
64     cin>>n>>m;
65     build(n);
66     for(int i=1;i<=m;i++){
67         int u,v,w;
68         cin>>u>>v>>w;
69         if(w>=0){
70             add(u,v,w);
71             add(v,u,w);
72         }
73         else add(u,v,w);
74     }
75     if(spfa()) cout<<"YES"<<endl;
76     else cout<<"NO"<<endl;
77 }

```

```

78 int main(){
79     cin>>t;
80     while(t--){
81         solve();
82     }
83 }

```

3.26 tarjan(强连通分量)

```

1 //强连通分量求解
2 int dfn[N], low[N], dfncnt, s[N], in_stack[N], tp;
3 int scc[N], sc; // 结点 i 所在 SCC 的编号
4 int sz[N]; // 强连通 i 的大小
5
6 void tarjan(int u) {
7     low[u] = dfn[u] = ++dfncnt, s[++tp] = u, in_stack[u] = 1;
8     for (int i = h[u]; i; i = e[i].nex) {
9         const int &v = e[i].t;
10        if (!dfn[v]) {
11            tarjan(v);
12            low[u] = min(low[u], low[v]);
13        } else if (in_stack[v]) {
14            low[u] = min(low[u], dfn[v]);
15        }
16    }
17    if (dfn[u] == low[u]) {
18        ++sc;
19        do {
20            scc[s[tp]] = sc;
21            sz[sc]++;
22            in_stack[s[tp]] = 0;
23        } while (s[tp--] != u);
24    }
25 }

```

3.27 边双连通分量

```

1 #include <algorithm>
2 #include <iostream>
3 #include <stack>
4 #include <vector>
5
6 using namespace std;
7 constexpr int N = 5e5 + 5, M = 2e6 + 5;
8 int n, m;
9
10 struct edge {
11     int to, nt;
12 } e[M << 2];
13
14 int hd[N << 1], tot = 1;
15
16 void add(int u, int v) { e[++tot] = edge{v, hd[u]}, hd[u] = tot; }
17
18 void uadd(int u, int v) { add(u, v), add(v, u); }
19
20 int bfnct, sum;
21 int dfn[N], low[N], bcc[N], bcc_cnt;
22 bool vis[N];
23 vector<vector<int>> ans;
24 stack<int> st;
25 void tarjan(int u, int in) {
26     low[u] = dfn[u] = ++bfnct;
27     st.push(u), vis[u] = true;
28     for (int i = hd[u]; i; i = e[i].nt) {
29         int v = e[i].to;
30         if (i == (in ^ 1)) continue;
31         if (!dfn[v])
32             tarjan(v, i), low[u] = min(low[u], low[v]);
33         else if (vis[v])
34             low[u] = min(low[u], dfn[v]);
35     }
36     if (dfn[u] == low[u]) {
37         bcc_cnt++;
38         bcc[u]=bcc_cnt;
39         vector<int> t;
40         t.push_back(u);
41         while (st.top() != u)
42             t.push_back(st.top()), bcc[st.top()]=bcc_cnt, vis[st.top()] = false, st.pop();
43         st.pop(), ans.push_back(t);

```

```
44 }
45 }
```

3.28 割边

```
1 void tarjan(int u,int pre) {
2     low[u] = dfn[u] = ++dfncnt;
3     for (int ei = head[u]; ei; ei = e[ei].nex) {
4         if((ei^1)==pre) continue;
5         const int &v = e[ei].t;
6         if (!dfn[v]) {
7             tarjan(v,ei);
8
9             low[u] = min(low[u], low[v]);
10            if(low[v]>dfn[u]){
11                cutedge[ei>>1]=1;//无向图割边
12                ans1++;
13            }
14        } else {
15            low[u] = min(low[u], dfn[v]);
16        }
17    }
18 }
```

3.29 点双连通分量

```
1 void tarjan(int u) {
2     dfn[u] = low[u] = ++bcc_cnt, sta[++top] = u;
3     if (u == root && hd[u] == 0) { //root=i;
4         dcc[++cnt].push_back(u);
5         return;
6     }
7     int f = 0;
8     for (int i = hd[u]; i; i = e[i].nt) {
9         int v = e[i].to;
10        if (!dfn[v]) {
11            tarjan(v);
12            low[u] = min(low[u], low[v]);
13            if (low[v] >= dfn[u]) {
14                if (++f > 1 || u != root) cut[u] = true;
15                cnt++;
16                do dcc[cnt].push_back(sta[top--]);
17                while (sta[top + 1] != v);
18                dcc[cnt].push_back(u);
19            }
20        } else
21            low[u] = min(low[u], dfn[v]);
22    }
23 }
```

4 数学

4.1 线性筛

```
1 vector<int> sieve(int n)
2 {
3     vector<bool> vis(n+1);
4     vector<int> mP(n+1,1);
5     vector<int> prime;
6     vis[1] = 1;
7     for (int i = 2; i <= n; ++i) {
8         if (!vis[i]) {
9             prime.push_back(i);
10            mP[i] = i;
11        }
12        for (auto p: prime) {
13            if (i*p > n) { break; }
14            vis[i*p] = 1;
15            mP[i*p] = p;
16            if (i%p == 0) { break; } // 线性筛关键优化
17        }
18    }
19    return prime;
20 }
21
22 // 分解质因数
23 map<int,int> cnt;
24 for (int y = x; y > 1; y /= mP[y]) {
25     ++cnt[mP[y]];
26 }
```

4.2 因数

```
1 const int X=1e5;
2 vector<int> fac[X+5];
3 for (int i = 1; i <= X; ++i) {
4     for (int j = i; j <= X; j += i) {
5         fac[j].push_back(i);
6     }
7 }
```

4.3 快速幂与乘法逆元

```
1 const int MOD=998244353;
2 ll qpow(ll a, ll b=MOD-2)
3 { // 快速幂函数, x 的乘法逆元为 qpow(x)
4     ll res=1;
5     for (; b; b>>=1) {
6         if (b & 1) { res=(res*a)%MOD; }
7         a = a*a%MOD; // 不要少 MOD
8     }
9     return res;
10 }
```

4.4 排列组合数

```
1 const int N=1e5;
2 ll fac[N+5], Inv[N+5];
3 ll C(ll n, ll k) { return n==0? 1: fac[n] * Inv[k] % MOD * Inv
4     [n-k] % MOD; }
5 ll P(ll n, ll k) { return n==0? 1: fac[n] * Inv[n-k] % MOD; }
6 // 预处理阶乘和阶乘的逆元
7 for (int i = 1; i <= N; ++i) {
8     fac[i] = fac[i-1] * i % MOD;
9     Inv[i] = qpow(fac[i]);
10 }
```

4.5 Lucas

```
1 const int MOD = 1000003;
2 ll C(ll a, ll b) {
3     if (a < b) { return 0; }
4     ll down = 1, up = 1;
5     for (int i = a, j = 1; j <= b; --i, ++j) {
6         up = up * i % MOD;
7         down = down * j % MOD;
8     }
9     return up * qpow(down) % MOD;
10 }
11 ll lucas(ll a, ll b) {
12     if (a < p && b < p) { return C(a, b); }
13     return C(a%MOD, b%MOD) * lucas(a/MOD, b/MOD) % MOD;
14 }
```

4.6 卡特兰数

```
1 ll Cat(ll n) { return C(2*n, n) * qpow(n+1) % MOD; }
```

4.7 数论分块

```
1 ll l=1, r=0;
2 while (l <= n) {
3     r = n / (n/l);
4     // 统计 ans
5     l = r + 1;
6 }
```

4.8 高斯消元

```
1 int Guess(vector<vector<db>> &a)
2 {
3     int n = a.size();
4     int c, r;
5     for (c = 0, r = 0; c < n; ++c) {
6         // 选主元
7         int t = r;
8         for (int i = r + 1; i < n; ++i) {
9             if (fabs(a[i][c]) > fabs(a[t][c])) { t = i; }
10        }
11    }
```

```

11     if (sgn(a[t][c]) == 0) { continue; }
12
13     // 行交换
14     for (int i = c; i <= n; ++i) { swap(a[t][i], a[r][i]); }
15
16     // 归一化
17     for (int i = n; i >= c; --i) { a[r][i] /= a[r][c]; }
18
19     // 前向消元
20     for (int i = r + 1; i < n; ++i) {
21         if (sgn(a[i][c]) != 0) {
22             for (int j = n; j >= c; --j) {
23                 a[i][j] -= a[r][j] * a[i][c];
24             }
25         }
26     }
27     ++r;
28 }
29
30 if (r < n) {
31     for (int i = r; i < n; ++i) {
32         if (sgn(a[i][n]) != 0) { return 2; } // 无解
33     }
34     return 1; // 无穷多解
35 }
36
37 // 回代
38 for (int i = n - 1; i >= 0; --i) {
39     for (int j = i + 1; j < n; ++j) {
40         a[i][n] -= a[i][j] * a[j][n];
41     }
42 }
43 return 0;
44 }

```

4.9 矩阵快速幂

```

1 using matrix = vector<vector<ll>>>;
2 matrix IM(int n) {
3     matrix I(n, vector<ll>(n));
4     for (int i = 0; i < n; ++i) { I[i][i] = 1; }
5     return I;
6 }
7 matrix operator* (matrix A, matrix B) {
8     assert(A.front().size() == B.size());
9     int n = A.size(), m = B.size(), k = B.front().size();
10    matrix C(n, vector<ll>(m));
11    for (int i = 0; i < n; ++i) {
12        for (int j = 0; j < k; ++j) {
13            for (int x = 0; x < m; ++x) {
14                C[i][j] = add(C[i][j], mul(A[i][x], B[x][j]));
15            }
16        }
17    }
18    return C;
19 }
20 matrix qpow(matrix a, ll b) {
21     matrix res = IM(a.size());
22     for (; b >>= 1, a=a*a) {
23         if (b & 1) { res = res * a; }
24     }
25     return res;
26 }

```

4.10 自适应辛普森法

```

1 double simpson(double l, double r) {
2     double mid = (l + r) / 2;
3     return (r - l) * (f(l) + 4 * f(mid) + f(r)) / 6; // 辛普
      森公式
4 }
5
6 double asr(double l, double r, double eps, double ans,
7     int step) { // step是递归的下限
8     double mid = (l + r) / 2;
9     double fl = simpson(l, mid), fr = simpson(mid, r);
10    if (abs(fl + fr - ans) <= 15 * eps && step < 0)
11        return fl + fr + (fl + fr - ans) / 15; // 足够相似的
      话就直接返回
12    return asr(l, mid, eps / 2, fl, step - 1) +
13        asr(mid, r, eps / 2, fr, step - 1); // 否则分割成

```

两段递归求解

```

14 }
15
16 double calc(double l, double r, double eps) {
17     return asr(l, r, eps, simpson(l, r), 12);
18 }

```

4.11 多项式: FFT

```

1 struct cp {
2     db x, y;
3     cp(db real=0, db imag=0): x(real), y(imag) { };
4     cp operator+ (cp b) const { return {x+b.x, y+b.y}; }
5     cp operator- (cp b) const { return {x-b.x, y-b.y}; }
6     cp operator* (cp b) const { return {x*b.x - y*b.y, x*b.y +
      y*b.x}; }
7 };
8 using vcp = vector<cp>;
9 using Poly = vector<int>;
10 namespace FFT {
11     const db PI = acos(-1.0);
12     vcp Omega(int L) {
13         vcp w(L);
14         w[1] = 1;
15         for (int i = 2; i < L; i <= 1) {
16             auto w0 = w.begin() + i / 2;
17             auto w1 = w.begin() + i;
18             cp wn(cos(PI/i), sin(PI/i));
19             for (int j = 0; j < i; j += 2) {
20                 w1[j] = w0[j]>>1;
21                 w1[j+1] = w1[j] * wn;
22             }
23         }
24         return w;
25     }
26     auto W = Omega(1 << 21); // NOLINT
27     void DIF(cp *a, int n) {
28         cp x, y;
29         for (int k = n>>1; k; k >= 1) {
30             for (int i = 0; i < n; i += k<<1) {
31                 for (int j = 0; j < k; ++j) {
32                     x = a[i+j];
33                     y = a[i+j+k];
34                     a[i+j+k] = (a[i+j]-y) * W[k+j];
35                     a[i+j] = x+y;
36                 }
37             }
38         }
39     }
40     void IDIT(cp *a, int n) {
41         cp x, y;
42         for (int k = 1; k < n; k <= 1) {
43             for (int i = 0; i < n; i += k << 1) {
44                 for (int j = 0; j < k; ++j) {
45                     x = a[i+j];
46                     y = a[i+j+k] * W[k+j];
47                     a[i+j+k] = x-y;
48                     a[i+j] = x+y;
49                 }
50             }
51         }
52         const db Inv = 1.0 / n;
53         for (int i = 0; i < n; ++i) {
54             a[i].x *= Inv;
55             a[i].y *= Inv;
56         }
57         reverse(a+1, a+n);
58     }
59 }
60
61 namespace Polynomial {
62     void DFT(vcp &a) { FFT::DIF(a.data(), a.size()); }
63     void IDFT(vcp &a) { FFT::IDIT(a.data(), a.size()); }
64     int norm(int n) { return 1 << (___lg(n-1) + 1); }
65     vcp &dot(vcp &a, vcp &b) {
66         for (int i = 0; i < int(a.size()); ++i) {
67             a[i] = a[i] * b[i];
68         }
69         return a;
70     }
71     Poly operator+ (Poly a, Poly b) {
72         int maxlen = max(a.size(), b.size());
73         Poly ans(maxlen + 1);

```

```

74     a.resize(maxlen + 1);
75     b.resize(maxlen + 1);
76     for (int i = 0; i < maxlen; ++i) { ans[i] = a[i] + b[i]
77         ]; }
78     return ans;
79 }
80 Poly operator* (ll k, Poly a) {
81     Poly ans;
82     for (auto i: a) { ans.push_back(k*i); }
83     return ans;
84 }
85 Poly operator* (Poly a, Poly b) {
86     int n = a.size() + b.size() - 1;
87     vcp c(norm(n));
88     for (int i = 0; i < int(a.size()); ++i) { c[i].x = a[i]
89         ]; }
90     for (int i = 0; i < int(b.size()); ++i) { c[i].y = b[i]
91         ]; }
92     DFT(c);
93     dot(c, c);
94     IDFT(c);
95     a.resize(n);
96     for (int i = 0; i < n; ++i) {
97         a[i] = int(c[i].y * 0.5 + 0.5);
98     }
99     return a;
100 }
101 using namespace Polynomial;

```

4.12 多项式: NTT

```

1  #include <bits/stdc++.h>
2  #define fp(i, a, b) for (int i = (a), i##_ = (b) + 1; i < i##_
3      ; ++i)
4  #define fd(i, a, b) for (int i = (a), i##_ = (b) - 1; i > i##_
5      ; --i)
6  using namespace std;
7  const int maxn = 2e5 + 5, P = 998244353;
8  using ll = int64_t;
9  #define ADD(a, b) (((a) += (b)) >= P ? (a) -= P : 0)
10 #define SUB(a, b) (((a) -= (b)) < 0 ? (a) += P : 0)
11 #define MUL(a, b) ((a) * (b) % P)
12 int POW(ll a, int b = P - 2, ll x = 1) {
13     for (; b >= 1; a = a * a % P)
14         if (b & 1) x = x * a % P;
15     return x;
16 }
17 namespace NTT {
18     const int g = 3;
19     vector<int> Omega(int L) {
20         int wn = POW(g, P / L);
21         vector<int> w(L);
22         w[L >> 1] = 1;
23         fp(i, L / 2 + 1, L - 1) w[i] = MUL(w[i - 1], wn);
24         fd(i, L / 2 - 1, 1) w[i] = w[i << 1];
25         return w;
26     }
27     auto W = Omega(1 << 21);
28     void DIF(int *a, int n) {
29         for (int k = n >> 1; k; k >>= 1) {
30             for (int i = 0, y; i < n; i += k << 1)
31                 fp(j, 0, k - 1) y = a[i + j + k],
32                     a[i + j + k] = MUL(a[i + j] - y +
33                         P, W[k + j]),
34                     ADD(a[i + j], y);
35         }
36     }
37     void IDIT(int *a, int n) {
38         for (int k = 1; k < n; k <= 1) {
39             for (int i = 0, x, y; i < n; i += k << 1)
40                 fp(j, 0, k - 1) x = a[i + j], y = MUL(a[i + j + k]
41                     ], W[k + j]),
42                     a[i + j + k] = x - y < 0 ? x - y +
43                         P : x - y,
44                     ADD(a[i + j], y);
45         }
46     }
47     int Inv = P - (P - 1) / n;
48     fp(i, 0, n - 1) a[i] = MUL(a[i], Inv);
49     reverse(a + 1, a + n);
50 }
51 // namespace NTT

```

```

47 namespace Polynomial {
48     using Poly = std::vector<int>;
49     Poly &operator*=(Poly &a, int b) {
50         for (auto &x : a) x = MUL(x, b);
51         return a;
52     }
53     Poly operator*(Poly a, int b) { return a *= b; }
54     Poly operator*(int a, Poly b) { return b * a; }
55     void DFT(Poly &a) { NTT::DIF(a.data(), a.size()); }
56     void IDFT(Poly &a) { NTT::IDIT(a.data(), a.size()); }
57     int norm(int n) { return 1 << (32 - __builtin_clz(n - 1)); }
58     void norm(Poly &a) {
59         if (!a.empty()) a.resize(norm(a.size()), 0);
60     }
61     Poly &dot(Poly &a, Poly &b) {
62         fp(i, 0, a.size() - 1) a[i] = MUL(a[i], b[i]);
63         return a;
64     }
65     Poly operator*(Poly a, Poly b) {
66         int n = a.size() + b.size() - 1, L = norm(n);
67         if (a.size() <= 8 || b.size() <= 8) {
68             Poly c(n);
69             fp(i, 0, a.size() - 1) fp(j, 0, b.size() - 1) c[i + j]
70                 =
71                 (c[i + j] + (ll)a[i] * b[j]) % P;
72             return c;
73         }
74         a.resize(L), b.resize(L);
75         DFT(a), DFT(b), dot(a, b), IDFT(a);
76         return a.resize(n), a;
77     }
78 } // namespace Polynomial
79 using namespace Polynomial;

```

4.13 快速幂

```

1  int qmi(int m, int k, int p)
2  {
3      int res = 1 % p, t = m;
4      while (k)
5      {
6          if (k & 1) res = res * t % p;
7          t = t * t % p;
8          k >>= 1;
9      }
10     return res;
11 }

```

4.14 乘法逆元

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4
5  ll fpm(ll x, ll power, ll mod) {
6      x %= mod;
7      ll ans = 1;
8      for (; power; power >>= 1, (x *= x) %= mod)
9          if (power & 1) (ans *= x) %= mod;
10     return ans;
11 }
12 int main() {
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15     int n, p;
16     cin >> n >> p;
17     for (int a = 1; a < n; ++a) {
18         cout << fpm(a, p - 2, p) << endl;
19         //x为a在mod p意义下的逆元
20     }

```

4.15 组合数求法

```

1  #include <iostream>
2
3  using namespace std;
4
5  typedef long long LL;
6
7  LL qmi(LL a, int k, int p)
8  {

```

```

9      LL res = 1;
10     while(k)
11     {
12         if (k & 1) res = res * a % p;
13         k >>= 1;
14         a = a * a % p;
15     }
16     return res;
17 }
18
19 int C(int a,int b,int p)
20 {
21     if(a < b) return 0;
22
23     LL res = 1;
24     for(int i = 1, j = a; i <= b; i ++ , j -- )
25     {
26         res = res * j % p;
27         res = res * qmi(i, p - 2, p) % p;
28     }
29     return res;
30 }
31
32 LL lucas(LL a, LL b, int p)
33 {
34     if(a < p && b < p) return C(a, b, p);
35     return C(a % p, b % p, p) * lucas(a / p, b / p, p) % p;
36 }
37
38 int main()
39 {
40     int n;
41     cin >> n;
42     while(n --)
43     {
44         LL a,b;
45         int p;
46         cin >> a >> b >> p;
47         cout << lucas(a, b, p) << endl;
48     }
49     return 0;
50 }

```

4.16 线性基

```

1 //普通消元法
2 void insert(long long x)
3 {
4     for(int i=63;~i;i--) if((x>>i)&1ll)
5     {
6         if(!place[i]) {place[i]=x;return;}
7         else x^=place[i];
8     }
9 }
10 void insert(int *num)
11 {
12     for(int j=63;~j;j--)
13     {
14         int p=0;
15         for(int i=1;(!p)&&i<=n;i++)
16             if((!v[i])&&((num[i]>>j)&1ll)) p=i;
17         if(!p) {place[j]=0;continue;}
18         v[p]=true,place[j]=num[p];
19         for(int i=1;i<=n;i++) if(i!=p&&((num[i]>>j)&1ll))
20             num[i]^=num[p];
21     }
22 }
23 bool check(long long x)
24 {
25     for(int i=63;~i;i--)
26         if((x>>i)&1ll) x^=place[i];
27     return x==0;
28 }
29 struct Base
30 {
31     long long place[N];
32     void insert(long long x)
33     {
34         for(int i=63;~i;i--) if((x>>i)&1ll)
35         {
36             if(!place[i]) {place[i]=x;return;}
37             else x^=place[i];
38         }
39     }
40 }

```

```

39     }
40 };
41 void merge(Base A,Base &B)
42 {
43     for(int i=63;~i;i--) if(A.place[i])
44         B.insert(A.place[i]);
45 }
46 long long mx()
47 {
48     long long ans=0;
49     for(int i=63;~i;i--)
50         if((ans^place[i])>ans)
51             ans^=place[i];
52     return ans;
53 }
54 long long mn()
55 {
56     for(int i=0;i<63;i++)
57         if(place[i]) return place[i];
58     return -1;
59 }

```

4.17 矩阵快速幂及高斯消元

```

1 // 1.矩阵快速幂
2 #include <bits/stdc++.h>
3 using namespace std;
4 #define int long long
5 using ll = long long;
6 const int MOD = 1e9 + 7;
7 // 矩阵结构体
8 struct Matrix {
9     int n, m; // n: 行数, m: 列数
10     vector<vector<ll>> a;
11     Matrix(int _n, int _m) : n(_n), m(_m), a(_n + 1, vector<ll>
12         (>(_m + 1, 0)) {}
13
14     static Matrix identity(int n) {
15         Matrix I(n, n);
16         for (int i = 1; i <= n; ++i) {
17             I.a[i][i] = 1;
18         }
19         return I;
20     };
21 // 矩阵乘法: C = A * B
22 // 时间复杂度: O(n^3) 或 O(A.n * A.m * B.m)
23 Matrix operator*(const Matrix& A, const Matrix& B) {
24     Matrix C(A.n, B.m);
25     for (int i = 1; i <= A.n; ++i) {
26         for (int j = 1; j <= B.m; ++j) {
27             for (int k = 1; k <= A.m; ++k) {
28                 C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]
29                     ) % MOD;
30             }
31         }
32     }
33     return C;
34 }
35 // 矩阵快速幂: A^p, 时间复杂度: O(n^3 * Log p), A必须是方阵
36 Matrix power(Matrix A, ll p) {
37     Matrix res = Matrix::identity(A.n);
38     while (p) {
39         if (p & 1) res = res * A;
40         A = A * A;
41         p >>= 1;
42     }
43     return res;
44 }
45 signed main(){
46     ios::sync_with_stdio(false);
47     cin.tie(0), cout.tie(0);
48     int n, k; cin >> n >> k;
49     Matrix A(n, n);
50     for(int i = 1; i <= n; i++){
51         for(int j = 1; j <= n; j++){
52             cin >> A.a[i][j];
53         }
54     }
55     Matrix res = power(A, k);
56     for(int i = 1; i <= n; i++){
57         for(int j = 1; j <= n; j++){

```



```

58     cout << res.a[i][j] << " ";
59 }
60 cout << endl;
61 }
62 }

```

5 字符串

5.1 前缀函数与 KMP

```

1 vector<int> prefixFunction(string s)
2 { // fail : 1-index
3     int n = s.length();
4     vector<int> fail(n + 1);
5     for (int i = 1, j = 0; i < n; ++i) {
6         while (j && s[i] != s[j]) { j = fail[j]; }
7         j += (s[i] == s[j]);
8         fail[i + 1] = j;
9     }
10    return fail;
11 }
12 void kmp()
13 {
14     string s, t;
15     cin >> s >> t;
16     int n = s.length();
17     int m = t.length();
18     auto fail = prefixFunction(t + "#" + s);
19     for (int i = m + 2; i <= n + m + 1; ++i) {
20         if (fail[i] == m) {
21             cout << i - 2 * m << "\n";
22         }
23     }
24     for (int i = 1; i <= m; ++i) { cout << fail[i] << " \n"[i == m]; }
25 }

```

5.2 Manacher

```

1 vector<int> manacher(string s)
2 {
3     vector<int> d(s.length());
4     d[0] = 1;
5     int r=0, mid=0;
6     for (int i = 1; i < s.length()-1; ++i) { // 最后一个字符
7         // 防越界使用，无需访问
8         d[i] = 1;
9         if (r >= i) { d[i] = min(d[2*mid-i], r-i+1); }
10        while (s[i-d[i]] == s[i+d[i]]) { ++d[i]; } // 需要保证不会越界
11        if (i + d[i] > r) {
12            r = i + d[i] - 1;
13            mid = i;
14        }
15    }
16    return d;
17 }
18 void solve()
19 {
20     string s, t;
21     cin >> s;
22     for (auto c: s) {
23         t.push_back('#');
24         t.push_back(c);
25     }
26     t = "." + t + "#!"; // 首位添加两个不同的字符保证不会越界
27     vector<int> d=manacher(t);
28     cout << (*max_element(d.begin(), d.end()))-1 << endl;
29 }

```

5.3 字符串哈希

```

1 typedef pair<ll,ll> hs;
2 const ll MOD1=1e9+7, MOD2=1e9+9;
3 const hs p = { 117, 131 };
4 hs operator+ (hs a, hs b) {
5     return hs{ (a.first+b.first)%MOD1, (a.second+b.second)%MOD2 };
6 }
7 hs operator- (hs a, hs b) {
8     return hs{ (a.first-b.first+MOD1)%MOD1, (a.second-b.second

```

```

+MOD2)%MOD2 };
9 }
10 hs operator* (hs a, hs b) {
11     return hs{ a.first*b.first%MOD1, a.second*b.second%MOD2 };
12 }
13 struct Hash {
14     int n;
15     vector<hs> hs1, hs2, Pow;
16     Hash(string s) { // 0-index
17         n = s.length();
18         hs1.resize(n + 2);
19         hs2.resize(n + 2);
20         Pow.resize(n + 2);
21         Pow[0] = { 1LL, 1LL };
22         for (int i = 1; i <= n; ++i) {
23             Pow[i] = Pow[i-1] * p;
24         }
25         for (int i = 1; i <= n; ++i) {
26             hs1[i] = hs1[i-1] * p + hs{s[i-1]-'a'+1, s[i-1]-'a'+1};
27         }
28         for (int i = n; i; --i) {
29             hs2[i] = hs2[i+1] * p + hs{s[i-1]-'a'+1, s[i-1]-'a'+1};
30         }
31     }
32     hs get1(int l, int r) {
33         return hs1[r] - hs1[l-1] * Pow[r-l+1];
34     }
35     hs get2(int l, int r) {
36         return hs2[l] - hs2[r+1] * Pow[r-l+1];
37     }
38 };

```

5.4 AC 自动机

```

1 struct AhoCorasick {
2     static constexpr int ALPHABET = 26;
3     struct Node {
4         int len;
5         int link;
6         array<int,ALPHABET> next;
7         Node(): len{0}, link{0}, next{} { }
8     };
9
10    vector<Node> t;
11
12    AhoCorasick() {
13        init();
14    }
15
16    void init() {
17        t.assign(2, Node());
18        t[0].next.fill(1);
19        t[0].len = -1;
20    }
21
22    int newNode() {
23        t.emplace_back();
24        return t.size() - 1;
25    }
26
27    int add(const string &a) {
28        int p = 1;
29        for (auto c: a) {
30            int x = c - 'a';
31            if (t[p].next[x] == 0) {
32                t[p].next[x] = newNode();
33                t[t[p].next[x]].len = t[p].len + 1;
34            }
35            p = t[p].next[x];
36        }
37        return p;
38    }
39
40    void work() {
41        queue<int> q;
42        q.push(1);
43
44        while (!q.empty()) {
45            int x = q.front();
46            q.pop();
47

```

```

48     for (int i = 0; i < ALPHABET; ++i) {
49         if (t[x].next[i] == 0) {
50             t[x].next[i] = t[t[x].link].next[i];
51         } else {
52             t[t[x].next[i]].link = t[t[x].link].next[i];
53             q.push(t[x].next[i]);
54         }
55     }
56 }
57
58 int next(int p, int x) {
59     return t[p].next[x];
60 }
61
62 int link(int p) {
63     return t[p].link;
64 }
65
66 int len(int p) {
67     return t[p].len;
68 }
69
70 int size() {
71     return t.size();
72 }
73
74 };
75
76 constexpr int N = 2e6+5;
77 vector<int> G[N];
78 int cnt[N];
79
80 void dfs(int u)
81 {
82     for (auto v: G[u]) {
83         dfs(v);
84         cnt[u] += cnt[v];
85     }
86 }
87
88 void solve()
89 {
90     int n;
91     cin >> n;
92
93     AhoCorasick AC;
94     vector<int> pos(n + 1);
95     for (int i = 1; i <= n; ++i) {
96         string s;
97         cin >> s;
98         pos[i] = AC.add(s);
99     }
100     AC.work();
101
102     string T;
103     cin >> T;
104     int p = 1;
105     for (int i = 0; i < int(T.size()); ++i) {
106         int x = T[i] - 'a';
107         p = AC.next(p, x);
108         ++cnt[p];
109     }
110
111     for (int i = 1; i < AC.size(); ++i) {
112         G[AC.link(i)].push_back(i);
113     }
114     dfs(1);
115
116     for (int i = 1; i <= n; ++i) {
117         cout << cnt[pos[i]] << "\n";
118     }
119 }

```

5.5 后缀自动机 SAM

```

1 //edited by piaoyun from some other's code
2 //必须#define int long long
3
4 struct SAM {
5     static const int N=1e6+5, S=28;
6     int tot=1, last=1, link[N<<1], ch[N<<1][S], len[N<<1],
        endpos[N<<1];

```

```

7 // 总点数 tot, 点的 index 属于 [1-tot], 空串/根为 1
8 // last 为上一次插入的点
9 // link 为点的 parent 树父节点 / 最长 出现位置与自己不同的
    后缀
10 // ch[n][s] 指节点 n 末尾加字符 s 所转移到的点
11 // len 指该节点的串的最长长度, 注意到 最短长度 等于 len[
    link[n]] + 1 即父节点最长 + 1
12 // endpos[n] 参考 get_endpos() 的注释
13 void clear() {
14     for (int i = 0; i <= tot; ++i) {
15         link[i] = len[i] = endpos[i] = 0;
16         for (int k = 0; k < S; ++k) { ch[i][k] = 0; }
17     }
18     tot = 1;
19     last = 1;
20 }
21
22 // 延长一个字符, 通常为 [1-26]
23 void extend(int w) {
24     int p = ++tot, x = last, r, q;
25     endpos[p] = 1;
26     for (len[last=p]=len[x]+1; x&&!ch[x][w]; x=link[x]) {
27         ch[x][w]=p; }
28     if (!x) { link[p] = 1; }
29     else if (len[x] + 1 == len[q=ch[x][w]]) { link[p]=q; }
30     else {
31         link[r=++tot] = link[q];
32         memcpy(ch[r], ch[q], sizeof(ch[r]));
33         len[r] = len[x] + 1;
34         link[p] = link[q] = r;
35         for (; x && ch[x][w] == q; x = link[x]) { ch[x][w]
            = r; }
36     }
37 }
38
39 // 注意 vector 占用的空间
40 vector<int> p[N<<1]; //建立 parent 树, 以便从上到下 dfs
41 void dfs(int u) {
42     int v;
43     for (int i = 0; i < int(p[u].size()); ++i) {
44         v = p[u][i];
45         dfs(v);
46         endpos[u] += endpos[v];
47     }
48 }
49
50 //注意! 在使用该方法前, endpos[] 代表每个点作为“终结点”
    的次数
51 //使用该方法后, endpos[] 指在串中出现总次数, 即原数组的子树
    求和
52 void get_endpos() {
53     for (int i = 1; i <= tot; ++i) { p[i].clear(); }
54     for (int i = 2; i <= tot; ++i) {
55         p[link[i]].push_back(i);
56     }
57     dfs(1);
58     for (int i = 1; i <= tot; ++i) { p[i].clear(); }
59 }
60
61 // 在您不确定是否有抄写错误时再使用该方法
62 // 必须在输入任何数据前自检, 此前的数据会被清空
63 static const int STC = 998244353;
64 void self_test() {
65     clear();
66     for (int i = 1; i <= 1000; ++i) { extend(i * i % 26 +
        1); }
67     ll tmp = 107 * last + 301 * tot;
68     for (int i = 1; i <= tot; ++i) {
69         tmp = (tmp * 33 + link[i] * 101 + len[i] * 97) %
            STC;
70         for (int k = 1; k < S; ++k) { tmp = (tmp + k * ch[
            i][k]) % STC; }
71     }
72     assert("stage 1" && tmp == 393281314); // stage1 :
        检查建树是否正确
73     tmp = 0;
74     get_endpos();
75     for (int i = 1; i <= tot; ++i) { tmp = (tmp * 33 +
        endpos[i]) % STC; }
76     assert("stage 2" && tmp == 178417668); // stage2 :
        检查endpos计算是否正确, 如果您修改了endpos[]的含
        义则会报错
77     cout << "Self Test Passed. Remember to delete this
        function's use. \n";

```

```

77     clear();
78 }
79 // 调试时可调用
80 void debug_print() {
81     for (int i = 1; i <= tot; ++i) {
82         cerr << "node : " << i << " father : " << link[i]
83             << " endpos : " << endpos[i] << " len : " <<
84             len[i] << "\n";
85     }
86 }
87 ll solve() {
88     // 在这里输入你自己的解题逻辑
89     ll ans = 0;
90     get_endpos();
91     for (int i = 1; i <= tot; ++i) {
92         if (endpos[i] >= 2) { ans = max(ans, (ll)endpos[i]
93             *len[i]); }
94     }
95     return ans;
96 } sam;
97
98 void solve()
99 {
100     // sam.self_test();
101     string s; cin >> s;
102     for (auto c: s) { sam.extend(c-'a'+1); }
103     cout << sam.solve() << "\n";
104 }

```

5.6 KMP 算法

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 void get_nxt(string s, int nxt[]){
4     int j = 0;
5     nxt[0] = 0;
6     for (int i = 1; i < s.length(); i++){
7         while(j>0&&s[i]!=s[j]) j=nxt[j-1];
8         if(s[i]==s[j])j++;
9         nxt[i]=j;
10    }
11 }
12
13 void solve(){
14     int cnt=0;
15     string s, t;
16     cin >> s >> t;
17     int* nxt = new int[t.length()];
18     get_nxt(t, nxt);
19     int j=0;
20     for (int i = 0; i < s.length(); i++) //这里的s代表主串
21     {
22         while(j > 0 && s[i] != t[j]) j = nxt[j-1];
23         if(s[i]==t[j]) j++;
24         if(j==t.length()){
25             cout<<i-t.length()+2<<endl;
26         }
27     }
28     for (int i = 0; i < t.length(); i++){
29         cout<<nxt[i]<<' ';
30     }
31 }
32
33 int main(){
34     solve();
35 }

```

5.7 Trie 树

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 int tree[3000005][70],pass[3000005],endp[3000005],cnt=1;
4 int getidx(char ch){
5     // 映射函数
6     if(ch>='a'&&ch<='z') return ch-'a';
7     if(ch>='A'&&ch<='Z') return ch-'A'+26;
8     return ch-'0'+52;
9 }
10 void insert(string s){//插入操作
11     int cur=1;

```

```

12     pass[cur]++;
13     for(int i=0;i<s.length();i++){
14         int idx=getidx(s[i]);
15         if(tree[cur][idx]==0) {
16             tree[cur][idx]=++cnt;
17         }
18         cur=tree[cur][idx];
19         pass[cur]++;
20     }
21     endp[cur]++;
22 }
23 int search_whole_s(string s){
24     //查询整个字符串出现次数
25     int cur=1;
26     for(int i=0;i<s.length();i++){
27         int idx=getidx(s[i]);
28         if(tree[cur][idx]==0) return 0;
29         cur=tree[cur][idx];
30     }
31     return endp[cur];
32 }
33 int search_prefix_s(string s){
34     //查询s作为前缀字符串出现次数
35     int cur=1;
36     for(int i=0;i<s.length();i++){
37         int idx=getidx(s[i]);
38         if(tree[cur][idx]==0) return 0;
39         cur=tree[cur][idx];
40     }
41     return pass[cur];
42 }
43 void delete1(string s)
44 {
45     if(search_whole_s(s)>0)
46     {
47         int cur=1;
48         for(int i=0;i<s.length();i++)
49         {
50             int idx=getidx(s[i]);
51             if(--pass[tree[cur][idx]]==0)
52             {
53                 tree[cur][idx]=0;
54                 return;
55             }
56             cur=tree[cur][idx];
57         }
58         endp[cur]--;
59     }
60 }
61 void clear()
62 {
63     for(int i=1;i<=cnt;i++)
64     {
65         for(int j=0;j<=69;j++) tree[i][j]=0;
66         pass[i]=0;
67         endp[i]=0;
68     }
69     cnt=1;
70 }

```

5.8 string 常用函数

```

1 三、基本属性
2 s.size(); // 长度（常用）
3 s.length(); // 同 size()
4 s.empty(); // 是否为空
5 四、访问与修改
6 s[i]; // 访问（不检查越界）
7 s.at(i); // 访问（会检查越界）
8
9 s.front(); // 第一个字符
10 s.back(); // 最后一个字符
11 五、拼接与修改
12 1. 拼接
13 s += "abc";
14 s = s + "abc";
15 2. 插入
16 s.insert(2, "abc"); // 在下标2插入
17 3. 删除
18 s.erase(2, 3); // 从下标2开始删除3个字符
19 4. 替换
20 s.replace(2, 3, "abc"); // 替换区间
21 六、子串操作

```

```
22 string t = s.substr(pos, len); // 从pos开始, 长度len
```

示例:

```
26 string s = "abcdef";
27 s.substr(2, 3); // "cde"
28 七、查找操作 (重点)
29 1. 查找子串
30 s.find("abc"); // 返回第一次出现的位置
```

找不到返回:

```
34 string::npos
```

示例:

```
38 if (s.find("abc") != string::npos) {
39     // 找到了
40 }
```

2. 反向查找

```
42 s.rfind("abc"); // 从后往前找
```

3. 查找字符

```
44 s.find('a');
```

八、比较

```
46 if (s == t) {}
47 if (s < t) {} // 字典序比较
```

底层是字典序 (lexicographical order)

九、排序

```
52 #include <algorithm>
```

```
54 sort(s.begin(), s.end());
```

示例:

```
58 string s = "dcba";
59 sort(s.begin(), s.end()); // "abcd"
```

十、反转

```
61 reverse(s.begin(), s.end());
```

十一、大小写转换

```
63 #include <cctype>
```

```
65 for (char &c : s) c = toupper(c);
```

```
66 for (char &c : s) c = tolower(c);
```

十二、统计字符

```
68 int cnt = count(s.begin(), s.end(), 'a');
```

6 动态规划

6.1 数位 DP

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using db = long double;
5 using i128 = __int128;
6
7 const int L=18;
8 int a[L+5];
9 ll f[L+5][15];
10 ll dfs(int p, bool limit, bool lead0, int last)
11 {
12     if (!p) { return 1; }
13     auto& now = f[p][last];
14     if (!limit && !lead0 && ~now) {
15         return now;
16     }
17     int up = limit ? a[p] : 9;
18     ll res = 0;
19     for (int i = 0; i <= up; ++i) {
20         if (!lead0 && abs(last-i) < 2) { continue; }
21         res += dfs(p-1, limit&&(i==up), lead0&&(i==0), i);
22     }
23     if (!limit && !lead0) {
24         now = res;
25     }
26     return res;
27 }
28 ll query(ll x)
29 {
30     int len = 0;
31     for (; x; x /= 10) {
```

```
32     a[++len] = x % 10;
33 }
34 return dfs(len, true, true, 0);
35 }
36
37 void solve()
38 {
39     ll a, b;
40     cin >> a >> b;
41     cout << query(b) - query(a-1) << "\n";
42 }
43
44 int main()
45 {
46     ios::sync_with_stdio(0); cin.tie(0); cout.tie(0);
47     memset(f, -1, sizeof(f)); // 不要忘记设为 -1
48     int t = 1;
49     // cin >> t;
50     while (t--) { solve(); }
51 }
```

6.2 单调队列优化 (滑动窗口)

```
1 auto work = [&](auto&& cmp) -> void {
2     deque<pair<int,int>> q;
3
4     auto update = [&](int id, int x) -> void {
5         while (!q.empty() && cmp(x, q.back().second)) {
6             q.pop_back();
7         }
8         q.emplace_back(id, x);
9     };
10
11     for (int i = 1; i <= k; ++i) {
12         update(i, a[i]);
13     }
14     for (int i = 1, j = i+k-1; j <= n; ++i, ++j) {
15         while (!q.empty() && q.front().first < i) { q.pop_front(); }
16         update(j, a[j]);
17         cout << q.front().second << " \n"[j == n];
18     }
19 };
```

6.3 无向图上回路计数

```
1 // CF111D
2 for (int i = 1; i <= n; ++i) {
3     f[1<<(i-1)][i] = 1;
4 }
5 for (int s = 0; s < (1<<n); ++s) {
6     for (int i = 1; i <= n; ++i) {
7         if (!f[s][i]) { continue; }
8         for (int j = 1; j <= n; ++j) {
9             if (!G[i][j]) { continue; }
10            if ((s&s) > 1<<(j-1)) { continue; } // 起点不能改
11            if (s>>(j-1)&1) {
12                if ((s&s) == 1<<(j-1)) { // 走回起点
13                    ans += f[s][i];
14                }
15            } else {
16                f[s|(1<<(j-1))][j] += f[s][i];
17            }
18        }
19    }
20 }
21 cout << (ans-m)/2 << "\n"; // 去除无向边, 环正反走算一个
```

6.4 SOSDP

```
1 for(int k=0;k<=19;k++)
2     for(int i=0;i<=maxn;i++)
3         if(((i>>k)&1) g[i] += g[i^(1<<k)]; // i 是超集,
// 有第 k 位
4 for(int k=0;k<=19;k++)
5     for(int i=0;i<=maxn;i++)
6         if(!((i>>k)&1)) g[i] += g[i^(1<<k)]; // i 是子
// 集, 无第 k 位
```

6.5 树形 DP

```

1 #include<bits/stdc++.h>
2 //define int long long
3 #define in(a) a = read_int()
4 using namespace std;
5 const int N = 2e5 + 5;
6 const int inf = INT_MAX;
7 inline int read_int() {
8     int x = 0;
9     char ch = getchar();
10    bool f = 0;
11    while('9' < ch || ch < '0') f |= ch == '-', ch = getchar();
12    while('0' <= ch && ch <= '9') x = (x << 3) + (x << 1) + ch - '0', ch = getchar();
13    return f ? -x : x;
14 }
15 struct Edge {
16     int n, v, w;
17 } edge[N << 1];
18 int head[N], eid;
19 void eadd(int u, int v, int w) {
20     edge[++eid].n = head[u], edge[eid].v = v, edge[eid].w = w;
21 }
22 int n;
23 int l[N], r[N], ans[N];
24 void dfs(int u, int fa) { // 树形 DP 部分
25     l[u] = 1; // 初始化左边界
26     for(int i = head[u]; i; i = edge[i].n) {
27         int v = edge[i].v, w = edge[i].w;
28         if(v != fa) {
29             r[v] = w - 1; // 初始化右边界
30             dfs(v, u);
31             r[u] = min(r[u], w - l[v]); // 转移过程
32             l[u] = max(l[u], w - r[v]);
33         }
34     }
35 }
36 queue<int>q;
37 signed main() {
38     in(n);
39     for(int i = 1; i < n; i++) {
40         int u, v, w;
41         in(u), in(v), in(w);
42         eadd(u, v, w), eadd(v, u, w);
43     }
44     r[1] = 1e9; // 初始化根节点的右边界
45     dfs(1, 0); // 求每个点权的取值范围
46     for(int i = 1; i <= eid; i += 2) { // 判无解部分
47         int u = edge[i].v, v = edge[i + 1].v, w = edge[i].w;
48         if(r[u] + r[v] < w || l[u] > r[u] || l[v] > r[v]) {
49             cout << "NO";
50             return 0;
51         }
52     }
53     ans[1] = l[1]; // 若有解, 根节点点权取左边界
54     q.push(1);
55     while(!q.empty()) { // 广搜部分
56         int u = q.front();
57         q.pop();
58         for(int i = head[u]; i; i = edge[i].n) {
59             int v = edge[i].v, w = edge[i].w;
60             if(ans[v]) continue;
61             ans[v] = w - ans[u]; // 递推求解
62         }
63     }
64     cout << "YES\n"; // 打印答案
65     for(int i = 1; i <= n; i++) cout << ans[i] << ' ';
66     return 0;
67 }
68 /*
69 #include<bits/stdc++.h>
70 using namespace std;
71 const int maxn=10000;
72 int cnt=1,n,head[maxn],yes[maxn],no[maxn],num[maxn],b[maxn];
73 struct graph{
74     int next,to;
75 }g[maxn];
76 void add(int u,int v){
77     g[cnt].next=head[u];
78     g[cnt].to=v;
79     head[u]=cnt++;
80 }

```

```

81 }
82 void dfs(int i){
83     no[i]=0;
84     yes[i]=num[i];
85     for(int ei=head[i],v;ei;ei=g[ei].next){
86         v=g[ei].to;
87         dfs(v);
88         no[i]+=max(yes[v],no[v]);
89         yes[i]+=no[v];
90     }
91 }
92 int main(){
93     cin>>n;
94     for(int i=1;i<=n;i++) cin>>num[i];
95     for(int i=1;i<=n;i++){
96         int u,v;
97         cin>>u>>v;
98         b[u]=1;
99         add(v,u);
100     }
101     for(int i=1;i<=n;i++){
102         if(!b[i]){
103             dfs(i);
104             cout<<max(yes[i],no[i])<<endl;
105             break;
106         }
107     }
108 }
109 */

```

6.6 最长不下降子序列

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 const int N = 1e5+9;
5 int n,k,m,a[N],b[N],dp[N],t[N<<2];
6
7 void updata(int i,int l,int r,int n,int val){
8     t[i]=max(val,t[i]);
9     if(l==r) return;
10    int mid=(l+r)>>1;
11    if(n<=mid) updata(i<<1,l,mid,n,val);
12    else updata(i<<1|1,mid+1,r,n,val);
13    return;
14 }
15 int getmax(int i,int l,int r,int nl,int nr){
16     if(l==nl&&r==nr){
17         return t[i];
18     }
19     int mid=(l+r)>>1;
20     if(mid==nr) return getmax(i<<1,l,mid,nl,nr);
21     else if(mid<nl) return getmax(i<<1|1,mid+1,r,nl,nr);
22     else return max(getmax(i<<1,l,mid,nl,mid),getmax(i<<1|1,mid+1,r,mid+1,nr));
23 }
24 void solve(){
25     cin>>n>>k;
26     for(int i=1;i<=n;i++)cin>>a[i],b[i]=a[i];
27     // 离散化
28     sort(b+1,b+n+1);
29     m=unique(b+1,b+n+1)-b-1;
30     for(int i=1;i<=n;i++)a[i]=lower_bound(b+1,b+m+1,a[i])-b;
31     for(int i=1;i<=n;i++){
32         dp[i]=getmax(1,1,m,1,a[i])+1;
33         updata(1,1,m,a[i],dp[i]);
34     }
35     //到这里dp 中存储的是以a[i]结尾的最长不下降子序列
36     int ans=0;
37     memset(t,0,sizeof t);
38     for(int i=n;i>k;i--){
39         ans=max(ans,dp[i-k]+k-1+getmax(1,1,m,a[i-k],m)+1); //
40         // 通过再次利用线段树找到往后最长 (题目要求)
41         int tem=getmax(1,1,m,a[i],m)+1;
42         updata(1,1,m,a[i],tem);
43     }
44     ans=max(ans,k+getmax(1,1,m,a[k+1],m));
45     cout<<ans<<'\n';
46 }
47 // 1 2 3 ... i-k i-k+1 i-k+2 i-k+3 ... i ...
48

```

```

49 int main(){
50     ios::sync_with_stdio(false), cin.tie(nullptr), cout.tie(
        nullptr);
51     int q=1;
52     //cin>>q;
53     while(q--){
54         solve();
55     }
56     return 0;
57 }

```

7 计算几何

7.1 向量

```

1 using T = ll;
2 struct Point {
3     T x, y;
4     Point(T x=0, T y=0): x(x), y(y) {}
5     friend istream& operator>> (istream& is, Point& p) { return
        is>>p.x>>p.y; }
6     friend ostream& operator<< (ostream& os, Point& p) { return
        os<<p.x<<" "<<p.y; }
7     db ang() { return atan2(y, x); }
8     bool operator== (const Point &p) const { return sgn(x-p.x)
        ==0 && sgn(y-p.y)==0; }
9     bool operator< (const Point &p) const { return x<p.x || (x==
        p.x && y<p.y); }
10    Point operator+ (const Point &p) const { return Point(x+p.x,
        y+p.y); }
11    Point operator- (const Point &p) const { return Point(x-p.
        x, y-p.y); }
12    Point operator* (const T &k) const { return Point(k*x, k*y);
        }
13    Point operator/ (const T &k) const { return Point(x/k, y/k);
        }
14    T operator* (const Point &p) const { return x*p.x + y*p.y; }
15    T operator^ (const Point &p) const { return x*p.y - y*p.x; }
        // 叉乘时打括号
16    T len2() { return (*this)*(*this); }
17    db len() { return sqrt1(len2()); } // 等价于 hypotl(x, y)
18    Point rot(db ang) { return Point(x*cos(ang)-y*sin(ang), x*
        sin(ang)+y*cos(ang)); }
19    Point trunc(db l) { return (*this) * (l/len()); }
20 };
21 using Vector = Point;
22
23 // 判断 c 是否在 ab 的逆时针方向
24 int toLeft(Point a, Point b, Point c) { return sgn((b-a)^(c-a)
        ); }
25
26 // 二维向量夹角
27 db getAngle(Vector a, Vector b) { return fabs(atan2(fabs(a^b),
        a*b)); }

```

7.2 点线操作

```

1 struct Line {
2     Point a, b;
3     Vector v;
4     Line(Point a, Point b): a(a), b(b) { v = b - a; }
5     int toLeft(Point p) { return sgn(v^(p-a)); }
6     bool onSegment(Point p) { return (toLeft(p)==0) && ((p-a)*(p
        -b)<=0); }
7     db distToLine(Point p) { return fabs((v^(p-a))/v.len()); }
8     Point proj(Point p) { return a+v*((v*(p-a))/(v*v)); }
9     // 用整数操作判断直线与点 c 的距离是否小于 (等于) 某个值 r
10    // 常见于判断直线与圆的位置关系
11    // 注意叉乘再相乘可能会爆 ll, 所以这里用 __int128
12    bool disLess(Point p, int128 r) {
13        int128 xmult = (v^(p-a));
14        return xmult*xmult <= r*r*v.len2();
15    }
16 };
17
18 // 判断两条直线是否平行
19 bool isParallel(Line l1, Line l2)
20 {
21     return sgn(l1.v^l2.v)==0 && sgn(l1.v^(l1.a-l2.a))!=0;
22 }
23
24 // 判断两条线段是否相交

```

```

25 // 先做两次跨立实验判断是否规范相交，再特判至少三点共线的情况
26 bool hasIntersection(Line L1, Line L2)
27 {
28     // 特判三点共线的情况
29     if (L1.onSegment(L2.a)) { return true; }
30     if (L1.onSegment(L2.b)) { return true; }
31     if (L2.onSegment(L1.a)) { return true; }
32     if (L2.onSegment(L1.b)) { return true; }
33     // 跨立实验判断两直线是否规范相交
34     // 线段端点分布在直线两侧，“线段与线段/线段与直线/直线与
        直线”是否相交对应修改即可
35     function<bool(Line,Point,Point)> crossStanding = [](Line L,
        Point a, Point b){
36         return L.toLeft(a) * L.toLeft(b) == -1;
37     };
38     return crossStanding(L1, L2.a, L2.b) && crossStanding(L2,
        L1.a, L1.b);
39 }
40
41 // 求两直线交点
42 // 求之前应该先判断有没有交点，避免精度误差，同时还要特判共线
        的情况
43 Point getIntersection(Line L1, Line L2)
44 {
45     Point P1=L1.a, P2=L2.a;
46     Point v1=L1.v, v2=L2.v;
47     return P1 + v1 * ( (v2^(P1-P2)) / (v1^v2) );
48 }
49
50 // 点P到线段AB的距离公式
51 double distToSeg(Point p, Point a, Point b) {
52     if(a == b) return (p-a).len();
53     Point v1=b-a, v2=p-a, v3=p-b;
54     if(sgn(v1*v2) < 0) return v2.len();
55     if(sgn(v1*v3) > 0) return v3.len();
56     return Line(a,b).distToLine(p);
57 }

```

7.3 多边形与凸包

```

1 struct Polygon: vector<Point> {
2     using vector<Point>::vector; // 直接使用 vector 的构造函数
        数
3     size_t nxt(size_t i) { return i+1==size()? 0: i+1; }
4     size_t pre(size_t i) { return i==0? size()-1: i-1; }
5     // 求多边形面积
6     // 为避免精度误差，可以返回 2*S，即去掉 "0.5*"
7     db area() {
8         db res = 0;
9         for (size_t i = 0; i < size(); ++i) {
10             res += 0.5 * ((*this)[i] ^ (*this)[nxt(i)]);
11         }
12         return fabs(res);
13     }
14     // Sunday's algorithm 光线回转法
15     // 在整数范围内判断一个点是否在多边形（不保证凸包）内部
16     // 亦可用于求某点相对于该多边形的回转数
17     pair<bool,int> winding(Point p) {
18         int wn = 0, n = this->size();
19         for (int i = 0; i < n; ++i) {
20             if (Line((*this)[i], (*this)[(i+1)%n]).onSegment(p)
                ) {
21                 return { true, -inf };
22             }
23             int k = Line((*this)[i], (*this)[(i+1)%n]).toLeft(
                p);
24             int d1 = sgn((*this)[i].y - p.y);
25             int d2 = sgn((*this)[(i+1)%n].y - p.y);
26             if (k>0 && d1<=0 && d2>0) { wn++; }
27             if (k<0 && d2<=0 && d1>0) { wn--; }
28         }
29         return { wn!=0, wn };
30     }
31 };

```

7.4 计算几何模板

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using point_t=long double; //全局数据类型，可修改为 long Long
        等

```

```

5
6 constexpr point_t eps=1e-8;
7 constexpr long double PI=3.14159265358979323841;
8
9 // 点与向量
10 template<typename T> struct point
11 {
12     T x,y;
13
14     bool operator==(const point &a) const {return (abs(x-a.x)
15         <=eps && abs(y-a.y)<=eps);}
16     bool operator<(const point &a) const {if (abs(x-a.x)<=eps)
17         return y<a.y-eps; return x<a.x-eps;}
18     bool operator>(const point &a) const {return !(*this<a ||
19         *this==a);}
20     point operator+(const point &a) const {return {x+a.x,y+a.y
21         };}
22     point operator-(const point &a) const {return {x-a.x,y-a.y
23         };}
24     point operator-() const {return {-x,-y};}
25     point operator*(const T k) const {return {k*x,k*y};}
26     point operator/(const T k) const {return {x/k,y/k};}
27     T operator*(const point &a) const {return x*a.x+y*a.y;}
28     // 点积
29     T operator^(const point &a) const {return x*a.y-y*a.x;}
30     // 叉积, 注意优先级
31     int toleft(const point &a) const {const auto t=(*this)^a;
32         return (t>eps)-(t<-eps);} // to-left 测试
33     T len2() const {return (*this)*(*this);} // 向量长度的平方
34     T dis2(const point &a) const {return (a-(*this)).len2();}
35     // 两点距离的平方
36
37     // 涉及浮点数
38     long double len() const {return sqrt1(len2());} // 向量长度
39     long double dis(const point &a) const {return sqrt1(dis2(a
40         ));} // 两点距离
41     long double ang(const point &a) const {return acos1(max
42         (-1.01,min(1.01,(((*this)*a)/(len()*a.len()))));} //
43         向量夹角
44     point rot(const long double rad) const {return {x*cos(rad)
45         -y*sin(rad),x*sin(rad)+y*cos(rad);} // 逆时针旋转
46         (给定角度)
47     point rot(const long double cosr,const long double sinr)
48         const {return {x*cosr-y*sinr,x*sinr+y*cosr;} // 逆
49         时针旋转 (给定角度的正弦与余弦)
50
51 };
52
53 using Point=point<point_t>;
54
55 // 极角排序
56 struct argcmp
57 {
58     bool operator()(const Point &a,const Point &b) const
59     {
60         const auto quad=[](const Point &a)
61         {
62             if (a.y<-eps) return 1;
63             if (a.y>eps) return 4;
64             if (a.x<-eps) return 5;
65             if (a.x>eps) return 3;
66             return 2;
67         };
68         const int qa=quad(a),qb=quad(b);
69         if (qa!=qb) return qa<qb;
70         const auto t=a^b;
71         return t>eps;
72     }
73 };
74
75 // 直线
76 template<typename T> struct line
77 {
78     point<T> p,v; // p 为直线上一点, v 为方向向量
79
80     bool operator==(const line &a) const {return v.toleft(a.v)
81         ==0 && v.toleft(p-a.p)==0;}
82     int toleft(const point<T> &a) const {return v.toleft(a-p)
83         };} // to-left 测试
84     bool operator<(const line &a) const // 半平面交算法定义的
85         排序
86     {
87         if (abs(v^a.v)<=eps && v*a.v>=-eps) return toleft(a.p)

```

```

68         ==-1;
69         return argcmp()(v,a.v);
70     }
71
72     // 涉及浮点数
73     point<T> inter(const line &a) const {return p+v*((a.v^(p-a
74         .p))/(v^a.v));} // 直线交点
75     long double dis(const point<T> &a) const {return abs(v^(a-
76         p))/v.len();} // 点到直线距离
77     point<T> proj(const point<T> &a) const {return p+v*((v*(a-
78         p))/(v*v));} // 点在直线上的投影
79 };
80
81 using Line=line<point_t>;
82
83 // 线段
84 template<typename T> struct segment
85 {
86     point<T> a,b;
87
88     bool operator<(const segment &s) const {return make_pair(a
89         ,b)<make_pair(s.a,s.b);}
90
91     // 判定性函数建议在整数域使用
92
93     // 判断点是否在线段上
94     // -1 点在线段端点 / 0 点不在线段上 / 1 点严格在线段上
95     int is_on(const point<T> &p) const
96     {
97         if (p==a || p==b) return -1;
98         return (p-a).toleft(p-b)==0 && (p-a)*(p-b)<-eps;
99     }
100
101     // 判断线段直线是否相交
102     // -1 直线经过线段端点 / 0 线段和直线不相交 / 1 线段和直线
103         严格相交
104     int is_inter(const line<T> &l) const
105     {
106         if (l.toleft(a)==0 || l.toleft(b)==0) return -1;
107         return l.toleft(a)!=l.toleft(b);
108     }
109
110     // 判断两线段是否相交
111     // -1 在某一线段端点处相交 / 0 两线段不相交 / 1 两线段严格
112         相交
113     int is_inter(const segment<T> &s) const
114     {
115         if (is_on(s.a) || is_on(s.b) || s.is_on(a) || s.is_on(
116             b)) return -1;
117         const line<T> l{a,b-a},ls{s.a,s.b-s.a};
118         return l.toleft(s.a)*l.toleft(s.b)==-1 && ls.toleft(a)
119             *ls.toleft(b)==-1;
120     }
121
122     // 点到线段距离
123     long double dis(const point<T> &p) const
124     {
125         if ((p-a)*(b-a)<-eps || (p-b)*(a-b)<-eps) return min(p
126             .dis(a),p.dis(b));
127         const line<T> l{a,b-a};
128         return l.dis(p);
129     }
130
131     // 两线段间距离
132     long double dis(const segment<T> &s) const
133     {
134         if (is_inter(s)) return 0;
135         return min({dis(s.a),dis(s.b),s.dis(a),s.dis(b)});
136     }
137 };
138
139 using Segment=segment<point_t>;
140
141 // 多边形
142 template<typename T> struct polygon
143 {
144     vector<point<T>> p; // 以逆时针顺序存储
145
146     size_t nxt(const size_t i) const {return i==p.size()-1?0:i
147         +1;}
148     size_t pre(const size_t i) const {return i==0?p.size()-1:i
149         -1;}
150
151     // 回转数

```

```

140 // 返回值第一项表示点是否在多边形边上
141 // 对于狭义多边形, 回转数为 0 表示点在多边形外, 否则点在多
    边形内
142 pair<bool,int> winding(const point<T> &a) const
143 {
144     int cnt=0;
145     for (size_t i=0;i<p.size();i++)
146     {
147         const point<T> u=p[i],v=p[nxt(i)];
148         if (abs((a-u)^(a-v))<=eps && (a-u)*(a-v)<=eps)
149             return {true,0};
150         if (abs(u.y-v.y)<=eps) continue;
151         const Line uv={u,v-u};
152         if (u.y<v.y-eps && uv.toleft(a)<=0) continue;
153         if (u.y>v.y+eps && uv.toleft(a)>=0) continue;
154         if (u.y<a.y-eps && v.y>a.y-eps) cnt++;
155         if (u.y>a.y-eps && v.y<a.y-eps) cnt--;
156     }
157     return {false,cnt};
158 }
159 // 多边形面积的两倍
160 // 可用于判断点的存储顺序是顺时针或逆时针
161 T area() const
162 {
163     T sum=0;
164     for (size_t i=0;i<p.size();i++) sum+=p[i]^p[nxt(i)];
165     return sum;
166 }
167 // 多边形的周长
168 long double circ() const
169 {
170     long double sum=0;
171     for (size_t i=0;i<p.size();i++) sum+=p[i].dis(p[nxt(i)
        ]);
172     return sum;
173 }
174 };
175 };
176
177 using Polygon=polygon<point_t>;
178
179 //凸多边形
180 template<typename T> struct convex: polygon<T>
181 {
182     // 闵可夫斯基和
183     convex operator+(const convex &c) const
184     {
185         const auto &p=this->p;
186         vector<Segment> e1(p.size()),e2(c.p.size()),edge(p.
            size()+c.p.size());
187         vector<point<T>> res; res.reserve(p.size()+c.p.size())
            ;
188         const auto cmp=[&](const Segment &u,const Segment &v) {
189             return argcmp()(u.b-u.a,v.b-v.a);};
190         for (size_t i=0;i<p.size();i++) e1[i]={p[i],p[this->
            nxt(i)}];
191         for (size_t i=0;i<c.p.size();i++) e2[i]={c.p[i],c.p[c.
            nxt(i)}];
192         rotate(e1.begin(),min_element(e1.begin(),e1.end(),cmp)
            ,e1.end());
193         rotate(e2.begin(),min_element(e2.begin(),e2.end(),cmp)
            ,e2.end());
194         merge(e1.begin(),e1.end(),e2.begin(),e2.end(),edge.
            begin(),cmp);
195         const auto check=[&](const vector<point<T>> &res,const
            point<T> &u)
196         {
197             const auto back1=res.back(),back2=*prev(res.end()
                ,2);
198             return (back1-back2).toleft(u-back1)==0 && (back1-
                back2)*(u-back1)>=eps;
199         };
200         auto u=e1[0].a+e2[0].a;
201         for (const auto &v:edge)
202         {
203             while (res.size()>1 && check(res,u)) res.pop_back
                ();
204             res.push_back(u);
205             u=u+v.b-v.a;
206         }
207         if (res.size()>1 && check(res,res[0])) res.pop_back();
208         return {res};
209     }
210 }

```

```

209 // 旋转卡壳
210 // func 为更新答案的函数, 可以根据题目调整位置
211 template<typename F> void rotcaliper(const F &func) const
212 {
213     const auto &p=this->p;
214     const auto area=[&](const point<T> &u,const point<T> &v
        ,const point<T> &w){return (w-u)^(w-v);};
215     for (size_t i=0,j=1;i<p.size();i++)
216     {
217         const auto nxti=this->nxt(i);
218         func(p[i],p[nxti],p[j]);
219         while (area(p[this->nxt(j)],p[i],p[nxti])>=area(p[
            j],p[i],p[nxti]))
220         {
221             j=this->nxt(j);
222             func(p[i],p[nxti],p[j]);
223         }
224     }
225 }
226
227 // 凸多边形的直径的平方
228 T diameter2() const
229 {
230     const auto &p=this->p;
231     if (p.size()==1) return 0;
232     if (p.size()==2) return p[0].dis2(p[1]);
233     T ans=0;
234     auto func=[&](const point<T> &u,const point<T> &v,
        const point<T> &w){ans=max({ans,w.dis2(u),w.dis2(
            v)});};
235     rotcaliper(func);
236     return ans;
237 }
238
239 // 判断点是否在凸多边形内
240 // 复杂度 O(Logn)
241 // -1 点在多边形边上 | 0 点在多边形外 | 1 点在多边形内
242 int is_in(const point<T> &a) const
243 {
244     const auto &p=this->p;
245     if (p.size()==1) return a==p[0]?-1:0;
246     if (p.size()==2) return segment<T>{p[0],p[1]}.is_on(a)
        ?-1:0;
247     if (a==p[0]) return -1;
248     if ((p[1]-p[0]).toleft(a-p[0])==-1 || (p.back()-p[0]).
        toleft(a-p[0])==1) return 0;
249     const auto cmp=[&](const Point &u,const Point &v){
250         return (u-p[0]).toleft(v-p[0])==1;};
251     const size_t i=lower_bound(p.begin()+1,p.end(),a,cmp)-
        p.begin();
252     if (i==1) return segment<T>{p[0],p[i]}.is_on(a)?-1:0;
253     if (i==p.size()-1 && segment<T>{p[0],p[i]}.is_on(a))
        return -1;
254     if (segment<T>{p[i-1],p[i]}.is_on(a)) return -1;
255     return (p[i]-p[i-1]).toleft(a-p[i-1])>0;
256 }
257
258 // 凸多边形关于某一方向的极点
259 // 复杂度 O(Logn)
260 // 参考资料: https://codeforces.com/blog/entry/48868
261 template<typename F> size_t extreme(const F &dir) const
262 {
263     const auto &p=this->p;
264     const auto check=[&](const size_t i){return dir(p[i]).
        toleft(p[this->nxt(i)]-p[i])>=0;};
265     const auto dir0=dir(p[0]); const auto check0=check(0);
266     if (!check0 && check(p.size()-1)) return 0;
267     const auto cmp=[&](const Point &v)
268     {
269         const size_t vi=&v-p.data();
270         if (vi==0) return 1;
271         const auto checkv=check(vi);
272         const auto t=dir0.toleft(v-p[0]);
273         if (vi==1 && checkv==check0 && t==0) return 1;
274         return checkv^(checkv==check0 && t<=0);
275     };
276     return partition_point(p.begin(),p.end(),cmp)-p.begin
        ();
277 }
278
279 // 过凸多边形外一点求凸多边形的切线, 返回切点下标
280 // 复杂度 O(Logn)
281 // 必须保证点在多边形外

```



```

282 pair<size_t, size_t> tangent(const point<T> &a) const
283 {
284     const size_t i=extreme([&](const point<T> &u){return u
285         -a;});
286     const size_t j=extreme([&](const point<T> &u){return a
287         -u;});
288     return {i,j};
289 }
290 // 求平行于给定直线的凸多边形的切线，返回切点下标
291 // 复杂度  $O(\log n)$ 
292 pair<size_t, size_t> tangent(const line<T> &a) const
293 {
294     const size_t i=extreme([&](...){return a.v;});
295     const size_t j=extreme([&](...){return -a.v;});
296     return {i,j};
297 }
298
299 using Convex=convex<point_t>;
300
301 // 圆
302 struct Circle
303 {
304     Point c;
305     long double r;
306
307     bool operator==(const Circle &a) const {return c==a.c &&
308         abs(r-a.r)<=eps;}
309     long double circ() const {return 2*PI*r;} // 周长
310     long double area() const {return PI*r*r;} // 面积
311
312     // 点与圆的关系
313     // -1 圆上 / 0 圆外 / 1 圆内
314     int is_in(const Point &p) const {const long double d=p.dis
315         (c); return abs(d-r)<=eps?-1:d<r-eps;}
316
317     // 直线与圆关系
318     // 0 相离 / 1 相切 / 2 相交
319     int relation(const Line &l) const
320     {
321         const long double d=l.dis(c);
322         if (d>r+eps) return 0;
323         if (abs(d-r)<=eps) return 1;
324         return 2;
325     }
326
327     // 圆与圆关系
328     // -1 相同 / 0 相离 / 1 外切 / 2 相交 / 3 内切 / 4 内含
329     int relation(const Circle &a) const
330     {
331         if (*this==a) return -1;
332         const long double d=c.dis(a.c);
333         if (d>r+a.r+eps) return 0;
334         if (abs(d-r-a.r)<=eps) return 1;
335         if (abs(d-abs(r-a.r))<=eps) return 3;
336         if (d<abs(r-a.r)-eps) return 4;
337         return 2;
338     }
339
340     // 直线与圆的交点
341     vector<Point> inter(const Line &l) const
342     {
343         const long double d=l.dis(c);
344         const Point p=l.proj(c);
345         const int t=relation(l);
346         if (t==0) return vector<Point>();
347         if (t==1) return vector<Point>{p};
348         const long double k=sqrt(r*r-d*d);
349         return vector<Point>{p-(l.v/l.v.len())*k,p+(l.v/l.v.
350             len())*k};
351     }
352
353     // 圆与圆交点
354     vector<Point> inter(const Circle &a) const
355     {
356         const long double d=c.dis(a.c);
357         const int t=relation(a);
358         if (t==1 || t==4) return vector<Point>();
359         Point e=a.c-c; e=e/e.len()*r;
360         if (t==1 || t==3)
361         {
362             if (r*r+d*d-a.r*a.r>=-eps) return vector<Point>{c+
363                 e};

```

```

364             return vector<Point>{c-e};
365         }
366         const long double costh=(r*r+d*d-a.r*a.r)/(2*r*d),
367             sinh=sqrt(1-costh*costh);
368         return vector<Point>{c+e.rot(costh,-sinh),c+e.rot(
369             costh,sinh)};
370     }
371
372     // 圆与圆交面积
373     long double inter_area(const Circle &a) const
374     {
375         const long double d=c.dis(a.c);
376         const int t=relation(a);
377         if (t==1) return area();
378         if (t<2) return 0;
379         if (t>2) return min(area(),a.area());
380         const long double costh1=(r*r+d*d-a.r*a.r)/(2*r*d),
381             costh2=(a.r*a.r+d*d-r*r)/(2*a.r*d);
382         const long double sinh1=sqrt(1-costh1*costh1),sinh2=
383             sqrt(1-costh2*costh2);
384         const long double th1=acos(costh1),th2=acos(costh2);
385         return r*r*(th1-costh1*sinh1)+a.r*a.r*(th2-costh2*
386             sinh2);
387     }
388
389     // 过圆外一点圆的切线
390     vector<Line> tangent(const Point &a) const
391     {
392         const int t=is_in(a);
393         if (t==1) return vector<Line>();
394         if (t==1)
395         {
396             const Point v=-(a-c).y,{a-c}.x;
397             return vector<Line>{{a,v}};
398         }
399         Point e=a-c; e=e/e.len()*r;
400         const long double costh=r/c.dis(a),sinh=sqrt(1-costh*
401             costh);
402         const Point t1=c+e.rot(costh,-sinh),t2=c+e.rot(costh,
403             sinh);
404         return vector<Line>{{a,t1-a},{a,t2-a}};
405     }
406
407     // 两圆的公切线
408     vector<Line> tangent(const Circle &a) const
409     {
410         const int t=relation(a);
411         vector<Line> lines;
412         if (t==1 || t==4) return lines;
413         if (t==1 || t==3)
414         {
415             const Point p=inter(a)[0],v=-(a.c-c).y,{a.c-c}.x
416                 };
417             lines.push_back({p,v});
418         }
419         const long double d=c.dis(a.c);
420         const Point e=(a.c-c)/(a.c-c).len();
421         if (t<=2)
422         {
423             const long double costh=(r-a.r)/d,sinh=sqrt(1-
424                 costh*costh);
425             const Point d1=e.rot(costh,-sinh),d2=e.rot(costh,
426                 sinh);
427             const Point u1=c+d1*r,u2=c+d2*r,v1=a.c+d1*a.r,v2=a
428                 .c+d2*a.r;
429             lines.push_back({u1,v1-u1}); lines.push_back({u2,
430                 v2-u2});
431         }
432         if (t==0)
433         {
434             const long double costh=(r+a.r)/d,sinh=sqrt(1-
435                 costh*costh);
436             const Point d1=e.rot(costh,-sinh),d2=e.rot(costh,
437                 sinh);
438             const Point u1=c+d1*r,u2=c+d2*r,v1=a.c-d1*a.r,v2=a
439                 .c-d2*a.r;
440             lines.push_back({u1,v1-u1}); lines.push_back({u2,
441                 v2-u2});
442         }
443         return lines;
444     }
445
446     // 圆的反演
447     tuple<int,Circle,Line> inverse(const Line &l) const

```

```

428 {
429     const Circle null_c={{0.0,0.0},0.0};
430     const Line null_l={{0.0,0.0},{0.0,0.0}};
431     if (l.toleft(c)==0) return {2,null_c,l};
432     const Point v=l.toleft(c)==1?Point{1.v.y,-1.v.x}:Point
433         {-1.v.y,1.v.x};
434     const long double d=r*r/1.dis(c);
435     const Point p=c+v/v.len()*d;
436     return {1,((c+p)/2,d/2),null_l};
437 }
438 tuple<int,Circle,Line> inverse(const Circle &a) const
439 {
440     const Circle null_c={{0.0,0.0},0.0};
441     const Line null_l={{0.0,0.0},{0.0,0.0}};
442     const Point v=a.c-c;
443     if (a.is_in(c)==-1)
444     {
445         const long double d=r*r/(a.r+a.r);
446         const Point p=c+v/v.len()*d;
447         return {2,null_c,{p,{-v.y,v.x}}};
448     }
449     if (c==a.c) return {1,{c,r*r/a.r},null_l};
450     const long double d1=r*r/(c.dis(a.c)-a.r),d2=r*r/(c.
451         dis(a.c)+a.r);
452     const Point p=c+v/v.len()*d1,q=c+v/v.len()*d2;
453     return {1,((p+q)/2,p.dis(q)/2),null_l};
454 }
455 };
456 // 圆与多边形面积交
457 long double area_inter(const Circle &circ,const Polygon &poly)
458 {
459     const auto cal=[](const Circle &circ,const Point &a,const
460         Point &b)
461     {
462         if ((a-circ.c).toleft(b-circ.c)==0) return 0.01;
463         const auto ina=circ.is_in(a),inb=circ.is_in(b);
464         const Line ab={a,b-a};
465         if (ina && inb) return ((a-circ.c)^(b-circ.c))/2;
466         if (ina && !inb)
467         {
468             const auto t=circ.inter(ab);
469             const Point p=t.size()==1?t[0]:t[1];
470             const long double ans=((a-circ.c)^(p-circ.c))/2;
471             const long double th=(p-circ.c).ang(b-circ.c);
472             const long double d=circ.r*circ.r*th/2;
473             if ((a-circ.c).toleft(b-circ.c)==1) return ans+d;
474             return ans-d;
475         }
476         if (!ina && inb)
477         {
478             const Point p=circ.inter(ab)[0];
479             const long double ans=((p-circ.c)^(b-circ.c))/2;
480             const long double th=(a-circ.c).ang(p-circ.c);
481             const long double d=circ.r*circ.r*th/2;
482             if ((a-circ.c).toleft(b-circ.c)==1) return ans+d;
483             return ans-d;
484         }
485         const auto p=circ.inter(ab);
486         if (p.size()==2 && Segment{a,b}.dis(circ.c)<=circ.r+
487             eps)
488         {
489             const long double ans=((p[0]-circ.c)^(p[1]-circ.c)
490                 )/2;
491             const long double th1=(a-circ.c).ang(p[0]-circ.c),
492                 th2=(b-circ.c).ang(p[1]-circ.c);
493             const long double d1=circ.r*circ.r*th1/2,d2=circ.r
494                 *circ.r*th2/2;
495             if ((a-circ.c).toleft(b-circ.c)==1) return ans+d1+
496                 d2;
497             return ans-d1-d2;
498         }
499         const long double th=(a-circ.c).ang(b-circ.c);
500         if ((a-circ.c).toleft(b-circ.c)==1) return circ.r*circ
501             .r*th/2;
502         return -circ.r*circ.r*th/2;
503     };
504     long double ans=0;
505     for (size_t i=0;i<poly.p.size();i++)
506     {
507         const Point a=poly.p[i],b=poly.p[poly.nxt(i)];
508         ans+=cal(circ,a,b);
509     }
510 }

```

```

503 }
504     return ans;
505 }
506 // 点集的凸包
507 // Andrew 算法, 复杂度  $O(n\log n)$ 
508 Convex convexhull(vector<Point> p)
509 {
510     vector<Point> st;
511     if (p.empty()) return Convex{st};
512     sort(p.begin(),p.end());
513     const auto check=[](const vector<Point> &st,const Point &u
514         )
515     {
516         const auto back1=st.back(),back2=*prev(st.end(),2);
517         return (back1-back2).toleft(u-back1)<=0;
518     };
519     for (const Point &u:p)
520     {
521         while (st.size()>1 && check(st,u)) st.pop_back();
522         st.push_back(u);
523     }
524     size_t k=st.size();
525     p.pop_back(); reverse(p.begin(),p.end());
526     for (const Point &u:p)
527     {
528         while (st.size()>k && check(st,u)) st.pop_back();
529         st.push_back(u);
530     }
531     st.pop_back();
532     return Convex{st};
533 }
534 // 半平面交
535 // 排序增量法, 复杂度  $O(n\log n)$ 
536 // 输入与返回值都是用直线表示的半平面集合
537 vector<Line> halfinter(vector<Line> l, const point_t lim=1e9)
538 {
539     const auto check=[](const Line &a,const Line &b,const Line
540         &c){return a.toleft(b.inter(c))<0;};
541     l.push_back({{-lim,0},{0,-1}}); l.push_back({{0,-lim
542         },{1,0}});
543     l.push_back({{lim,0},{0,1}}); l.push_back({{0,lim
544         },{-1,0}});
545     sort(l.begin(),l.end());
546     deque<Line> q;
547     for (size_t i=0;i<l.size();i++)
548     {
549         if (i>0 && l[i-1].v.toleft(l[i].v)==0 && l[i-1].v*[i
550             ].v>eps) continue;
551         while (q.size()>1 && check(l[i],q.back(),q[q.size()
552             -2])) q.pop_back();
553         while (q.size()>1 && check(l[i],q[0],q[1])) q.
554             pop_front();
555         if (!q.empty() && q.back().v.toleft(l[i].v)<=0) return
556             vector<Line>{};
557         q.push_back(l[i]);
558     }
559     while (q.size()>1 && check(q[0],q.back(),q[q.size()-2])) q
560         .pop_back();
561     while (q.size()>1 && check(q.back(),q[0],q[1])) q.
562         pop_front();
563     return vector<Line>(q.begin(),q.end());
564 }
565 // 点集形成的最小最大三角形
566 // 极角序扫描线, 复杂度  $O(n^2\log n)$ 
567 // 最大三角形问题可以使用凸包与旋转卡壳做到  $O(n^2)$ 
568 pair<point_t,point_t> minmax_triangle(const vector<Point> &vec
569     )
570 {
571     if (vec.size()<=2) return {0,0};
572     vector<pair<int,int>> evt;
573     evt.reserve(vec.size()*vec.size());
574     point_t maxans=0,minans=numeric_limits<point_t>::max();
575     for (size_t i=0;i<vec.size();i++)
576     {
577         for (size_t j=0;j<vec.size();j++)
578         {
579             if (i==j) continue;
580             if (vec[i]==vec[j]) minans=0;
581             else evt.push_back({i,j});
582         }
583     }
584 }

```

```

576 sort(evt.begin(), evt.end(), [&](const pair<int, int> &u,
577 const pair<int, int> &v)
578 {
579     const Point du = vec[u.second] - vec[u.first], dv = vec[v.
580 second] - vec[v.first];
581     return argcmp({{du.y, -du.x}, {dv.y, -dv.x}});
582 });
583 vector<size_t> vx(vec.size()), pos(vec.size());
584 for (size_t i=0; i<vec.size(); i++) vx[i]=i;
585 sort(vx.begin(), vx.end(), [&](int x, int y){return vec[x]<
586 vec[y]});
587 for (size_t i=0; i<vx.size(); i++) pos[vx[i]]=i;
588 for (auto [u, v]: evt)
589 {
590     const size_t i=pos[u], j=pos[v];
591     const size_t l=min(i, j), r=max(i, j);
592     const Point vecu=vec[u], vecv=vec[v];
593     if (l>0) minans=min(minans, abs((vec[vx[l-1]]-vecu)^
594 (vec[vx[l]]-vecv)));
595     if (r<vx.size()-1) minans=min(minans, abs((vec[vx[r
596 +1]]-vecu)^((vec[vx[r]]-vecv)));
597     maxans=max({maxans, abs((vec[vx[0]]-vecu)^((vec[vx[0]]-
598 vecv)), abs((vec[vx.back()-vecu]^((vec[vx.back()-
599 vecv))));
600     if (i<j) swap(vx[i], vx[j]), pos[u]=j, pos[v]=i;
601 }
602 return {minans, maxans};
603 }
604 // 判断多条线段是否有交点
605 // 扫描线, 复杂度  $O(n \log n)$ 
606 bool segs_inter(const vector<Segment> &segs)
607 {
608     if (segs.empty()) return false;
609     using seq_t=tuple<point_t, int, Segment>;
610     const auto seqcmp=[&](const seq_t &u, const seq_t &v)
611     {
612         const auto [u0, u1, u2]=u;
613         const auto [v0, v1, v2]=v;
614         if (abs(u0-v0)<=eps) return make_pair(u1, u2)<make_pair
615 (v1, v2);
616         return u0<v0-eps;
617     };
618     vector<seq_t> seq;
619     for (auto seg: segs)
620     {
621         if (seg.a.x>seg.b.x+eps) swap(seg.a, seg.b);
622         seq.push_back({seg.a.x, 0, seg});
623         seq.push_back({seg.b.x, 1, seg});
624     }
625     sort(seq.begin(), seq.end(), seqcmp);
626     point_t x_now;
627     auto cmp=[&](const Segment &u, const Segment &v)
628     {
629         if (abs(u.a.x-u.b.x)<=eps || abs(v.a.x-v.b.x)<=eps)
630             return u.a.y<v.a.y-eps;
631         return ((x_now-u.a.x)*(u.b.y-u.a.y)+u.a.y*(u.b.x-u.a.x
632 ))*(v.b.x-v.a.x)<((x_now-v.a.x)*(v.b.y-v.a.y)+v.a
633 .y*(v.b.x-v.a.x))*(u.b.x-u.a.x)-eps;
634     };
635     multiset<Segment, decltype(cmp)> s{cmp};
636     for (const auto [x, o, seg]: seq)
637     {
638         x_now=x;
639         const auto it=s.lower_bound(seg);
640         if (o==0)
641         {
642             if (it!=s.end() && seg.is_inter(*it)) return true;
643             if (it!=s.begin() && seg.is_inter(*prev(it)))
644                 return true;
645             s.insert(seg);
646         }
647         else
648         {
649             if (next(it)!=s.end() && it!=s.begin() && (*prev(
650 it)).is_inter(*next(it))) return true;
651             s.erase(it);
652         }
653     }
654     return false;
655 }
656 // 多边形面积并
657 // 轮廓积分, 复杂度  $O(n^2 \log n)$ ,  $n$  为边数

```

```

647 // ans[i] 表示被至少覆盖了 i+1 次的区域的面积
648 vector<long double> area_union(const vector<Polygon> &polys)
649 {
650     const size_t siz=polys.size();
651     vector<vector<pair<Point, Point>>> segs(siz);
652     const auto check=[&](const Point &u, const Segment &e){
653         return !((u<e.a && u<e.b) || (u>e.a && u>e.b));
654     };
655     auto cut_edge=[&](const Segment &e, const size_t i)
656     {
657         const Line le{e.a, e.b-e.a};
658         vector<pair<Point, int>> evt;
659         evt.push_back({e.a, 0}); evt.push_back({e.b, 0});
660         for (size_t j=0; j<polys.size(); j++)
661         {
662             if (i==j) continue;
663             const auto &pj=polys[j];
664             for (size_t k=0; k<pj.p.size(); k++)
665             {
666                 const Segment s={pj.p[k], pj.p[pj.nxt(k)]};
667                 if (le.toleft(s.a)==0 && le.toleft(s.b)==0)
668                 {
669                     evt.push_back({s.a, 0});
670                     evt.push_back({s.b, 0});
671                 }
672                 else if (s.is_inter(le))
673                 {
674                     const Line ls{s.a, s.b-s.a};
675                     const Point u=le.inter(ls);
676                     if (le.toleft(s.a)<0 && le.toleft(s.b)>=0)
677                         evt.push_back({u, -1});
678                     else if (le.toleft(s.a)>=0 && le.toleft(s.
679 b)<0) evt.push_back({u, 1});
680                 }
681             }
682         }
683     };
684     sort(evt.begin(), evt.end());
685     if (e.a>e.b) reverse(evt.begin(), evt.end());
686     int sum=0;
687     for (size_t i=0; i<evt.size(); i++)
688     {
689         sum+=evt[i].second;
690         const Point u=evt[i].first, v=evt[i+1].first;
691         if (! (u==v) && check(u, e) && check(v, e)) segs[sum
692 ].push_back({u, v});
693         if (v==e.b) break;
694     }
695     };
696     for (size_t i=0; i<polys.size(); i++)
697     {
698         const auto &pi=polys[i];
699         for (size_t k=0; k<pi.p.size(); k++)
700         {
701             const Segment ei={pi.p[k], pi.p[pi.nxt(k)]};
702             cut_edge(ei, i);
703         }
704     }
705     vector<long double> ans(siz);
706     for (size_t i=0; i<siz; i++)
707     {
708         long double sum=0;
709         sort(segs[i].begin(), segs[i].end());
710         int cnt=0;
711         for (size_t j=0; j<segs[i].size(); j++)
712         {
713             if (j>0 && segs[i][j]==segs[i][j-1]) segs[i][++cnt
714 ].push_back(segs[i][j]);
715             else cnt=0, sum+=segs[i][j].first^segs[i][j].second
716 ;
717         }
718         ans[i]=sum/2;
719     }
720     return ans;
721 }
722 // 圆面积并
723 // 轮廓积分, 复杂度  $O(n^2 \log n)$ 
724 // ans[i] 表示被至少覆盖了 i+1 次的区域的面积
725 vector<long double> area_union(const vector<Circle> &circs)
726 {
727     const size_t siz=circs.size();
728     using arc_t=tuple<Point, long double, long double, long
729 double>;

```

```

724 vector<vector<arc_t>> arcs(siz);
725 const auto eq=[](const arc_t &u,const arc_t &v)
726 {
727     const auto [u1,u2,u3,u4]=u;
728     const auto [v1,v2,v3,v4]=v;
729     return u1==v1 && abs(u2-v2)<=eps && abs(u3-v3)<=eps &&
        abs(u4-v4)<=eps;
730 };
731
732 auto cut_circ=[](const Circle &ci,const size_t i)
733 {
734     vector<pair<long double,int>> evt;
735     evt.push_back({-PI,0}); evt.push_back({PI,0});
736     int init=0;
737     for (size_t j=0;j<circs.size();j++)
738     {
739         if (i==j) continue;
740         const Circle &cj=circs[j];
741         if (ci.r<cj.r-eps && ci.relation(cj)>=3) init++;
742         const auto inters=ci.inter(cj);
743         if (inters.size()==1) evt.push_back({atan2l((
            inters[0]-ci.c).y,(inters[0]-ci.c).x),0});
744         if (inters.size()==2)
745         {
746             const Point dl=inters[0]-ci.c,dr=inters[1]-ci.
                c;
747             long double argl=atan2l(dl.y,dl.x),argr=atan2l
                (dr.y,dr.x);
748             if (abs(argl+PI)<=eps) argl=PI;
749             if (abs(argr+PI)<=eps) argr=PI;
750             if (argl>argr+eps)
751             {
752                 evt.push_back({argl,1}); evt.push_back({PI
                    ,-1});
753                 evt.push_back({-PI,1}); evt.push_back({
                    argr,-1});
754             }
755             else
756             {
757                 evt.push_back({argl,1});
758                 evt.push_back({argr,-1});
759             }
760         }
761     }
762     sort(evt.begin(),evt.end());
763     int sum=init;
764     for (size_t i=0;i<evt.size();i++)
765     {
766         sum+=evt[i].second;
767         if (abs(evt[i].first-evt[i+1].first)>eps) arcs[sum
            ].push_back({ci.c,ci.r,evt[i].first,evt[i+1].
                first});
768         if (abs(evt[i+1].first-PI)<=eps) break;
769     }
770 };
771
772 const auto oint=[](const arc_t &arc)
773 {
774     const auto [cc,cr,l,r]=arc;
775     if (abs(r-l-PI-PI)<=eps) return 2.0l*PI*cr*cr;
776     return cr*cr*(r-l)+cc.x*cr*(sin(r)-sin(l))-cc.y*cr*(
        cos(r)-cos(l));
777 };
778
779 for (size_t i=0;i<circs.size();i++)
780 {
781     const auto &ci=circs[i];
782     cut_circ(ci,i);
783 }
784 vector<long double> ans(siz);
785 for (size_t i=0;i<siz;i++)
786 {
787     long double sum=0;
788     sort(arcs[i].begin(),arcs[i].end());
789     int cnt=0;
790     for (size_t j=0;j<arcs[i].size();j++)
791     {
792         if (j>0 && eq(arcs[i][j],arcs[i][j-1])) arcs[i++](++
            cnt).push_back(arcs[i][j]);
793         else cnt=0,sum+=oint(arcs[i][j]);
794     }
795     ans[i]=sum/2;
796 }
797 return ans;

```

798 }

8 其它

8.1 矩阵加速

```

1 //1939
2 // #include <iostream>
3 // #include <cstdio>
4 // #include <cstring>
5
6 // using namespace std;
7
8 // typedef Long Long LL;
9 // const int Mod = 1e9+7;
10 // int T, n;
11 // struct mat{
12 //     LL m[5][5];
13 // }Ans, base;
14 // inline void init() {
15 //     memset(Ans.m, 0, sizeof(Ans.m));
16 //     for(int i=1; i<=3; i++) Ans.m[i][i] = 1;
17 //     memset(base.m, 0, sizeof(base.m));
18 //     base.m[1][1] = base.m[1][3] = base.m[2][1] = base.m[3][2]
        = 1;
19 // }
20 // inline mat mul(mat a, mat b) {
21 //     mat res;
22 //     memset(res.m, 0, sizeof(res.m));
23 //     for(int i=1; i<=3; i++) {
24 //         for(int j=1; j<=3; j++) {
25 //             for(int k=1; k<=3; k++) {
26 //                 res.m[i][j] += (a.m[i][k] % Mod) * (b.m[k][j] % Mod)
                ;
27 //                 res.m[i][j] %= Mod;
28 //             }
29 //         }
30 //     }
31 //     return res;
32 // }
33 // inline void Qmat_pow(int p) {
34 //     while (p) {
35 //         if(p & 1) Ans = mul(Ans, base);
36 //         base = mul(base, base);
37 //         p >>= 1;
38 //     }
39 // }
40
41 // int main() {
42 //     scanf("%d", &T);
43 //     while (T--) {
44 //         scanf("%d", &n);
45 //         if(n <= 3) {
46 //             printf("1\n");
47 //             continue;
48 //         }
49 //         init();
50 //         Qmat_pow(n);
51 //         printf("%lld\n", Ans.m[2][1]);
52 //     }
53 // }
54 /*
55 f[i-1]      f[i]
56 f[i-2] ----> f[i-1]
57 f[i-3]      f[i-2]
58 f[i] = f[i-1] * 1 + f[i-2] * 0 + f[i-3] * 1
59 f[i-1] = f[i-1] * 1 + f[i-2] * 0 + f[i-3] * 0
60 f[i-2] = f[i-1] * 0 + f[i-2] * 1 + f[i-3] * 0
61 so
62 1 0 1
63 1 0 0
64 0 1 0
65 */
66 //P6569
67 #include<iostream>
68 #include<cstdio>
69 #include<cstring>
70 #include<algorithm>
71 using namespace std;
72 struct matrix{//用结构体存矩阵敲好使哒~
73     long long qwq[105][105];
74     int x,y;//x,y就是矩阵的行数和列数
75     matrix(){x=0,y=0;memset(qwq,0,sizeof(qwq));};//构造函数，刚

```

```

76 };
77 matrix operator * (const matrix &a,const matrix &b){//重载“异
    或版矩阵乘法”的运算符
78     matrix c;
79     c.x=a.x,c.y=b.y;
80     for(int i=1;i<=a.x;i++){
81         for(int j=1;j<=b.y;j++){
82             for(int k=1;k<=a.y;k++){
83                 c.qwq[i][j]^=a.qwq[i][k]*b.qwq[k][j];//和矩阵乘法别无二
                    致，只不过就是改成异或
84             }
85         }
86     }
87     return c;
88 }
89 long long f[105];//每个城市的初始魔法值（提醒：十年OI一场空
    .....）
90 int main(){
91     int n,m,q;
92     scanf("%d%d%d",&n,&m,&q);
93     for(int i=1;i<=n;i++)
94         scanf("%lld",&f[i]);
95     wyx[0].x=n,wyx[0].y=n;//初始化一开始的邻接矩阵（也就是2的0次
        方）的大小
96     for(int i=1;i<=m;i++){
97         int a,b;
98         scanf("%d%d",&a,&b);
99         wyx[0].qwq[a][b]=wyx[0].qwq[b][a]=1;//给邻接矩阵连上边
100     }
101     for(int i=1;i<=32;i++)wyx[i]=wyx[i-1]*wyx[i-1];//利用快速幂的
        思想，处理出邻接矩阵2的i次方
102     while(q--){
103         long long x;
104         scanf("%lld",&x);
105         matrix ans;
106         for(int i=1;i<=n;i++)ans.qwq[1][i]=f[i];//一开始就是初始f
            值
107         ans.x=1,ans.y=n;
108         for(int i=0;i<=32;i++){//开始进行二进制拆分
109             if((x>>i)&1)//如果这一位是1
110                 ans=ans*wyx[i];//就乘上对应的2的i次方
111         }
112         printf("%lld\n",ans.qwq[1][1]);
113     }
114     return 0;
115 }

```

8.2 并查集

```

1 //推导部分和
2 #include <bits/stdc++.h>
3 using namespace std;
4 typedef long long ll;
5 const ll N=1e5+10;
6 struct tree{
7     ll n,w,to;
8 }e[N<<1];
9 ll head[N<<1],tot,fa[N],vis[N],sum[N];
10 void add(int u,int v,ll w){
11     e[tot].to=v;
12     e[tot].n=head[u];
13     e[tot].w=w;
14     head[u]=tot++;
15 }
16 int find(int i){
17     if(fa[i]!=i) return fa[i]=find(fa[i]);
18     return i;
19 }
20 void unite(int x,int y){
21     x=find(x);
22     y=find(y);
23     if(x!=y) fa[x]=y;
24 }
25 void bfs(int x){
26     vis[x]=1;
27     queue<int>q; q.push(x);
28     sum[x]=0;
29     while(!q.empty()){
30         int u=q.front();
31         q.pop();
32         for(int i=head[u];i!=-1;i=e[i].n){

```

```

33         int v=e[i].to;
34         if(vis[v]) continue;
35         vis[v]=1;
36         sum[v]=sum[u]+e[i].w;
37         q.push(v);
38     }
39 }
40 }
41 int main()
42 {
43     memset(head,-1,sizeof(head));
44     int n,m,q;
45     cin>>n>>m>>q;
46     for(int i=1;i<=n;i++) fa[i]=i;
47     for(int i=1;i<=m;i++){
48         ll l,r,w;
49         cin>>l>>r>>w;
50         add(l-1,r,w);
51         add(r,l-1,-w);
52         unite(l-1,r);
53     }
54     for(int i=0;i<=n;i++){
55         if(!vis[i]) bfs(i);
56     }
57     for(int i=1;i<=m;i++){
58         int l,r;
59         cin>>l>>r;
60         if(find(l-1)!=find(r)) cout<<"UNKNOWN"<<endl;
61         else cout<<sum[r]-sum[l-1]<<endl;
62     }
63     return 0;
64 }

```

8.3 FFT (快速傅里叶变换)

```

1 // 当vector用就可以了
2 #include <bits/stdc++.h>
3 #define fp(i, a, b) for (int i = (a), i##_ = (b) + 1; i < i##_
    ; ++i)
4 #define fd(i, a, b) for (int i = (a), i##_ = (b) - 1; i > i##_
    ; --i)
5 using namespace std;
6 using ll = int64_t;
7 using db = double;
8 /*
    -----
9 */
10 struct cp {
11     db x, y;
12     cp(db real = 0, db imag = 0) : x(real), y(imag) {};
13     cp operator+(cp b) const { return {x + b.x, y + b.y}; }
14     cp operator-(cp b) const { return {x - b.x, y - b.y}; }
15     cp operator*(cp b) const { return {x * b.x - y * b.y, x *
        b.y + y * b.x}; }
16 };
17 using vcp = vector<cp>;
18 using Poly = vector<int>;
19 namespace FFT {
20     const db pi = acos(-1);
21     vcp Omega(int L) {
22         vcp w(L);
23         w[1] = 1;
24         for (int i = 2; i < L; i <= 1) {
25             auto w0 = w.begin() + i / 2, w1 = w.begin() + i;
26             cp wn(cos(pi / i), sin(pi / i));
27             for (int j = 0; j < i; j += 2)
28                 w1[j] = w0[j >> 1], w1[j + 1] = w1[j] * wn;
29         }
30         return w;
31     }
32     auto W = Omega(1 << 21); // NOLINT
33     void DIF(cp *a, int n) {
34         cp x, y;
35         for (int k = n >> 1; k; k >>= 1)
36             for (int i = 0; i < n; i += k << 1)
37                 for (int j = 0; j < k; ++j)
38                     x = a[i + j], y = a[i + j + k],
39                     a[i + j + k] = (a[i + j] - y) * W[k + j], a[i
40                         + j] = x + y;
41     }
42     void IDIT(cp *a, int n) {
43         cp x, y;
44         for (int k = 1; k < n; k <= 1)

```

```

43     for (int i = 0; i < n; i += k << 1)
44         for (int j = 0; j < k; ++j)
45             x = a[i + j], y = a[i + j + k] * W[k + j], a[i
46                 + j + k] = x - y,
47                 a[i + j] = x + y;
48     const db Inv = 1. / n;
49     fp(i, 0, n - 1) a[i].x *= Inv, a[i].y *= Inv;
50     reverse(a + 1, a + n);
51 }
52 // namespace FFT
53 namespace Polynomial {
54 // basic operator
55 void DFT(vcp &a) { FFT::DIF(a.data(), a.size()); }
56 void IDFT(vcp &a) { FFT::IDIT(a.data(), a.size()); }
57 int norm(int n) { return 1 << (lg(n - 1) + 1); }
58 // Poly mul
59 vcp &dot(vcp &a, vcp &b) {
60     fp(i, 0, a.size() - 1) a[i] = a[i] * b[i];
61     return a;
62 }
63 Poly operator+(Poly a, Poly b) {
64     int maxlen = max(a.size(), b.size());
65     Poly ans(maxlen + 1);
66     a.resize(maxlen + 1), b.resize(maxlen + 1);
67     for (int i = 0; i < maxlen; i++) ans[i] = a[i] + b[i];
68     return ans;
69 }
70 Poly operator*(ll k, Poly a) {
71     Poly ans;
72     for (auto i : a) ans.push_back(k * i);
73     return ans;
74 }
75 Poly operator*(Poly a, Poly b) {
76     int n = a.size() + b.size() - 1;
77     vcp c(norm(n));
78     fp(i, 0, a.size() - 1) c[i].x = a[i];
79     fp(i, 0, b.size() - 1) c[i].y = b[i];
80     DFT(c), dot(c, c), IDIT(c), a.resize(n);
81     fp(i, 0, n - 1) a[i] = int(c[i].y * .5 + .5);
82     return a;
83 }
84 }
85 // namespace Polynomial
86 /*
87 -----
88 */
89 using namespace Polynomial;

```

8.4 NTT (数论变换)

```

1 #include <bits/stdc++.h>
2 #define fp(i, a, b) for (int i = (a), i##_ = (b) + 1; i < i##_
3     ; ++i)
4 #define fd(i, a, b) for (int i = (a), i##_ = (b) - 1; i > i##_
5     ; --i)
6 using namespace std;
7 const int maxn = 2e5 + 5, P = 998244353;
8 using ll = int64_t;
9 #define ADD(a, b) (((a) += (b)) >= P ? (a) -= P : 0)
10 #define SUB(a, b) (((a) -= (b)) < 0 ? (a) += P : 0)
11 #define MUL(a, b) (ll(a) * (b) % P)
12 int POW(ll a, int b = P - 2, ll x = 1) {
13     for (; b >= 1, a = a % P; b >>= 1, a = a % P;
14         if (b & 1) x = x * a % P;
15     return x;
16 }
17 namespace NTT {
18     const int g = 3;
19     vector<int> Omega(int L) {
20         int wn = POW(g, P / L);
21         vector<int> w(L);
22         w[L >> 1] = 1;
23         fp(i, L / 2 + 1, L - 1) w[i] = MUL(w[i - 1], wn);
24         fd(i, L / 2 - 1, 1) w[i] = w[i << 1];
25         return w;
26     }
27     auto W = Omega(1 << 21);
28     void DIF(int *a, int n) {
29         for (int k = n >> 1; k; k >>= 1) {
30             for (int i = 0, y; i < n; i += k << 1)
31                 fp(j, 0, k - 1) y = a[i + j + k],

```

```

31                 a[i + j + k] = MUL(a[i + j] - y +
32                     P, W[k + j]),
33                 ADD(a[i + j], y);
34         }
35     }
36     void IDIT(int *a, int n) {
37         for (int k = 1; k < n; k <= 1) {
38             for (int i = 0, x, y; i < n; i += k << 1)
39                 fp(j, 0, k - 1) x = a[i + j], y = MUL(a[i + j + k
40                     ], W[k + j]),
41                 a[i + j + k] = x - y < 0 ? x - y +
42                     P : x - y,
43                 ADD(a[i + j], y);
44         }
45         int Inv = P - (P - 1) / n;
46         fp(i, 0, n - 1) a[i] = MUL(a[i], Inv);
47         reverse(a + 1, a + n);
48     }
49 }
50 // namespace NTT
51 namespace Polynomial {
52 using Poly = std::vector<int>;
53 Poly &operator*=(Poly &a, int b) {
54     for (auto &x : a) x = MUL(x, b);
55     return a;
56 }
57 Poly operator*(Poly a, int b) { return a *= b; }
58 Poly operator*(int a, Poly b) { return b * a; }
59 void DFT(Poly &a) { NTT::DIF(a.data(), a.size()); }
60 void IDFT(Poly &a) { NTT::IDIT(a.data(), a.size()); }
61 int norm(int n) { return 1 << (32 - builtin_clz(n - 1)); }
62 void norm(Poly &a) {
63     if (!a.empty()) a.resize(norm(a.size()), 0);
64 }
65 Poly &dot(Poly &a, Poly &b) {
66     fp(i, 0, a.size() - 1) a[i] = MUL(a[i], b[i]);
67     return a;
68 }
69 Poly operator*(Poly a, Poly b) {
70     int n = a.size() + b.size() - 1, L = norm(n);
71     if (a.size() <= 8 || b.size() <= 8) {
72         Poly c(n);
73         fp(i, 0, a.size() - 1) fp(j, 0, b.size() - 1) c[i + j]
74             =
75             (c[i + j] + (ll)a[i] * b[j]) % P;
76         return c;
77     }
78     a.resize(L), b.resize(L);
79     DFT(a), DFT(b), dot(a, b), IDFT(a);
80     return a.resize(n), a;
81 }
82 }
83 // namespace Polynomial
84 using namespace Polynomial;

```

8.5 扫描线算法

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 ll arr[15010],n,height[15010];
5
6 struct node{
7     ll l,r,h;
8 }a[15010];
9
10 struct compare{
11     bool operator()(pair<ll,ll> a,pair<ll,ll> b){
12         if(a.first!=b.first) return a.first<b.first;
13         return a.second>b.second;
14     }
15     //大根堆
16 }
17 bool cmp(node x,node y){
18     return x.l<y.l;
19 }
20 int main(){
21     ll l,r,h,m=0;
22     while(scanf("%lld %lld %lld",&l,&h,&r)!=EOF){
23         a[++m].l=l;
24         a[m].r=r-1;
25         a[m].h=h;
26         arr[++n]=l;
27         arr[++n]=r;
28         arr[++n]=r-1;

```

```

29     }
30     sort(arr+1, arr+n+1);
31     ll len = unique(arr+1, arr+n+1) - arr - 1;
32     for (int j = 1; j <= m; j++) {
33         a[j].l = lower_bound(arr+1, arr+len+1, a[j].l) - arr;
34         a[j].r = lower_bound(arr+1, arr+len+1, a[j].r) - arr;
35     }
36     sort(a+1, a+m+1, cmp);
37     priority_queue<pair<ll, ll>, vector<pair<ll, ll>>, compare>
        heap;
38     for (int i = 1, j = 0; i <= len; i++) {
39         for (; j <= m && a[j].l <= i; j++) {
40             heap.push(make_pair(a[j].h, a[j].r));
41         }
42         while (!heap.empty() && heap.top().second < i)
43             heap.pop();
44     }
45
46     if (!heap.empty()) {
47         height[i] = heap.top().first;
48     }
49 }
50
51 ll ansn = 0;
52 pair<ll, ll> ans[15010];
53 for (ll i = 1, pre = 0; i <= len; i++) {
54     if (pre != height[i]) {
55         ans[++ansn] = make_pair(arr[i], height[i]);
56     }
57     pre = height[i];
58 }
59 for (int i = 1; i <= ansn; i++) {
60     cout << ans[i].first << " "<< ans[i].second << " ";
61 }
62 }

```

8.6 最长上升子序列

```

1 #include<cstdio>
2 #include<algorithm>
3 const int MAXN=200001;
4 int a[MAXN];
5 int d[MAXN];
6 int main()
7 {
8     int n;
9     scanf("%d",&n);
10    for (int i=1; i<=n; i++)
11        scanf("%d",&a[i]);
12    d[1]=a[1];
13    int len=1;
14    for (int i=2; i<=n; i++)
15    {
16        if (a[i]>d[len])
17            d[++len]=a[i];
18        else
19        {
20            int j=std::lower_bound(d+1, d+len+1, a[i]) - d;
21            d[j]=a[i];
22        }
23    }
24    printf("%d\n", len);
25    return 0;
26 }

```

8.7 全排列输出

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 int main() {
6     std::vector<int> arr = {1, 2, 3}; // 示例数组
7
8     // 先排序（必须步骤，确保生成所有排列）
9     std::sort(arr.begin(), arr.end());
10
11    // 输出所有排列组合
12    do {
13        for (int num : arr) {
14            std::cout << num << " ";
15        }

```

```

16         std::cout << "\n";
17     } while (std::next_permutation(arr.begin(), arr.end()));
18
19     return 0;
20 }

```

8.8 位运算用法

/*为了更好的理解状压dp，首先介绍位运算相关的知识。

'&'符号， $x \& y$ ，会将两个十进制数在二进制下进行与运算（都1为1，其余为0）然后返回其十进制下的值。例如 $3(11) \& 2(10) = 2(10)$ 。

'/'符号， x / y ，会将两个十进制数在二进制下进行或运算（都0为0，其余为1）然后返回其十进制下的值。例如 $3(11) / 2(10) = 3(11)$ 。

'^'符号， $x \wedge y$ ，会将两个十进制数在二进制下进行异或运算（不同为1，其余为0）然后返回其十进制下的值。例如 $3(11) \wedge 2(10) = 1(01)$ 。

'~'符号， $\sim x$ ，按位取反。例如 $\sim 101 = 010$ 。

'<<'符号，左移操作， $x \ll 2$ ，将x在二进制下的每一位向左移动两位，最右边用0填充， $x \ll 2$ 相当于让x乘以4。'>>'符号，是右移操作， $x \gg 1$ 相当于给x/2，去掉x二进制下的最右一位

1. 判断一个数字x二进制下第i位是不是等于1。（最低第1位）

方法：if(((1<<(i-1))&x)>0) 将1左移i-1位，相当于制造了一个只有第i位上是1，其他位上都是0的二进制数。然后与x做与运算，如果结果>0，说明x第i位上是1，反之则是0。
2. 将一个数字x二进制下第i位更改成1。

方法： $x = x | (1 \ll (i-1))$ 证明方法与1类似。
3. 将一个数字x二进制下第i位更改成0。

方法： $x = x \& \sim(1 \ll (i-1))$
4. 把一个数字二进制下最靠右的第一个1去掉。

方法： $x = x \& (x-1)$

__builtin_popcount() 二进制数中一的个数

```

/*
#include<iostream>
using namespace std;
struct state {
    int st[1 << 12+1];
    int num;
} a[15];
int m, n;
int f[15][1 << 12+1];
const int MOD = 100000000;
void getstate(int i, int t) {
    a[i].num = 0;
    for (int j = 0; j < (1 << n); j++) {
        if ((j & (j << 1)) || (j & t))
            continue;
        a[i].st[++a[i].num] = j;
    }
}
void dp() {
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= a[i].num; j++)
            f[i][j] = 0;
    for (int j = 1; j <= a[1].num; j++)
        f[1][j] = 1;
    for (int i = 2; i <= m; i++) {
        for (int j = 1; j <= a[i].num; j++) {
            int s1 = a[i].st[j];
            for (int k = 1; k <= a[i-1].num; k++) {
                int s2 = a[i-1].st[k];
                if (s1 & s2) continue;
                f[i][j] = (f[i][j] + f[i-1][k]) % MOD;
            }
        }
    }
    int ans = 0;
    for (int j = 1; j <= a[m].num; j++)
        ans = (ans + f[m][j]) % MOD;
    cout << ans << endl;
}
int main() {
    cin >> m >> n;
    for (int i = 1; i <= m; i++) {
        int t = 0;
        for (int j = 0; j < n; j++) {

```

```

68         int x;
69         cin >> x;
70         t = (t << 1) + (1 - x);
71     }
72     getstate(i, t);
73 }
74 dp();
75 return 0;
76 }

```

8.9 Map 的使用

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 int main(){
4     //什么是map? map是干什么用的
5     map<int,int>mp; //定义了一个名叫mp的map容器
6     mp[10001]=82;mp[100002]=82;
7     //输出10001信息
8     cout<<mp[10001]<<endl; //可能会溢出
9     //安全的方法
10    if(mp.find(10005)!=mp.end()){
11        cout<<mp[10005]<<endl;
12    }
13
14    else cout<<"没有此人信息"<<endl;
15    //储存学生姓名对应成绩
16    map<string,int>stu;
17    stu["xiaoming"]=85;
18    stu["xiaohong"]=77;
19    cout<<stu["xiaohong"]<<endl;
20    //map的遍历
21    //1.
22    for(auto it:stu){ //将容器从头到尾遍历了一遍
23        //map自动按照首位字典序从小到大排序
24        cout<<it.first<<" "<<it.second<<endl;
25    }
26    //2.迭代器类型
27    map<string,int>::iterator it; //定义map<string,int>类型的迭代器it
28    for(it=stu.begin();it!=stu.end();it++){
29        cout<<(*it).first<<" "<<(*it).second<<endl;
30    }
31 }

```

8.10 优先队列 (堆)

```

1 //自定义类的优先队列
2 struct Student {
3     std::string name;
4     int score;
5     int age;
6
7     //重载 < 运算符,用于大根堆 (按score从大到小)
8     bool operator<(const Student& other) const {
9         //注意: priority_queue默认用 < 比较,但实际表现是最大堆
10        //所以这里返回 true 表示优先级更低
11        return score < other.score; //大根堆
12    }
13 };
14 priority_queue<Student>q1; //大根堆
15 priority_queue<int>q2; //默认大根堆
16 priority_queue<int,vector<int>,greater<int>>q3; //小根堆

```

8.11 扫描线 + 线段树 (矩形面积并)

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 ll arr[15010],n,height[15010];
5
6 struct node{
7     ll l,r,h;
8 }a[15010];
9
10 struct compare{
11     bool operator()(pair<ll,ll> a,pair<ll,ll> b){
12         if(a.first!=b.first) return a.first<b.first;
13         return a.second>b.second;
14         //大根堆

```

```

15     }
16 };
17 bool cmp(node x,node y){
18     return x.l<y.l;
19 }
20 int main(){
21     ll l,r,h,m=0;
22     while(scanf("%lld %lld %lld",&l,&h,&r)!=EOF){
23         a[++m].l=l;
24         a[m].r=r-1;
25         a[m].h=h;
26         arr[++n]=1;
27         arr[++n]=r;
28         arr[++n]=r-1;
29     }
30     sort(arr+1,arr+n+1);
31     ll len=unique(arr+1,arr+n+1)-arr-1;
32     for(int j=1;j<=m;j++){
33         a[j].l=lower_bound(arr+1,arr+len+1,a[j].l)-arr;
34         a[j].r=lower_bound(arr+1,arr+len+1,a[j].r)-arr;
35     }
36     sort(a+1,a+m+1,cmp);
37     priority_queue<pair<ll,ll>, vector<pair<ll,ll>>, compare>
38     heap;
39     for(int i=1,j=0;i<=len;i++){
40         for(;j<=m&&a[j].l<=i;j++){
41             heap.push(make_pair(a[j].h,a[j].r));
42         }
43         while(!heap.empty()&&heap.top().second<i)
44             heap.pop();
45     }
46
47     if(!heap.empty()){
48         height[i]=heap.top().first;
49     }
50 }
51 ll ansn=0;
52 pair<ll,ll>ans[15010];
53 for(ll i=1,pre=0;i<=len;i++){
54     if(pre!=height[i]){
55         ans[++ansn]=make_pair(arr[i],height[i]);
56     }
57     pre=height[i];
58 }
59 for(int i=1;i<=ansn;i++){
60     cout<<ans[i].first<<" "<<ans[i].second<<" ";
61 }
62 }

```

8.12 建树参考

```

1 #include<bits/stdc++.h>
2 //define int long long
3 #define in(a) a = read_int()
4 using namespace std;
5 const int N = 2e5 + 5;
6 const int inf = INT_MAX;
7 inline int read_int() {
8     int x = 0;
9     char ch = getchar();
10    bool f = 0;
11    while('9' < ch || ch < '0') f |= ch == '-', ch = getchar();
12    while('0' <= ch && ch <= '9') x = (x << 3) + (x << 1) + ch - '0', ch = getchar();
13    return f ? -x : x;
14 }
15 struct Edge {
16     int n , v , w;
17 } edge[N << 1];
18 int head[N] , eid;
19 void eadd(int u , int v , int w) {
20     edge[++ eid].n = head[u] , edge[eid].v = v , edge[head[u]] = eid , w = w;
21 }
22 int n;
23 int l[N] , r[N] , ans[N];
24 void dfs(int u , int fa) { // 树形 DP 部分
25     l[u] = 1; // 初始化左边界
26     for(int i = head[u] ; i ; i = edge[i].n) {
27         int v = edge[i].v , w = edge[i].w;
28         if(v != fa) {
29             r[v] = w - 1; // 初始化右边界

```



```

30     dfs(v , u);
31     r[u] = min(r[u] , w - l[v]); // 转移过程
32     l[u] = max(l[u] , w - r[v]);
33 }
34 }
35 }
36 queue<int>q;
37 signed main() {
38     in(n);
39     for(int i = 1 ; i < n ; i ++ ) {
40         int u , v , w;
41         in(u) , in(v) , in(w);
42         eadd(u , v , w) , eadd(v , u , w);
43     }
44     r[1] = 1e9; // 初始化根节点的右边界
45     dfs(1 , 0); // 求每个点权的取值范围
46     for(int i = 1 ; i <= eid ; i += 2) { // 判无解部分
47         int u = edge[i].v , v = edge[i + 1].v , w = edge[i].w;
48         if(r[u] + r[v] < w || l[u] > r[u] || l[v] > r[v]) {
49             cout<<"NO";
50             return 0;
51         }
52     }
53     ans[1] = l[1]; // 若有解，根节点点权取左边界
54     q.push(1);
55     while(!q.empty()) { // 广搜部分
56         int u = q.front();
57         q.pop();
58         for(int i = head[u] ; i ; i = edge[i].n) {
59             int v = edge[i].v , w = edge[i].w;
60             if(ans[v]) continue;
61             ans[v] = w - ans[u]; // 递推求解
62             q.push(v);
63         }
64     }
65     cout<<"YES\n"; // 打印答案
66     for(int i = 1 ; i <= n ; i ++ ) cout<<ans[i]<<' ';
67     return 0;
68 }

```

8.13 Hash 树

```

1 //树同构
2
3 #include <bits/stdc++.h>
4
5 typedef long long LL;
6 const int N = 60, MO = 998244353;
7
8 struct Edge {
9     int nex, v;
10 }edge[N << 1]; int tp;
11
12 int e[N], n, m, turn, fr[N], p[1000010], top, siz[N], h[N], g[N];
13 std::vector<int> v[N];
14 bool vis[1000010];
15
16 inline void add(int x, int y) {
17     tp++;
18     edge[tp].v = y;
19     edge[tp].nex = e[x];
20     e[x] = tp;
21     return;
22 }
23
24 inline bool equal(int a, int b) {
25     int len = v[a].size();
26     if(len != v[b].size()) return false;
27     for(int i = 0; i < len; i++) {
28         if(v[a][i] != v[b][i]) return false;
29     }
30     return true;
31 }
32
33 inline void getp(int n) {
34     for(int i = 2; i <= n; i++) {
35         if(!vis[i]) {
36             p[++top] = i;
37         }
38         for(int j = 1; j <= top && i * p[j] <= n; j++) {
39             vis[i * p[j]] = 1;
40             if(i % p[j] == 0) {

```

```

41                 break;
42             }
43         }
44     }
45     return;
46 }
47
48 void DFS_1(int x, int f) {
49     siz[x] = 1;
50     h[x] = 1;
51     for(int i = e[x]; i; i = edge[i].nex) {
52         int y = edge[i].v;
53         if(y == f) continue;
54         DFS_1(y, x);
55         h[x] = (h[x] + 1ll * h[y] * p[siz[y]] % MO) % MO;
56         siz[x] += siz[y];
57     }
58     return;
59 }
60
61 void DFS_2(int x, int f, int V) {
62     g[x] = (h[x] + 1ll * V * p[n - siz[x]] % MO) % MO;
63     v[turn].push_back(g[x]);
64     V = (1ll * V * p[n - siz[x]] % MO + 1) % MO;
65     for(int i = e[x]; i; i = edge[i].nex) {
66         int y = edge[i].v;
67         if(y == f) {
68             continue;
69         }
70         DFS_2(y, x, ((LL)V + h[x] - 1 - 1ll * h[y] * p[siz[y]] % MO + MO) % MO);
71     }
72     return;
73 }
74
75 int main() {
76     getp(1000009);
77     scanf("%d", &m);
78     for(turn = 1; turn <= m; turn++) {
79         scanf("%d", &n);
80         tp = 0;
81         memset(e + 1, 0, n * sizeof(int));
82         for(int i = 1, x; i <= n; i++) {
83             scanf("%d", &x);
84             if(x) {
85                 add(x, i);
86                 add(i, x);
87             }
88         }
89         DFS_1(1, 0);
90         DFS_2(1, 0, 0);
91         std::sort(v[turn].begin(), v[turn].end());
92     }
93
94     for(int i = 1; i <= m; i++) {
95         fr[i] = i;
96     }
97     for(int i = 2; i <= m; i++) {
98         for(int j = 1; j < i; j++) {
99             if(equal(i, j)) {
100                 fr[i] = fr[j];
101                 break;
102             }
103         }
104     }
105     for(int i = 1; i <= m; i++) {
106         printf("%d\n", fr[i]);
107     }
108     return 0;
109 }

```